









ETH Department of Computer Science (D-INFK)
Distributed Computing Group
in collaboration with
MIT Media Lab
Responsive Environments Group

Master's Thesis

Tonic: Energy-Based Transformers for Symbolic Music Generation and Single- and Multi-Attribute Control

A Systematic Comparison with Autoregressive Transformers and Alternative Steering Methods

Rebecca Ceccatelli

Supervisor, ETH Zürich:	Prof. Dr. Roger Wattenhofer
Supervisor, MIT Media Lab:	Prof. Joseph A. Paradiso
Co-Advisor, MIT Media Lab:	Lancelot Blanchard

November 2026

Acknowledgements

This thesis was carried out under the supervision of Professor Roger Wattenhofer, at the Distributed Computing Group of the Department of Computer Science at ETH Zürich. I am sincerely grateful for his support in making it possible to carry out this work abroad.

The work itself took shape over six months as a visiting student researcher in the Responsive Environments group at the MIT Media Lab, led by Professor Joseph A. Paradiso, whom I warmly thank for welcoming me into the group and for making this a precious life experience.

My gratitude goes to Lancelot Blanchard, a doctoral student in the Responsive Environments group, who was my main point of reference throughout this project. I appreciated his patience, his availability, and the care he brought to advising me.

I would also like to thank Alexi Gladstone, a doctoral student at the University of Illinois Urbana-Champaign and first author of the original paper on Energy-Based Transformers, for his feedback on the feasibility of my idea and for the insights he shared on how these models work.

Many thanks to everyone who took part in the listening survey. Your time and answers were invaluable!

This research stay would not have been possible without generous financial support from Switzerland, a country that I have come to consider home. I gratefully acknowledge the scholarship from ETH's Swiss-European Mobility Programme, as well as the Traugott und Josefine Niederberger-Kobold Foundation — with special thanks to Bruno Giger — and the Hasler Foundation, whose generosity made this collaboration a reality.

Abstract

Generative models of music have advanced rapidly and can now produce music that sounds convincing and realistic. The control they offer over what they generate, however, remains limited. Most models accept only coarse conditions, such as genre, mood or instrumentation, which are set once and for all during training. Methods that steer a trained model at generation time do exist, but they can only nudge the model from the outside, since it commits to each note immediately and cannot go back. Finer-grained requests are rarely supported, and the models behave like black boxes whose choices are difficult to interpret or correct.

Energy-Based Transformers (EBTs), a new class of models introduced in 2025, offer a different approach. An EBT begins with a rough guess for each note and improves it over several steps, lowering a learned energy that scores how well the guess fits the music so far. A precise musical request, such as softer, longer or higher notes, can enter this energy as an extra term with its own target and strength, set at generation time and without retraining. EBTs are therefore a natural candidate for music generation that is easier to steer and to inspect.

This thesis brings EBTs to music. We introduce Tonic, to our knowledge the first EBT for music, trained on the GigaMIDI corpus in the REMI and Anticipation tokenizations. Tonic models music about as well as autoregressive Transformers trained with the same recipe, and listeners found its REMI continuations as musical as the human originals. On top of the model, and as a step towards generation that is both controllable and interpretable, we add control of one or multiple attributes at once through small regressors on its frozen embeddings: on REMI, note velocity moves in the requested direction in 95% of the samples, and two attributes together in 75%.

We compare Tonic systematically with two standard autoregressive models, GPT-2 and Llama, trained under comparable settings, along three dimensions: language modelling performance, musical quality and attribute-control accuracy. We also implement and evaluate three alternative steering methods at inference time on these models: best-of-N, PPLM and expectation tilting. To our knowledge, the last two had not been used before to steer low-level musical attributes, and both prove effective; Tonic's control is as accurate as the best of them, with limited side effects on musicality. We further analyse why guidance succeeds in some cases and not in others, and complement these measurements with a blind listening study.

We also contribute an open-source extension of the `anticipation` library that tokenizes GigaMIDI and a statistical analysis of the corpus for each controlled attribute, and we release all our code and an outreach website with an interactive demo, a showcase and a listening archive of over 20,000 clips.

Tonic's training and control are currently more computationally expensive than those of autoregressive models, and its control requires more careful tuning. Moving from the simple System 1 recipe we used to the System 2 recipe, which lets EBTs "think" longer, and training at a larger scale could make control more robust and turn the extra computation into better results. We believe this is an exciting route towards a new paradigm in which AI tools become an integral part of musicians' creative process, supporting them as they experiment with ideas and refine their music iteratively.

A Note on Ethics, from the Author

This work uses machine learning models to generate music, but it is not meant to replace musicians and artists; it aims to give them better tools.

I am a musician, and many of my friends make their living from music. I know how much effort and passion go into it.

Most models that generate music today work like a black box: they return a result, but leave little say in how it turns out. This thesis investigates how to give the person using a model more control over what it produces, for instance softer or louder playing, longer or shorter notes, or a higher or lower register, and a clearer view of how the model responds.

Decades ago, musicians took up synthesizers and electronic instruments and made them their own. I see this work as a small step in the same direction: models as instruments for shaping one's own ideas, and perhaps one day for playing with them live.

All models in this thesis are trained on GigaMIDI [36], a public collection of MIDI files. Access requires agreeing to use it only for non-commercial research or education, which is how it is used here. No music was collected for this work, and the models generate symbolic scores, not recordings of anyone's performance.

Rebecca

Contents

Acknowledgements i

Abstract ii

A Note on Ethics, from the Author iii

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Research Questions . . . 3
- 1.3 Approach and Main Findings . . . 3
- 1.4 Contributions . . . 4
- 1.5 Thesis Outline . . . 7

2 Background and Related Work 9
- 2.1 Audio and Symbolic Music . . . 9
- 2.2 Representing Symbolic Music as Tokens . . . 10
  - 2.2.1 REMI . . . 10
  - 2.2.2 Anticipation (Arrival-Time) . . . 11
- 2.3 The State of the Art: Autoregressive Transformers . . . 11
  - 2.3.1 Autoregressive Language Models . . . 11
  - 2.3.2 From Recurrent Networks to Music Language Models . . . 12
  - 2.3.3 What AR Models Lack for Control . . . 12
- 2.4 Energy-Based Models and Energy-Based Transformers . . . 13
  - 2.4.1 Energy-Based Models . . . 13
  - 2.4.2 Composing Energies . . . 14
  - 2.4.3 Energy-Based Transformers . . . 15
- 2.5 Controllable Music Generation . . . 18
  - 2.5.1 Training-Time Conditioning . . . 18
  - 2.5.2 Latent-Space Control . . . 19
  - 2.5.3 Inference-Time (Plug-and-Play) Control . . . 19
  - 2.5.4 Rule-Based Control . . . 22
  - 2.5.5 The Cost of Strong Guidance . . . 22
- 2.6 Evaluating Generated Music . . . 22
- 2.7 Positioning of This Work . . . 23

3 Method 24
- 3.1 Energy-Based Transformers for Symbolic Music . . . 24
- 3.2 Attribute Regressors on Frozen Embeddings . . . 25
- 3.3 Attribute Guidance in the Refinement Loop . . . 27
- 3.4 Combined-Attribute Control . . . 28
- 3.5 Guidance Baselines for Autoregressive Models . . . 29
- 3.6 Computational Cost . . . 30

4 Experimental Setup **32**
4.1 Data . . . 32
4.1.1 Tokenizing GigaMIDI with Anticipation . . . 32
4.1.2 Attribute Distributions in the Corpus . . . 33
4.2 Models and Training . . . 35
4.2.1 Architectures and Optimisation . . . 35
4.2.2 Training Tonic . . . 36
4.2.3 Training the Baselines . . . 37
4.2.4 Training Cost and Checkpoints . . . 38
4.2.5 Attribute Regressors . . . 39
4.3 Attribute Guidance Protocol . . . 40
4.4 Evaluation Metrics . . . 43
4.4.1 Language Modelling . . . 43
4.4.2 Musical Quality . . . 43
4.4.3 Control . . . 45
4.4.4 Compute . . . 45
4.5 Statistical Analysis . . . 46
4.6 Listening Study . . . 46
4.7 Approaches Tried and Not Adopted . . . 48
4.8 Implementation and Reproducibility . . . 49

5 Results **51**
5.1 Language Modelling Performance . . . 51
5.2 Unguided Music Quality . . . 54
5.3 Attribute Control with REMI . . . 55
5.4 Attribute Control with Anticipation . . . 58
5.5 Combined Control . . . 58
5.6 Control Strength versus Musicality . . . 59
5.7 Listening Study . . . 62

6 Discussion **65**
6.1 Interpretation of Results . . . 65
6.1.1 How Well Tonic Models Music . . . 66
6.1.2 Control in the Refinement Loop . . . 66
6.1.3 Why Strong Guidance Degrades the Music . . . 67
6.1.4 Why One Direction Is Easier . . . 68
6.1.5 Control with Anticipation . . . 68
6.1.6 Thinking Longer . . . 69
6.1.7 Measured and Heard Change . . . 69
6.2 Limitations . . . 70

7 Conclusion **72**
7.1 Answers to the Research Questions . . . 72
7.2 Further Insights . . . 73
7.3 Future Work . . . 74
7.4 Closing Remarks . . . 77

A Anticipation Tokenization Pipeline for GigaMIDI **78**
A.1 GigaMIDI Driver 78
A.2 Resumable Tokenization on a Shared Cluster 78
A.3 Packaging 79

B Additional Results **80**
B.1 Language Modelling 80
B.2 Guidance 80

C Listening Study: Detailed Results **84**

D The Tonic Website **88**
D.1 Front Page 88
D.2 Interactive Demo 89
D.3 Showcase 92
D.4 Listening Archive 94
D.5 Tonic, Simply Explained 95
D.6 Listening Survey 96
D.7 Survey Results 97

Bibliography **98**

Chapter 1

Introduction

Music generation with machine learning has advanced quickly in the last few years [38]. Audio models such as Jukebox [11], MusicLM [2] and MusicGen [6] produce minutes of realistic-sounding music from a text description. Symbolic models, which generate notes and their timing rather than sound, are trained on corpora of more than a million MIDI files [36] and can continue, harmonise or fill in a musical passage [67, 71]. Most of these systems, in audio and in symbols, are large Transformers that generate one token at a time.

As the quality of these models has improved, the open question is less whether they can produce convincing music and more *how precisely a musician can shape what they produce*. A composer sketching an idea, a producer filling a gap in an arrangement or a performer improvising with a machine rarely wants an arbitrary continuation. They want one that is softer, higher or more sustained than what the model would produce on its own, by a chosen amount, while still sounding like music. Today's models mostly offer coarse, global conditions such as genre or mood; what is missing is *fine-grained control over individual musical attributes*.

In this thesis, we study how to steer a pretrained model of symbolic music towards a requested value of a musical attribute while it generates, without retraining the model. We do this with *Energy-Based Transformers* (EBTs) [19], a recent type of model, introduced in 2025, that does not predict the next token in a single step but refines its prediction by minimising a learned energy. Each attribute request can be added to that energy as a separate term, so the model's own refinement loop does the steering.

We call our model Tonic: an EBT for symbolic music, together with the attribute control that acts inside its refinement loop. To our knowledge, it is the first EBT applied to music. Throughout the thesis, Tonic refers to our trained models and "EBT" to the model type in general.

The name comes from music theory: the *tonic* of a key is the "home" where harmonic tension resolves, much as an EBT's prediction settles into an energy minimum.

1.1 Motivation

The problem. We want a musician to be able to ask a generative model for specific musical properties, such as louder, longer or higher notes, and to set, combine and weight these requests freely at generation time. We work with symbolic music (MIDI) rather than audio, because every attribute we control is written in the output, so we can check exactly whether a request was met; with audio, it would

have to be estimated by another model, with errors of its own (Section 2.1).

How control is done today. The state of the art in symbolic generation is dominated by *autoregressive* (AR) Transformers trained on tokenized MIDI, from the Music Transformer [26] and the Pop Music Transformer [27] to the Anticipatory Music Transformer [67] and language-model-scale systems such as MIDI-LLM [71]. These models predict one token at a time and commit to each choice immediately.

Most ways of controlling them work at *training time*: attributes are added to the input as conditioning tokens or latent codes, and the model is trained to follow them [72, 39, 31]. This works well, but adding a new attribute means retraining or fine-tuning the model itself, and attributes such as mood or genre also need labels collected by hand. The set of controls is fixed once the model is trained, and many systems offer coarse or global ones, such as an emotion class, the instruments to use or a note density per bar, rather than continuous targets on individual attributes.

The alternative is control at *inference time*: leave the trained model frozen and steer its generations with a separate, cheap component, such as a classifier, a regressor or a direction in activation space. This *plug-and-play* family comes from language modelling, with PPLM [8], and has recently reached symbolic music through activation steering [52, 51]. It is *much lighter to set up* than training-time control: the base model is never retrained, and a new attribute only needs a small *regressor*, a model that predicts a numerical value (here, the attribute) from the music, whose labels come directly from the MIDI (Section 2.5.3). The cost moves to generation time instead, where guidance adds computation to every generated token.

Why steering AR models is limited. For AR models, every inference-time method has to work around the fact that the model *never revisits a decision*. Guidance can only nudge the next-token distribution, rerank whole samples, or perturb hidden states: it is added from outside a model that was never designed to take it.

Why Energy-Based Transformers. Energy-Based Transformers (EBTs) [19] offer a different structure. An EBT does not output a prediction directly. It assigns an energy to each candidate prediction and finds the prediction by *iteratively minimising* that energy, starting from noise. Energy-based models compose naturally: adding a second energy term changes what the minimisation converges to [14]. An attribute requirement can therefore enter the model's own inference loop as an extra energy, rather than being bolted on from outside.

This is the intuition the thesis is built on. Because an EBT reaches its output by minimising an explicit energy step by step, its behaviour is more open to inspection than a single forward pass, and each musical requirement can be written as its own energy term, with its own target value and weight. We therefore expect EBTs to be more interpretable and more naturally controllable *attribute by attribute*, giving *finer-grained control* than current symbolic models offer. EBTs have been studied for text and images, but not, to our knowledge, for music. This thesis tests whether this expectation holds. In short, the control works as well as the best methods

for autoregressive models, with room to improve in cost and in the range of usable strengths (Section 1.3).

1.2 Research Questions

The research questions fall into two groups. The first is whether Energy-Based Transformers can generate music *at all*, and how well compared with the models used today (RQ1 and RQ2). This was not a given: EBTs had not been applied to music, and a music vocabulary is large and highly structured, so a single wrong token can break the bar structure or the note grammar of a sequence. The second is whether the EBT's inference loop is a good place to put musical control, and how it compares with established inference-time methods (RQ3 and RQ4).

RQ1 Can an Energy-Based Transformer be trained on symbolic music and generate coherent, musically plausible continuations?

RQ2 How does it compare with autoregressive Transformers of the same size, trained on the same data with the same recipe, in likelihood and musical quality?

RQ3 Can a frozen EBT be steered towards target values of musical attributes, singly and in combination, by adding attribute energies to its inference loop, without retraining the model?

RQ4 How does this compare with plug-and-play steering of AR models under the same constraints, in control accuracy, musical quality and compute?

1.3 Approach and Main Findings

TONIC consists of our Energy-Based Transformers for symbolic music, one per tokenization, and the attribute control that acts inside their refinement loop.

Data. GigaMIDI [36] is a corpus of over a million MIDI files. We train on its multi-instrument part only, about 171,000 files, in two tokenizations: REMI [27, 17] and the arrival-time encoding of the Anticipatory Music Transformer [67]. For the latter, we implement and release an open-source extension of the `anticipation` library that tokenizes GigaMIDI. We also analyse how the attributes we control are distributed across the corpus; these distributions set the scale of our guidance targets.

Models and control. We pretrain TONIC in both tokenizations, as EBTs trained with the stable System 1 recipe of Gladstone et al. [19], and GPT-2 [54] and Llama [68] baselines with matched size, data, context length and training steps. For control, small regressors trained on the frozen TONIC embeddings predict note velocity, note duration and pitch register. Their error to a target value becomes an *attribute*

energy, whose gradient enters TONIC's own minimisation steps with a strength λ. Each requested attribute adds its own energy, so several attributes can be controlled at once. On the AR baselines we implement three plug-and-play methods for comparison: PPLM-style logit gradients, expectation tilting and best-of-N.

Evaluation. We evaluate the models from several complementary angles:

- *Standard machine-learning metrics:* training and validation loss curves, compared at matched context and at matched numbers of training tokens, and the effect of extra computation at inference.
- *Musicality metrics:* MIDI-level statistics of the generated music, such as pitch-class entropy, scale consistency, dissonance and groove consistency, compared with those of real music [74].
- *Control metrics:* accuracy, progress towards and error to the target, together with the cost in musical quality and in compute, with operating points tuned on held-out prompts and paired significance tests.
- *Human evaluation:* a blind listening study, supported by a listening showcase and an interactive demo.

Main findings. An EBT models symbolic music about as well as AR Transformers trained with the same recipe, and its refinement loop is a workable place for control: TONIC can be steered without retraining, on one attribute or on two at once, about as accurately as the best plug-and-play methods for AR models, and gently on the music. Steering an AR model's output with a gradient step reaches similar accuracy at a lower cost, and TONIC works best within a narrow range of strengths and with the REMI encoding. We trace this to one mechanism: guidance acts while an EBT's prediction is still being formed, so strong guidance *flattens* the model's choice among the tokens that carry the attribute, and the shape of the corpus then decides which direction of control is easy. TONIC also gains nothing from extra computation at inference, which suggests that the advantages promised by energy-based models need a different training recipe. The answers to the four research questions, with the evidence, are given in Section 7.1.

1.4 Contributions

This thesis makes the following contributions. For each, we point to the part of the thesis where it is developed.

1. **An Anticipation tokenization pipeline for GigaMIDI, and an analysis of the corpus.** We release an open-source extension of the `anticipation` library [67], building on the JAM_BOT fork [4], that tokenizes GigaMIDI in the arrival-time encoding. It adds a GigaMIDI driver for all three splits, resumable tokenization for shared clusters with job time limits, a vocabulary variant without the anticipation control block for purely autoregressive models,

and an installable package. MIDI-LLM [71] also trains on GigaMIDI with this encoding, but does not release its data pipeline.

We use the pipeline to train arrival-time models from scratch, and we characterise how the attributes we control are distributed across the corpus. These distributions set the scale of our guidance targets (in standard deviations of the corpus) and help explain why steering is easier in one direction than the other (Sections 4.1, 4.1.1, 4.1.2 and 6.1.4 and Appendix A).

2. **Energy-Based Transformers for symbolic music.** To our knowledge, this is the first application of EBTs to music. We train TONIC, our EBTs, on GigaMIDI in two tokenizations, REMI and arrival-time Anticipation, and identify what it takes to train them stably on a large, structured music vocabulary. We train them with the System 1 recipe of Gladstone et al. [19] and test whether they benefit from extra computation at inference, through more refinement steps or by choosing among several candidates. We compare them with GPT-2 and Llama models of matched size, trained on the same data with the same recipe (context length, batch size, training steps and learning-rate schedule), so that differences reflect the model family rather than the training setup (Sections 3.1, 4.2, 5.1 and 5.2).

3. **Plug-and-play attribute control inside the EBT's inference loop.** We steer a frozen EBT by adding the gradient of an attribute energy to its own energy-minimisation steps, with a single strength parameter λ. The attribute energies come from small regressors (about 0.2 million parameters each) trained on the frozen model's embeddings, for note velocity, note duration and pitch register. Their labels are computed directly from the MIDI, so neither the model nor its training data needs attribute annotations, and a new attribute can be added without retraining the model (Sections 3.2, 3.3, 5.3 and 5.4).

4. **Combined-attribute control.** Several attributes can be requested in the same generation, each with its own target and strength, by giving each its own energy term. We study joint control of two attributes in each tokenization, reusing the regressors trained for single attributes, with no change to the model:

   - REMI: velocity and duration;
   - Anticipation: duration and pitch register.

   We measure how much the two controls interfere. Combined control works as well as PPLM in both tokenizations and as tilting on REMI, and far better than best-of-N.

   Scope. Because guidance only acts on the tokens that can change an attribute, the two attributes of each pair are steered on different tokens of the same note, one after the other: no single refinement step has to satisfy both energies at once. Our experiments therefore show that attribute controls can be stacked in one generation, but not yet that attribute energies compose within a single

step, which would need attributes carried by the same token; preliminary runs with such attributes are reported in Section 4.7 (Sections 3.4 and 6.2 and Chapter 5).

5. **A controlled benchmark of inference-time control for symbolic music.** We compare the guidance of TONIC with three plug-and-play methods transferred from language modelling and applied to the AR baselines: PPLM-style logit gradients, expectation tilting of the next-token distribution, and best-of-N selection. All methods share the same constraint (a frozen model), the same prompts, targets and metrics. To our knowledge, this is the first time PPLM has been used to steer low-level musical attributes towards continuous target values, that expectation tilting of the next-token distribution has been evaluated on symbolic music, and that these plug-and-play methods have been compared with each other under one protocol (Section 2.5.3). The benchmark covers:

   - *language modelling:* validation loss, compared at matched context and at matched numbers of training tokens, and the effect of extra refinement steps;
   - *musicality:* MIDI-level metrics, such as pitch-class entropy, scale consistency, dissonance and groove consistency, compared with real music [74], and the side effects of guidance on the token grammar and on the continuity with the prompt;
   - *control:* whether the attribute moves in the requested direction (directional accuracy), how far it moves towards the target, and how far from the target it ends. A tiny move in the right direction already counts as a success for directional accuracy, so on its own it can make control look better than it is. We also measure how accurately the regressors predict each attribute;
   - *compute:* cost in forward-pass equivalents per token, including compute-matched comparisons with best-of-N (Sections 3.5, 4.3 and 4.4 and Chapter 5).

6. **An analysis of how guidance fails.** We characterise the trade-off between guidance strength and musicality, and show that the guidance of TONIC has a narrow usable range of strengths for some attributes. We identify the mechanism behind the failure at high strength: because TONIC's guidance acts while the prediction is still being formed, strong guidance *flattens* the model's distribution over attribute tokens, pushing the attribute towards the value a random token would give. This explains the asymmetric control, the cacophony at high strength and occasional violations of the token grammar, and, together with the corpus analysis of contribution 1, predicts in which direction control saturates. It differs from the probability concentration reported for guided discrete diffusion (Sections 5.6, 6.1.3 and 6.1.4).

7. **A demo, a showcase, a public listening archive and a listening study.** To make the results audible as well as measurable, we release the following.

All of it is collected on the thesis web page, `https://rebeccaceccatelli.github.io/tonic/`, and described with screenshots in Appendix D.

- an *interactive demo*, in which a user chooses a prompt, the attributes to control, their targets and a guidance strength, and listens to the guided continuation. It runs on a GPU from the code repository[1];
- a *showcase*, a non-interactive public page[2] with curated examples from all systems, side by side;
- a *listening archive*, a public site with all the outputs of our evaluation sweeps (over 20,000 clips): unguided and guided continuations from every system, tokenization, attribute, target and strength, so that any result in this thesis can be listened to[3];
- a *blind listening survey*, in English[4] and Italian[5], in which human listeners compare the systems. Its results are reported in Chapter 5, and summarised in plain terms on the thesis web page[6];
- the *code* of all experiments, the demo and the sites[7], and our extension of the `anticipation` library[8] (contribution 1).

1.5 Thesis Outline

The rest of the thesis presents these contributions as follows.

- Chapter 2 introduces symbolic music representations, autoregressive and energy-based models, and the literature on controllable music generation and how generated music is evaluated, and places our work among existing control methods.
- Chapter 3 describes TONIC (contribution 2), attribute control in its inference loop, singly and combined (contributions 3 and 4), the AR guidance baselines and how we count the compute of each method.
- Chapter 4 covers the data, including our extension of the `anticipation` library and the corpus analysis (contribution 1), the training setup, the evaluation protocol of the benchmark (contribution 5), the design of the listening study (contribution 7), and the approaches we tried and did not adopt.
- Chapter 5 reports the results of the benchmark: language modelling, musical quality, single and combined control, the trade-off between control strength and musicality, and the listening study (contributions 2 to 5 and 7).

[1] `https://github.com/RebeccaCeccatelli/music_EBT`, script `demo/launch_demo.sh`.
[2] `https://rebeccaceccatelli.github.io/music_EBT/`
[3] `https://rebeccaceccatelli.github.io/listening-archive/`
[4] `https://rebeccaceccatelli.github.io/listening-study/survey/`
[5] `https://rebeccaceccatelli.github.io/listening-study/survey/it/`
[6] `https://rebeccaceccatelli.github.io/tonic/survey-results/`
[7] `https://github.com/RebeccaCeccatelli/music_EBT`
[8] `https://github.com/RebeccaCeccatelli/anticipation-musicEBT`

- Chapter 6 interprets these results: how well TONIC models music, how control in the refinement loop compares with the AR methods, why strong guidance degrades the music (contribution 6) and why one direction is easier, which follows from the corpus (contribution 1), control with Anticipation, thinking longer, and how the measured change relates to what listeners hear; it then discusses limitations.
- Chapter 7 answers the research questions, summarises further insights and outlines future work.
- The appendices describe the Anticipation tokenization pipeline, give additional results, report the listening study in detail and present the Tonic website (demo, showcase, archive, survey and code).

Chapter 2

Background and Related Work

This chapter introduces the concepts the thesis builds on and positions it in the literature. It moves from the general to the specific. We first contrast audio and symbolic music generation (Section 2.1) and review how symbolic music is turned into tokens, including the two representations used in this work (Section 2.2). We then survey the current state of the art in symbolic generation, which is dominated by autoregressive transformers (Section 2.3). Section 2.4 introduces energy-based models, their compositional properties and Energy-Based Transformers. Finally, Section 2.5 reviews controllable music generation, with a focus on the difference between training-time and inference-time control. This difference motivates the approach of this thesis.

2.1 Audio and Symbolic Music

Music can be generated in two broad forms. *Audio* models generate the sound itself. Early systems such as Jukebox [11] modelled compressed waveforms directly. Current systems such as MusicLM [2] and MusicGen [6] generate the discrete codes of a neural audio codec [9] and decode them to a waveform. Audio models capture timbre, performance and production, and can be prompted with text. They are, however, expensive to train and run, and their output is a finished recording: it cannot easily be edited note by note, and its musical content (which notes, how loud, how long) can only be recovered with a transcription model.

Symbolic models instead generate a description of the music, typically in MIDI: a list of note events, each with an onset, pitch, duration, velocity (loudness) and instrument [38]. Symbolic generation separates *what* is played from *how it sounds*. The output is compact, editable in any notation program or digital audio workstation, and can be rendered with any instrument. It also fits the way many musicians work, by writing and arranging notes rather than recording finished audio. Symbolic music has two limitations: it leaves out timbre and production, and it relies on MIDI data, which is far less abundant than audio. Large corpora have nonetheless become available, from the Lakh MIDI Dataset of about 176,000 files [55] to GigaMIDI, with over 1.4 million files [36], a subset of which this thesis uses.

For this thesis, the decisive property of symbolic music is that its attributes can be *measured exactly*. The average velocity, note length or pitch register of a generated passage can be read off the representation, so whether a generation meets a request can be checked without a perception model. This makes the symbolic domain a natural setting for studying *control*, and it is the domain of everything that follows.

Table 2.1: The two tokenizations used in this thesis.

	REMI	Anticipation (arrival-time)
Time	bar + position on a beat grid	absolute onset, 10 ms steps
Note	pitch, velocity, duration tokens	onset, duration, instrument–pitch
Velocity	yes	no
Metrical grid	explicit	none
Token roles	partly fixed grammar	fixed by position in triplet
Designed for	metrical structure	infilling with anticipated controls

2.2 Representing Symbolic Music as Tokens

Transformer language models operate on sequences of discrete tokens, so MIDI must first be serialised. Unlike text, a piece of music has several simultaneous voices and several attributes per note, and there is no single natural way to flatten it into one sequence. The choice of tokenization determines the sequence length, the vocabulary size, which musical regularities are explicit, and, as this thesis shows, what can be controlled. Libraries such as MidiTok [17] implement the common schemes behind one interface.

Most schemes belong to one of four families:

- **MIDI-like (time-shift) encodings** translate MIDI messages almost directly into `Note-On`, `Note-Off`, `Velocity` and `Time-Shift` tokens, as introduced for expressive piano performance by Oore et al. [43] and used by the Music Transformer [26]. They preserve expressive timing, but metre is implicit and must be inferred by the model.

- **Beat-based encodings** such as REMI [27] place events on an explicit metrical grid of bars and positions (Section 2.2.1).

- **Compound encodings** group the attributes of a note into a single multi-part token, which shortens sequences considerably. Examples are the Compound Word Transformer [25] and the multitrack encoding of the Multitrack Music Transformer [12]. They require models with several output heads.

- **Absolute-time encodings** such as the arrival-time encoding of the Anticipatory Music Transformer [67] write each note's absolute onset rather than a time shift (Section 2.2.2). This allows events to be reordered without losing their timing.

We use one beat-based and one absolute-time representation. Their design differences, summarised in Table 2.1, turn out to matter for both modelling and control.

2.2.1 REMI

REMI (*revamped MIDI-derived events*) [27] was designed to make the metrical structure of music explicit. Instead of the time-shift events of earlier MIDI-like encodings,

it marks every bar with a `Bar` token and places every event on a grid within the bar with a `Position` token. Each note is then written as a short sequence of tokens: pitch, velocity and duration. The multi-track variant we use also includes `Program` tokens that identify the instrument. Because the metrical grid is explicit, the model can readily learn beat and bar structure, and the token types follow a partly fixed grammar (for example, a pitch token is always followed by a velocity token). Both properties are used by the guidance methods of Chapter 3.

2.2.2 Anticipation (Arrival-Time)

The Anticipatory Music Transformer [67] models music as a temporal point process. Each note is encoded as a triplet of tokens: arrival time (absolute onset), duration, and a note token that jointly encodes instrument and pitch. Times are quantised at 10 ms, which preserves expressive timing but makes the vocabulary very large (55,028 tokens in the variant we use). The representation was designed for *anticipation*: control events (for example, a fixed melody) are interleaved into the event sequence shortly before the time they occur, so that a left-to-right model can condition on future constraints and perform infilling tasks such as generating an accompaniment. A mode token at the start of each sequence indicates whether it contains anticipated controls. Unlike REMI, this representation has no bar grid and no velocity. Its triplet structure is fixed by position, so the role of the next token is always known.

The reference implementation, the `anticipation` library, processes the Lakh MIDI Dataset; Section 4.1.1 describes how we extended it to GigaMIDI.

These design choices make Anticipation much more expensive to train on than REMI, for two reasons. First, its vocabulary is about two orders of magnitude larger. The input embedding and the output layer grow with it, and so does the cost of every prediction: an AR model computes a softmax over the whole vocabulary at every position, and an EBT refines a full distribution over it with gradient steps (Section 2.4.3). Second, anticipatory training multiplies the data: each song is tokenized ten times, once plainly and nine times with a different random choice of notes moved ahead as controls, which teaches the model to fill in music around notes given in advance. The training set is therefore an order of magnitude larger than the corpus it comes from. Sections 4.1.1 and 4.2 describe what this meant for our data and training.

2.3 The State of the Art: Autoregressive Transformers

2.3.1 Autoregressive Language Models

An autoregressive (AR) model factorises the probability of a token sequence as $p(x) = \prod_t p(x_t \mid x_{<t})$ and is trained by maximum likelihood to predict each token from its prefix. A decoder-only transformer computes the next-token distribution in a single forward pass, and generation proceeds by sampling one token at a time,

typically with a temperature and nucleus (top-p) truncation. We use two standard transformer architectures as baselines: GPT-2 [54] and Llama [68]. The latter replaces GPT-2's learned position embeddings, layer normalisation and GELU feed-forward layers with rotary position embeddings, RMS normalisation and gated (SwiGLU) feed-forward layers, a recipe sometimes referred to as Transformer++ [19].

2.3.2 From Recurrent Networks to Music Language Models

Early neural models. Neural symbolic generation has followed the broader trends of deep learning. Recurrent models such as Performance RNN [43] generated expressive piano performances event by event, and MusicVAE [57] learned latent spaces of short phrases with a hierarchical decoder for longer-term structure. The Music Transformer [26] showed that self-attention with relative position information can generate minute-long piano performances with coherent structure.

Transformer language models. Since then, transformer language models over event tokens have been the dominant approach. The Pop Music Transformer [27] introduced REMI, one of the two encodings used in this thesis. The Anticipatory Music Transformer [67] introduced the other, adding infilling and accompaniment through anticipated controls, and is the basis of real-time systems such as the JAM_BOT [4]. Closest to our setting, MIDI-GPT [45] and MIDI-LLM [71] train on GigaMIDI, the latter by adapting Llama 3.2 to arrival-time MIDI tokens.

Non-autoregressive alternatives. The main alternatives are diffusion models, applied to continuous latent representations of symbolic music [40], in a cascade from song form down to notes [70], and to compressed phrase-level representations [44]. AR transformers nevertheless remain the standard for symbolic generation, especially for continuation and interactive use, and they are the baselines of this thesis.

Why control matters. The motivation for control in this thesis comes from interactive use. Systems such as the JAM_BOT, developed at the MIT Media Lab to improvise live with a keyboard virtuoso [4], and ReaLJam [61] put transformer music models in real-time dialogue with musicians. In such settings, a musician needs to steer *what kind* of music the model plays (denser, softer, higher) at the moment it is needed, without retraining the model for every new kind of request. It has also been argued that data-driven music generators inherit reductive assumptions from their training data and representations, and do not cover the full space of musical possibilities [41]. Giving users direct handles on the output is one way to counter this.

2.3.3 What AR Models Lack for Control

In an AR model, the computation spent on each token is fixed. There is no point during generation at which the model "reconsiders" a prediction, and no explicit quantity that measures how well a candidate fits the context. Control therefore has

to be imposed from the outside: by conditioning the model during training, or by modifying its distribution or activations at inference (Section 2.5). Both routes have a cost. The first fixes the available controls before training. The second must act on a model that has already committed to each token, and has no internal notion of how far a modification moves it away from plausible music.

2.4 Energy-Based Models and Energy-Based Transformers

2.4.1 Energy-Based Models

Energy-based models are among the oldest ideas in neural networks. They come from statistical physics, where a system tends towards states of low energy. In 1982, the Hopfield network stored memories as minima of an energy function and recalled them by descending it [24], and in 1985 the Boltzmann machine turned the energy into a probability distribution that could be learned from data [1]. Training such models was long hampered by the cost of sampling from them, which contrastive divergence [20] made practical. LeCun et al. [35] later framed energy-based learning as a general approach, in which a model only has to rank good configurations below bad ones. Since around 2019, EBMs have been scaled up with deep networks for image generation [13] and connected to diffusion models and to compositional generation [14], the line of work this thesis builds on.

The idea. An energy-based model (EBM) does not generate an output directly. It *scores* one: a network E_θ assigns a single number, the *energy* $E_\theta(x)$, to every possible configuration x, such as an image or a sequence of tokens. Plausible configurations get low energy and implausible ones high energy. The energy defines a probability distribution

$$p_\theta(x) = \frac{\exp\big(-E_\theta(x)\big)}{Z(\theta)}, \qquad Z(\theta) = \sum_{x'} \exp\big(-E_\theta(x')\big), \tag{2.1}$$

in which lower energy means higher probability [14].

The normalising constant $Z(\theta)$ sums over every possible configuration and is far too expensive to compute, so EBMs leave it out and work with the energy alone. This gives great freedom in designing E_θ, since any network that outputs a number will do, but it comes at a price. Likelihoods cannot be computed exactly, and producing an output means *searching* for a configuration of low energy rather than reading it off in one step. This search is usually iterative: a common choice is Langevin dynamics, a Markov chain Monte Carlo (MCMC) method that repeatedly moves a candidate along the negative energy gradient, with some added noise. Energy-Based Transformers (Section 2.4.3) use the same idea for next-token prediction.

Energy as a lens on modern architectures. Several widely used architectures can be read as minimising an energy. Modern Hopfield networks store patterns as

minima of an energy function, and their update rule is equivalent to transformer attention [56]. The Energy Transformer builds a transformer whose layers explicitly minimise an engineered energy [22]. Diffusion models closely resemble associative memories, which denoise by descending an energy landscape [23]; seen this way, the transition from memorising the training data to generalising appears as the emergence of new low-energy states [49].

EBMs for sequences and music. For discrete sequences, energy-based diffusion language models use a sequence-level EBM to correct the approximations made by discrete diffusion, approaching the perplexity of AR models [73]. In music, MusicAOG [53] defines an EBM over a hand-designed hierarchical graph of musical structure and uses it for fine-grained, interpretable control, and CGRNN [37] combines a recurrent network with an EBM to capture how musical elements relate over time. Our work also uses an energy for control, but the energy is learned end to end by a transformer over standard music tokens, rather than defined over a hand-crafted representation or built on a recurrent model.

2.4.2 Composing Energies

Because an energy is just a number assigned to each configuration, energies from different models can be *added*. Adding two energies corresponds to multiplying the two distributions they define, a combination known as a *product of experts* [20]:

$$p_1(x)\,p_2(x) \;\propto\; \exp\big(-[E_1(x) + E_2(x)]\big). \tag{2.2}$$

A configuration has low total energy only if it has low energy under *both* models, that is, only if it satisfies both. Two models trained separately, each on its own concept, can therefore be combined at inference time into a model of their conjunction, with no retraining. A weight in front of each energy sets how much each concept counts.

Du et al. [14] develop this idea for diffusion models. They show that guidance and conditioning are forms of composition: classifier guidance, for example, adds $\lambda \nabla_x \log p(y \mid x)$ to the model's own gradient, which steers an unconditional model towards a condition y, and guidance scales larger than one often work best in practice. They also find that composition often fails because of the *sampler*, not the models: plain reverse diffusion does not sample from the composed distribution, whereas MCMC samplers approximate it much better. The same principle has been used to compose text-conditioned human motions [77] and to solve larger reasoning problems by summing energies learned on small subproblems [42].

This thesis carries the idea to symbolic music. The energy of a music model, which scores how well a continuation fits the music so far, is added to the energy of an attribute regressor, which scores how close the continuation is to a requested attribute value. Minimising the sum produces music that is both plausible and close to the request. Because each attribute contributes its own term, attributes can be added, weighted and combined independently: this is the attribute-by-attribute control motivated in Section 1.1.

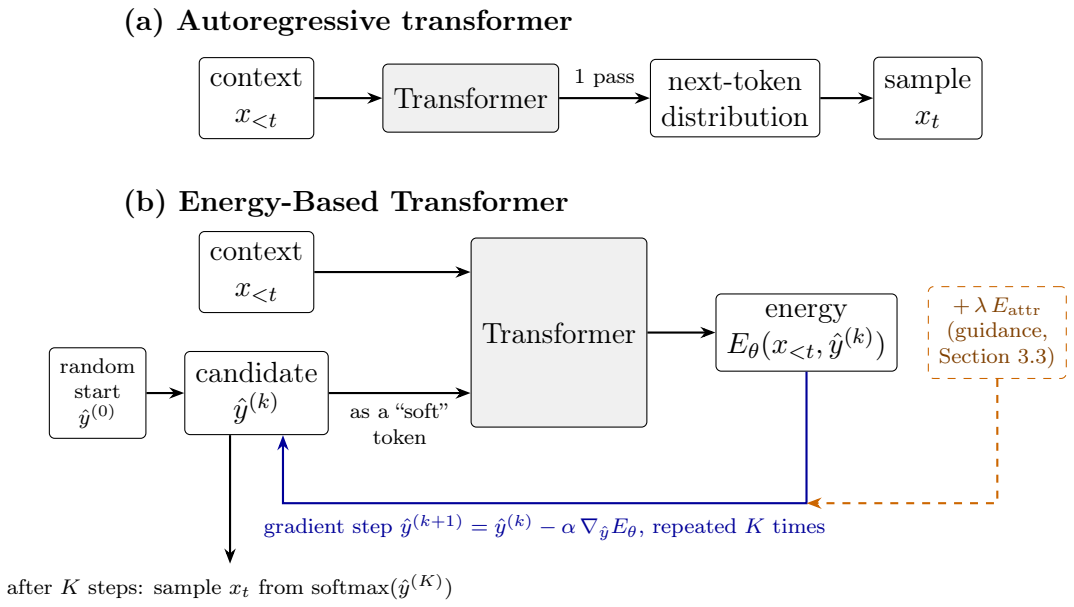


Figure 2.1: How an AR transformer and an EBT predict the next token. (a) The AR model maps the context to a next-token distribution in one forward pass. (b) The EBT scores a candidate prediction with an energy and improves it by gradient descent on that energy, starting from a random candidate. The candidate is a continuous vector over the vocabulary, given to the transformer as a probability-weighted average of token embeddings. Because the prediction is the result of an optimisation, further energy terms, such as an attribute energy (dashed), can enter the same loop.

2.4.3 Energy-Based Transformers

Energy-Based Transformers (EBTs), introduced by Gladstone et al. [19] in 2025, are a much more recent architecture that brings this old idea to the transformers used in today's language and vision models. An EBT is a transformer trained as an EBM for prediction. An ordinary transformer *generates* the next token: it maps the context to a distribution over tokens. An EBT instead acts as a learned *verifier*: given the context and a *candidate* prediction, it outputs a single number, the energy, that measures how well the candidate fits the context. The prediction is then found by searching for the candidate with the lowest energy. Gladstone et al. [19] present this as a form of "System 2 thinking" that emerges from unsupervised training alone, and point to three properties that ordinary transformers lack: (i) the computation spent on a prediction can vary, through more or fewer search steps; (ii) the energy expresses how confident the model is in a prediction; and (iii) a prediction can be checked by its energy, so that several candidates can be generated and the best one kept.

How an EBT makes a prediction. Figure 2.1 contrasts the two models. An AR transformer reads the context $x_{<t}$ and outputs the next-token distribution in one forward pass. An EBT reads the context *and* a candidate $\hat{y}$ for the next token,

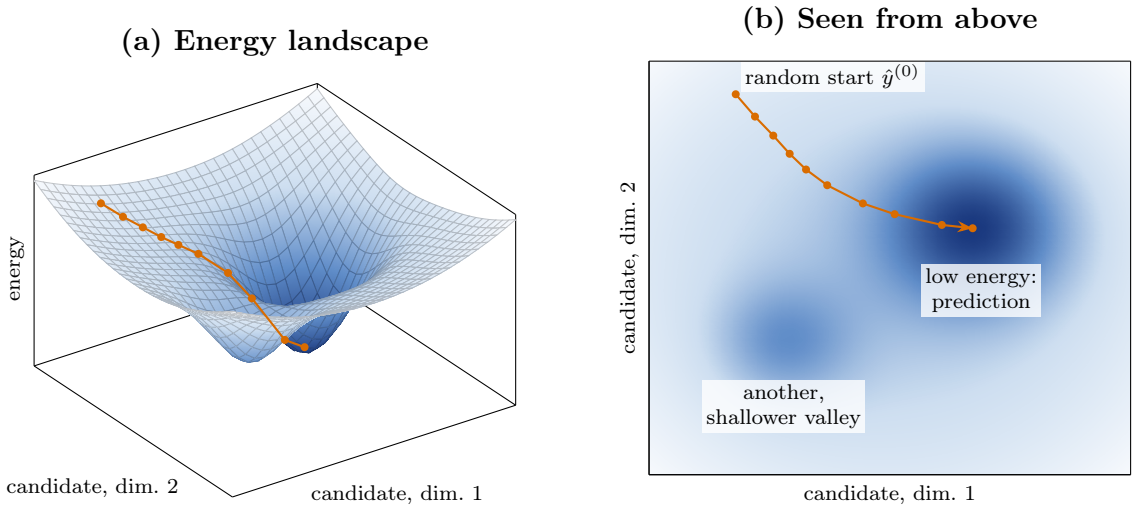


Figure 2.2: How an EBT finds a prediction, as descent into a valley of its energy (schematic). The energy $E_\theta(x_{<t}, \hat{y})$ assigns a value to every candidate prediction $\hat{y}$ for a fixed context; candidates that fit the context lie in valleys of low energy. Starting from a random candidate, each gradient step of Eq. (2.3) (orange dots) moves the candidate downhill, until it settles in a valley, which gives the prediction. (a) The landscape in 3-D; (b) the same landscape from above, with darker colours for lower energy. The real candidate is a vector of logits over the whole vocabulary (427 dimensions for REMI, 55,028 for Anticipation), shown here in two dimensions. The path shows many steps for clarity; TONIC uses $K = 2$ (Section 3.1).

and outputs an energy $E_\theta(x_{<t}, \hat{y})$. Because tokens are discrete, the candidate is not a single token but a vector of scores (logits), one per token of the vocabulary, so that it can be changed gradually. The transformer reads it as a "soft" token: the average of all token embeddings, weighted by the probabilities the candidate assigns to them.

The search for a low-energy candidate is gradient descent. Prediction starts from a random candidate $\hat{y}^{(0)}$ and repeatedly moves it downhill on the energy:

$$\hat{y}^{(k+1)} = \hat{y}^{(k)} - \alpha \nabla_{\hat{y}} E_\theta\big(x_{<t}, \hat{y}^{(k)}\big), \qquad k = 0, \ldots, K-1, \tag{2.3}$$

with a learned step size α. After K steps, the softmax of the refined candidate is a distribution over tokens, from which the next token is sampled as for an AR model. Figure 2.2 pictures this as a landscape: for a given context, the energy is a surface over all possible candidates, with valleys where the candidates fit the context. Each step moves the candidate further downhill, and the prediction is the point where it settles. Following the EBT literature, we call this refinement loop "MCMC", although without added noise it is plain gradient descent.

The cost of a prediction. Each refinement step needs the gradient of the energy with respect to the candidate, that is, a forward and a backward pass through the transformer. An EBT with K steps therefore costs several forward passes per

generated token, where an AR model needs one. This is the price of "thinking longer" about each prediction, and it carries over to guided generation (Section 3.6).

How an EBT is trained. Classical EBMs are trained *contrastively*: the energy of real data is pushed down, and the energy of "negative" samples drawn from the model by MCMC is pushed up. Drawing those negatives is slow and often unstable [14]. EBTs avoid this by training the search itself [19]. During training, the refinement loop of Eq. (2.3) is run for a few steps; each refined candidate is compared with the true next token through the usual cross-entropy loss; and the loss, averaged over the steps, is back-propagated through the refinement. The model thus learns an energy landscape in which descending from a random start leads to the right token. Because each refinement step already contains a gradient of the energy, training needs gradients of gradients (second-order gradients), which makes an EBT more memory-intensive to train than an AR model of the same size.

Gladstone et al. [19] describe two recipes. *System 1* EBTs are tuned for stable training: a fixed number of steps, a learned step size, the prediction detached from the computation graph between steps, and no regularisation; most of their scaling experiments use them. *System 2* EBTs add regularisers that make the learned landscape smoother and let the model benefit from more steps at inference: Langevin-style noise on each step, randomised step sizes and numbers of steps, and a replay buffer of refined predictions that simulates longer optimisation trajectories [19, App. C.2]. TONIC uses the System 1 recipe (Section 3.1).

Variants and scaling. EBTs come in a decoder-only (autoregressive) variant, like GPT, and a bidirectional variant; this thesis uses the former. In the decoder-only variant, all positions of a sequence are refined in parallel during training, with causal masking as in GPT. Across text and images, Gladstone et al. [19] report that EBTs scale up to 35% faster than the Transformer++ recipe with respect to data, batch size, parameters, compute and depth, and gain more from additional computation at inference, especially on out-of-distribution data.

EBTs and diffusion models. Diffusion models also produce their output by iterative refinement, and they too have been guided towards musical attributes without retraining [50, 28]. They refine a whole segment at once, however. An EBT keeps the left-to-right decoding of an AR model: like an AR model, it commits to each token once it is sampled and never revisits it, and what it adds is an optimisation *before* each commitment. It can therefore continue a prompt token by token, as interactive use requires (Section 2.3), and it can be compared directly with AR models of the same size, trained on the same data with the same recipe.

Why this matters for control. An AR transformer produces each prediction in a single pass, so control can only act from outside: on its output distribution or on its internal activations. An EBT produces each prediction by *minimising an energy*, so a control objective can be added to that energy and minimised *together* with it,

Table 2.2: Families of control for music generation. "New attribute" is what it takes to add a control that was not planned for.

Family	What is trained	New attribute	Examples
Training-time conditioning	the whole generative model, with attribute labels as input	new labels, retraining or fine-tuning	[31, 72, 60, 39, 45]
Latent-space control	an encoder–decoder with a regularised latent space	retraining the encoder–decoder	[57, 76, 47, 66]
Inference-time (plug-and-play)	nothing, or a small scorer on a frozen model	a new small scorer or steering direction; the base model is untouched	[8, 30, 16, 28, 52, 51], this thesis
Rule-based	nothing (rules written by hand)	new rules	[64, 18]

as in the composition of Section 2.4.2. The refinement loop then descends

$$E_\theta(x_{<t}, \hat{y}) + \lambda\, E_{\text{attr}}(\hat{y}), \tag{2.4}$$

where E_{attr} is low when the candidate has the requested attribute value and λ sets the strength of the request (Section 3.3). To our knowledge, this work is the first to apply EBTs to music and to use their inference-time optimisation for controllable music generation.

2.5 Controllable Music Generation

Composers want generative systems that can be steered towards their intent [39]. Approaches to control differ mainly in *when* control enters the system: during training, through a structured latent space learned during training, or at inference on a frozen model; outside machine learning, rules can also be written by hand. Most existing work in symbolic music falls in the first two families. They differ sharply in what has to be trained and when a new attribute can be added (Table 2.2).

2.5.1 Training-Time Conditioning

The most direct approach is to train the model with the controls as input. These range from rhythm density and pitch register at every time step of an LSTM [31] and bar-level rhythmic intensity and polyphony in a transformer VAE [72], to bar-level descriptions of instruments, chords, note density and mean pitch in FIGARO [60]. Composer's Assistant 2 [39] and MIDI-GPT [45] build wide sets of such controls, including note density, pitch range and polyphony, into composition tools, and the Anticipatory Music Transformer [67] builds infilling control into its training data.

These methods give precise and reliable control, and they are the most widely adopted, but the set of controls is fixed when the model is trained. Every attribute must be decided before training, labelled throughout the training data (automatically from the MIDI for low-level attributes, by hand for mood or genre), and learned by the full model. Adding a new attribute, or a finer-grained version of an existing one, means new data preparation and a new training or fine-tuning run of a large model. Many systems also expose controls that are coarse or global, such as an emotion class [29], the instruments to use [60, 45], or a density or polyphony level per bar [72, 45], rather than continuous targets on individual musical attributes.

2.5.2 Latent-Space Control

A second family controls the music through a learned, compressed representation of it. A variational autoencoder (VAE) encodes a musical phrase into a short vector, its *latent code*, and a decoder turns the code back into music. If the latent space is organised so that an attribute corresponds to a known direction, moving the code along that direction changes the attribute in the decoded music. Methods differ in how they organise the space:

- *Attribute vectors.* MusicVAE [57] adds attribute vectors to the latent code of a phrase, for arithmetic on musical phrases.
- *Separate factors.* EC^2-VAE [76] encodes pitch and rhythm in separate parts of the code, conditioned on chords, so that one can be changed while the other is kept (musical analogies).
- *One dimension per attribute.* Pati and Lerch [46, 47] regularise single latent dimensions to vary monotonically with an attribute, so that each acts as a slider. Music FaderNets [66] use such "faders" for rhythm and note density to control a high-level attribute, arousal.

These ideas have been built into interactive tools [3] and compared across architectures and datasets [5]. They have two limits. A well-separated latent space is of little use if the decoder does not respond to it reliably: disentanglement does not imply controllability [48]. And the encoder–decoder has to be trained around the attributes from the start, so latent-space control shares the main cost of training-time conditioning.

2.5.3 Inference-Time (Plug-and-Play) Control

The third family, to which this thesis belongs, leaves a pretrained model untouched and steers it during generation. It originates in language modelling and has reached music only recently. Its methods differ in *where* they act: on the model's hidden states or activations, on its next-token distribution, or on whole candidates generated by an unchanged model.

Origins in language modelling. The Plug and Play Language Model (PPLM) [8] is the reference method of the family. A small attribute classifier is trained on the hidden states of a frozen language model. At every generation step, the gradient of the classifier's log-probability for the desired attribute is used to nudge the model's cached hidden states (its keys and values for the past positions) for a few iterations, and the next-token distribution is recomputed from the nudged states. To keep the text fluent, a KL penalty keeps the new distribution close to the unguided one, and the two are mixed before sampling. The model's weights never change: only its activations during generation do. FUDGE [75] and GeDi [34] instead reweight the next-token distribution with a lightweight discriminator. Activation addition [69] adds a fixed steering vector to the model's activations, without any optimisation. The simplest member of the family, generating several candidates and keeping the one a scorer prefers (best-of-N or reranking), needs no access to the model's internals at all, at the cost of generating many candidates. For diffusion models, classifier guidance [10] adds the gradient of a classifier's log-likelihood to the model's score, the compositional view taken by Du et al. [14] (Section 2.4.2).

Classifiers and regressors. The scorers in these methods are mostly *classifiers*, which predict one of a fixed set of classes, such as a topic or a sentiment. A *regressor* instead predicts a continuous value. When the target is a value on a scale rather than a class, such as a mean velocity of 0.7 rather than "loud", a regressor is the natural scorer: its output can be compared directly with the target, and the distance between them can be minimised. This thesis uses regressors throughout.

Exponential tilting. A distribution can also be reweighted with no scorer at all, when the attribute value of each outcome is known. If a distribution q must be changed so that the expected value of an attribute v reaches a target, the change that departs least from q, in KL divergence, is an *exponential tilt*,

$$q_\theta(x) \;\propto\; q(x)\, e^{\theta v(x)}, \tag{2.5}$$

with θ set so that the constraint holds [7]. Khalifa, Elsahar, and Dymetman [32] use this principle for controlled text generation: they impose expected-value constraints on the distribution of whole sequences of a language model, and the resulting target distribution is itself an energy-based model. Applied to a single next-token distribution, over tokens whose attribute value can be read off directly, the same tilt gives a cheap per-step control, which we call *expectation tilting* and use as a baseline (Section 3.5).

Plug-and-play control in music. These ideas have reached music only recently, in three main forms.

Search, reranking and guided sampling. Most plug-and-play work in music searches over, or corrects, the outputs of an unchanged model. Ferreira et al. [16] steer a frozen symbolic language model towards a target emotion with Monte Carlo tree search guided by an emotion classifier. Dual-level beam search [15] reranks

token- and bar-level candidates of a transformer to follow a target tonal-tension curve, and Text2midi-InferAlign [59] searches over continuations of a text-to-MIDI model with rewards for text and key consistency. For symbolic diffusion models, fine-grained guidance (FGG) [79] corrects the sampling process to enforce harmonic and rhythmic constraints, and SCHmUBERT [50] steers a discrete diffusion model towards a note density with classifier guidance. Closest to the low-level, continuous targets studied here, Stochastic Control Guidance [28] steers a pretrained diffusion model towards a target pitch histogram, note density or chord progression, using only forward evaluations of the rule.

Activation steering. The second form changes what the model computes rather than what it outputs. While it generates, a transformer represents the music in its internal activations. If a direction in that space tracks an attribute, adding a multiple of it at every step pushes the music towards more of that attribute. Methods differ in how they find the direction and how strongly they apply it. Directions come from classifier probes on attention heads, in the audio model of SMITIN [33]; from sparse autoencoders that expose interpretable musical concepts in MusicGen [63]; or from latent directions for harmonic resolutions in the Aria model [62]. Closest to this thesis, Prokopiou et al. [52] take the difference between the mean activations of high- and low-valued examples on the Multitrack Music Transformer, for average pitch and average duration, two of the attributes we study, and orthogonalise the two directions to steer both at once. The strength matters as much as the direction: with a fixed strength, the model's own prior leaves a persistent gap to the target, which Prokopiou et al. [51] close by adapting the strength at every step with a PID feedback controller. Amadeus [65] also reports training-free control of note attributes.

PPLM. PPLM itself has been applied to symbolic music: Imasato et al. [30] steer a GPT-2 model of piano music towards one of four perceived-emotion classes, a categorical attribute, with a discriminator on its hidden states, and select among candidates generated with different PPLM settings.

To our knowledge, PPLM has not yet been used to steer low-level musical attributes towards continuous target values, expectation tilting of the next-token distribution has not been evaluated on symbolic music, and these plug-and-play methods have not been compared with each other under one protocol. This thesis does all three: it implements PPLM, expectation tilting and best-of-N as baselines for Tonic (Section 3.5) and compares them under the same prompts, targets and metrics.

Why inference-time control is lighter. Compared with training-time conditioning, inference-time control moves the cost of a new attribute from the generative model to a small auxiliary one. The large model is pretrained once, without any attribute labels, and never modified. Adding an attribute means training a scorer (in this thesis, a regressor of about 0.2 million parameters) on the frozen model's features. For symbolic music the labels can be computed directly from the MIDI, so no human annotation is required. Attributes can then be added, removed, weighted and combined at generation time, which allows finer and more flexible control than

a fixed set of training-time conditions. The price is paid at generation time: every guided token costs extra computation, and because the base model was never trained to follow the control, pushing it too hard can degrade the music. Activation-steering work reports exactly this trade-off and has to tune or adaptively control the steering strength to avoid it [33, 51].

2.5.4 Rule-Based Control

Outside machine learning, constraint programming offers full control by construction, for example to generate four-voice harmonisations that follow the rules of tonal music [64] or to arrange samples under structural constraints [18]. Such systems are transparent and editable but need the musical rules to be written by hand.

2.5.5 The Cost of Strong Guidance

Every inference-time method has a strength parameter, and pushing it too far degrades the output. In diffusion models, classifier guidance and classifier-free guidance trade sample diversity for fidelity to the condition as the guidance scale grows [10, 21]. For masked discrete diffusion, Rojas et al. [58] show that strong guidance *concentrates* probability mass on a single mode of the distribution, and that strong guidance early in sampling harms quality. Music steering work reports the same trade-off. SMITIN has to limit its intervention strength because too much makes the music incoherent [33], guiding note density in SCHmUBERT costs some sample quality [50], emotion guidance of a symbolic diffusion model trades guidance scale against quality [78], and Prokopiou et al. [52] track how tonal and rhythmic consistency drop as steering strength grows, which motivates the adaptive strength of Prokopiou et al. [51]. Less is known about *how* a token-level model's output distribution changes under strong guidance, and in which direction a controlled attribute drifts when control saturates. We return to this question for EBT guidance in Section 5.6, where we observe the opposite of concentration.

2.6 Evaluating Generated Music

Evaluating generative models of music is hard: there is no single correct continuation, and musical quality is partly subjective [74]. Three kinds of evaluation are common, and controlled generation adds a fourth. This thesis uses all four (Section 4.4).

Likelihood. Language models are compared by their loss or perplexity on held-out data. This is cheap and objective, but it is only comparable between models with the same tokenization and training objective, and a lower loss does not guarantee better-sounding music.

Objective musical metrics. A second family computes musically meaningful statistics of the generated MIDI, such as pitch-class entropy, scale consistency or

groove consistency, and compares their distribution with that of real music. Yang and Lerch [74] propose comparing the two distributions through the overlapping area of their kernel density estimates, which we adopt. Such metrics are reproducible and interpretable, but they only capture what they measure, and simple non-musical outputs can score well on them [50].

Listening tests. Human judgement remains the reference for musical quality. Listening tests ask participants to rate or compare samples, ideally without knowing which system produced them. They are costly, noisy and hard to compare across studies, so they usually complement objective metrics rather than replace them [74].

Measuring control. For controlled generation, one must also measure whether the request was met. Steering work often reports whether the attribute moved in the requested direction, for example as "steering success" [52]. A directional measure ignores how far the attribute moved relative to the target: a method that nudges the attribute slightly in the right direction every time scores as well as one that reaches the target. We therefore also report progress towards the target (Section 4.4.3), together with the musical cost of control and its compute.

2.7 Positioning of This Work

Inference-time methods are attractive because a single pretrained model can serve any attribute for which a scorer exists, without retraining. In an AR transformer, however, they must act from the outside, by perturbing logits or activations, reranking samples, or tilting the output distribution, and they cannot easily balance the control objective against the model's own judgement of what is plausible. An EBT offers a different hook: since each prediction is the minimum of an energy, the control objective can be added to that energy, and the two are traded off by the same optimisation that produces the prediction [14, 19]. Each attribute contributes its own energy term with its own target and weight, which gives continuous, attribute-by-attribute control, and several attributes can be requested in the same generation.

This thesis studies whether this makes EBTs easier to steer than AR transformers. It uses attributes (velocity, note duration and pitch register) of the kind that both earlier conditioning work [31] and recent steering work [52] have targeted, and it compares EBT guidance with the plug-and-play methods above under the same prompts, targets and metrics.

Chapter 3

Method

This chapter describes the components of our approach: how an Energy-Based Transformer (EBT) generates symbolic music (Section 3.1), how small attribute regressors are trained on its frozen embeddings (Section 3.2), and how their gradients steer the EBT's inference-time optimisation towards a target attribute value (Section 3.3), also for several attributes at once (Section 3.4). Section 3.5 describes the plug-and-play guidance methods we use for the autoregressive baselines, and Section 3.6 compares the computational cost of all methods.

A defining property of every method in this chapter is that the generative model itself is *frozen*: control is added at inference time, without retraining or conditioning the model on attribute labels. Figure 3.1 summarises the resulting pipeline, TONIC, in three stages.

3.1 Energy-Based Transformers for Symbolic Music

We use the decoder-only EBT described in Section 2.4.3. For each position, the prediction $\hat{y}_t \in \mathbb{R}^V$ is a vector of logits over the vocabulary of size V, initialised from random noise, $\hat{y}_t^{(0)} \sim \mathcal{N}(0, I)$, and refined by gradient steps on the energy $E_\theta(x_{<t}, \hat{y}_t)$ as in Eq. (2.3). The candidate is given to the transformer as the probability-weighted average of the token embeddings, $\text{softmax}(\hat{y}_t)^\top W$, where $W \in \mathbb{R}^{V \times d}$ is the embedding table. The real context embeddings and the predicted embeddings are processed jointly, and the transformer outputs one energy per position. Our specific choices are as follows.

Refinement loop. We use $K = 2$ refinement steps. The step size α is a learnable parameter, clamped to $[10^{-4}, 2]$, and the gradient of each step is clamped element-wise.

Training. Training follows Section 2.4.3: the cross-entropy between each refined prediction $\hat{y}_t^{(k)}$ and the ground-truth next token is averaged over the refinement steps and back-propagated through the refinement loop. The second-order gradients this requires (including with respect to α) are the main reason our EBTs train with smaller per-device batches than AR models of the same size (Section 4.2). Our EBTs are System 1 models (Section 2.4.3), with two steps, each with its own step embedding. This is the recipe for stable pretraining, and it was the one we could afford: the System 2 recipe keeps the computation graph across steps and adds a replay buffer, both of which increase memory, already the limiting factor. The trade-

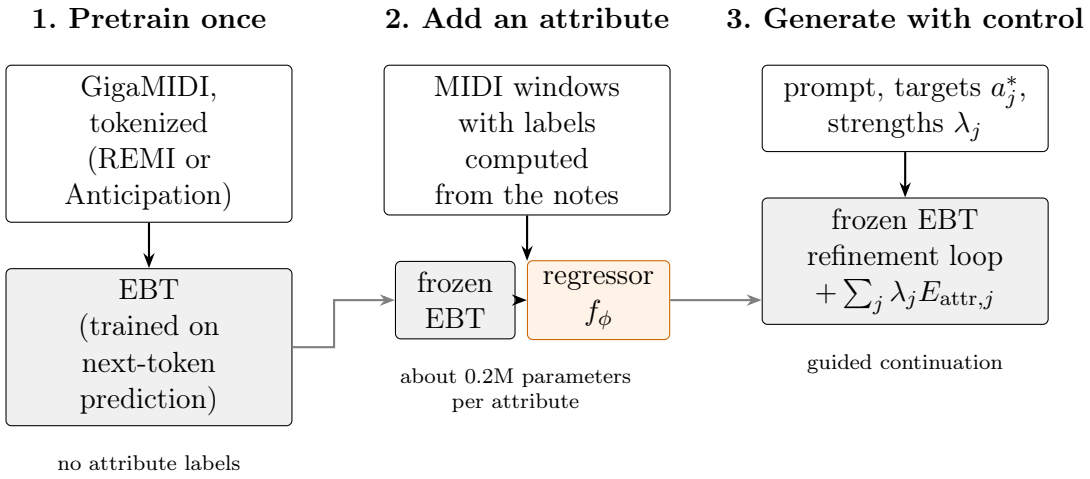


Figure 3.1: Overview of TONIC. (1) An EBT is pretrained once on tokenized MIDI, without any attribute labels. (2) To add a controllable attribute, a small regressor is trained on the frozen EBT's embeddings, with labels computed directly from the MIDI. (3) At generation time, each requested attribute contributes an energy, weighted by its strength, to the frozen EBT's own refinement loop. The generative model is never retrained.

off is that a System 1 EBT is not trained to improve with more steps (Section 5.1). A run with some of the System 2 regularisers is described in Section 4.2.

Generation. At inference the EBT is used like any left-to-right model: at each decoding step the refinement loop of Eq. (2.3) produces logits for the next position, a token is sampled from them (temperature and nucleus sampling, as for the baselines), and it is appended to the context. Only the prediction at the last position is sampled; the refinement loop is what makes this decoding step an optimisation problem, and therefore something that additional energy terms can act on (Section 3.3).

Tokenizations. We train EBTs on both representations of Section 2.2: REMI, with a vocabulary of 427 tokens, and Anticipation, with 55,028. Two of their differences matter for control. REMI has explicit velocity tokens, whereas Anticipation has none. And in both, the role of the next token is known in advance, through REMI's partly fixed grammar and Anticipation's fixed triplet cycle, which is what the gating of guidance relies on (Section 3.3).

3.2 Attribute Regressors on Frozen Embeddings

To steer generation towards an attribute value, we need a differentiable function that predicts that attribute from the model's current, partly uncertain, prediction. We use a regressor, which predicts a continuous value (Section 2.5.3), rather than a classifier, because our targets are continuous attribute values rather than categories. We train one small regressor per attribute and model.

Attributes. Each attribute $a(\cdot)$ is a rule-based function that maps a token sequence to a scalar; the same function provides the regressor's training labels and is used to evaluate the generated music (Section 4.4). We use:

- **Velocity** (REMI only): mean MIDI velocity of the velocity tokens, divided by 127, from soft, *piano* (0), to loud, *forte* (1). Anticipation has no velocity information, so this attribute is undefined for it.
- **Duration**: mean normalised note length, from staccato (0) to sustained (1). For REMI this is the mean rank of the duration tokens within the 64 ordered duration bins; for Anticipation, the duration in seconds divided by the tokenizer's maximum duration.
- **Pitch register**: mean MIDI pitch divided by 127, from low (0) to high (1). Drum notes (REMI's `PitchDrum` tokens, Anticipation's drum instrument) are excluded, since they identify a percussion instrument rather than a pitch.

These are low-level, objectively measurable attributes of the kind used throughout the controllable-generation literature: rhythm density and pitch register were among the first explicit controls of a neural music generator [31], and average pitch and average duration are the attributes steered in recent activation-steering work on a frozen music transformer [52, 51].

Architecture. The regressor f_ϕ is a three-layer MLP ($d \to 256 \to 128 \to 1$, ReLU activations) that takes the *mean token embedding* of a window of tokens as input. The embedding table W is taken from the generative model and kept frozen, so the regressor operates in that model's own embedding space. This has two consequences. First, the same regressor can read a *soft* token, a probability distribution p over the vocabulary, through its expected embedding $p^\top W$, which is what makes it differentiable with respect to the model's prediction. Second, a regressor is only valid for the checkpoint whose embeddings it was trained on: embeddings keep moving during pretraining, so we always pair a regressor with exactly the checkpoint it was trained against.

Training data. The regressor is trained to match the input it will receive during generation: a *local* window of $H = 48$ committed ("hard") tokens followed by one soft next-token distribution. Training windows are cut at random offsets from training-set songs; the soft token is the true next token with randomised noise, and the label is the attribute value of the window including the true next token. For Anticipation's pitch register, which is only carried by the note token of each triplet, the window is built from the last $H + 1$ *note* tokens rather than the last $H + 1$ raw tokens, so that time and duration tokens do not dilute it. The regressors are trained with a mean-squared-error loss (Section 4.2).

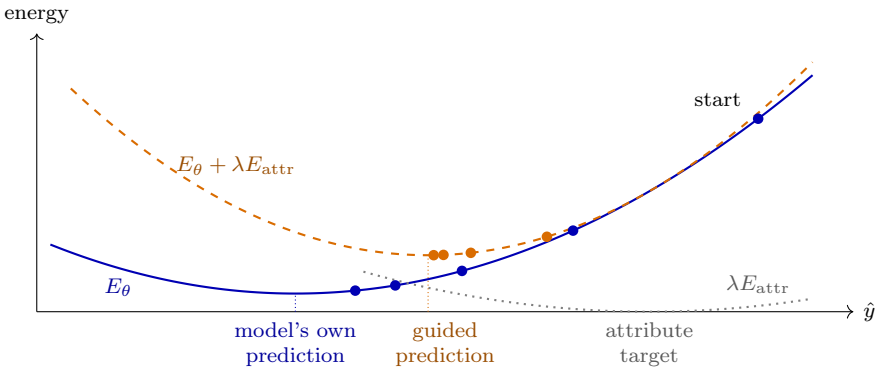


Figure 3.2: Refinement as descent on an energy landscape (schematic: the horizontal axis stands for the candidate prediction $\hat{y}$, shown in one dimension). Without guidance, gradient steps (blue dots) move a random starting candidate to the minimum of the model's energy E_θ. Adding an attribute energy λE_{attr}, which is lowest at the requested attribute value, reshapes the landscape: the same refinement loop (orange dots) now converges to a prediction between what the model finds plausible and the target. The strength λ sets the balance between the two.

3.3 Attribute Guidance in the Refinement Loop

We follow the compositional view of energy-based models of Du et al. [14]: conditioning on an attribute y amounts to adding the attribute's energy to the model's own, $\nabla \log p(x \mid y) = \nabla \log p(x) + \nabla \log p(y \mid x)$. For an EBT, the first term is already computed in every refinement step, so guidance only needs a second gradient added to the same update. Figure 3.2 illustrates the effect: the added energy moves the minimum that the refinement loop finds towards the requested attribute value.

Attribute energy. At decoding step t, let $e_1, \ldots, e_H$ be the embeddings of the last H committed tokens and $p_t = \text{softmax}(\hat{y}_t)$ the current soft prediction for the next token. The regressor's estimate of the local attribute value is

$$\hat{a}_t = f_\phi(\bar{e}_t)\,, \qquad \bar{e}_t = \frac{1}{H+1}\left(\sum_{i=1}^{H} e_i + p_t^\top W\right), \tag{3.1}$$

and the attribute energy is the scaled squared distance to the target a^*:

$$E_{\text{attr}}(\hat{y}_t) = (H+1)\,(\hat{a}_t - a^*)^2\,. \tag{3.2}$$

Only the prediction at the last position is ever sampled, so the energy is computed from that position alone; computing it from every position in the refinement state would spread the gradient over predictions that are discarded.

Guided update. In every refinement step, the gradient of Eq. (2.3) is replaced by

$$g = \nabla_{\hat{y}} E_\theta + \lambda \frac{\|\nabla_{\hat{y}} E_\theta\|}{\|\nabla_{\hat{y}} E_{\text{attr}}\|} \nabla_{\hat{y}} E_{\text{attr}}, \tag{3.3}$$

that is, the attribute gradient is rescaled to the norm of the EBT's own gradient before being weighted by the guidance strength λ. This makes λ interpretable independently of the regressor's output scale: $\lambda = 1$ means "as strong as the model's own energy gradient". Both norms are taken over the whole refinement state, that is, over all positions of the sequence, but the attribute gradient is non-zero only at the last position, whereas the model's own gradient is spread over all of them. At the position being predicted, the attribute gradient is therefore much larger, relative to the model's own gradient there, than λ suggests, and values of λ far below 1 already have a strong effect: all our operating points lie between 0.005 and 0.055 (Section 6.1.3). The target a^* and the strength λ are the only two control parameters. Several attributes can be combined by summing their energies before the normalisation in Eq. (3.3) (Section 3.4).

Gating by token type. An attribute can only be changed by tokens of certain types: velocity only by velocity tokens, pitch register only by pitch tokens. Applying guidance at every step would push, for example, the choice of a bar or position token towards a "louder" target, which can only harm coherence. We therefore *gate* guidance so that it is only active on steps where the next token can actually move the attribute. For REMI, the gates are read off the grammar: the token after a pitch token is always a velocity token, and the token after a velocity token is always a duration token, so velocity and duration guidance are active exactly on those steps, and pitch-register guidance is active after program tokens, which almost always precede a pitch token. For Anticipation, the gate is the matching position in the fixed $(\text{time}, \text{duration}, \text{note})$ cycle.

3.4 Combined-Attribute Control

Because energies add (Section 2.4.2), several attributes can be requested in the same generation by summing their energies. Each attribute j keeps its own regressor f_{ϕ_j}, target a_j^*, context window H_j and gate. At a decoding step, only the attributes whose gate is active contribute, and their energies are combined as

$$E_{\text{comb}}(\hat{y}_t) = \sum_{j \in \mathcal{A}_t} w_j \, E_{\text{attr},j}(\hat{y}_t), \tag{3.4}$$

where $\mathcal{A}_t$ is the set of attributes active at step t and w_j are relative weights. The combined energy then replaces E_{attr} in the guided update of Eq. (3.3), including its normalisation to the norm of the EBT's own gradient. No new training is needed: the regressors are the ones trained for single-attribute control.

Per-attribute strengths. Different attributes need different strengths: the usable range of λ differs between attributes and tokenizations (Sections 5.3 and 5.4). We therefore let each attribute carry its own strength λ_j. At each step, the overall strength is the sum of the strengths of the active attributes, $\lambda_t = \sum_{j \in \mathcal{A}_t} \lambda_j$, and each active attribute's weight is its share of that sum, $w_j = \lambda_j / \lambda_t$. When a single

attribute is active, this reduces exactly to its single-attribute guidance at strength λ_j, so each attribute can be run at the operating point tuned for it on its own.

Interaction with gating. Gating determines whether the energies of two attributes are ever summed at the same step. They are only if the two attributes share a gate. In the pairs we study, they do not: on REMI, velocity guidance is active on the step after a pitch token and duration guidance on the step after a velocity token; on Anticipation, duration and pitch register occupy different slots of the $(\text{time}, \text{duration}, \text{note})$ cycle. At every step, therefore, at most one attribute energy is active, and combined control steers each attribute on its own tokens, one after the other within each note. The attributes still influence each other through the generated context, since every token conditions the following ones, but no single refinement step has to trade one attribute energy off against another. A case in which two energies are minimised jointly at the same step would require attributes that share a gate; we discuss this in Section 6.2.

3.5 Guidance Baselines for Autoregressive Models

To ask whether TONIC is easier to steer than an AR model, the AR baselines (GPT-2 and Llama) need guidance methods under the same constraint: a frozen model and no retraining. Methods that build control into training (attribute tokens, cross-attention, classifier-free guidance) solve a different problem and are not compared here (Section 2.5). Activation steering [52, 63] also works on a frozen model, but it controls the model through its internal representations rather than its predictions; we leave a comparison with it to future work. We use three plug-and-play methods, each spanning a range of strengths from "barely moves" to "clearly saturated".

PPLM-style gradient guidance. Closest to our method, in the spirit of Dathathri et al. [8]: at a gated step, the AR model's next-token logits are perturbed by one normalised gradient step of the same attribute energy E_{attr}, computed by a regressor trained in the AR model's own embedding space. The strength is the L2 norm of the logit perturbation, in raw logit units, so it is not on the same scale as TONIC's λ. We apply it to Llama in both tokenizations. This is simpler than the original PPLM, as also used for music by Imasato et al. [30], which perturbs the cached key/value pairs of past positions over several iterations, adds a KL term towards the unperturbed distribution and fuses the two distributions. Our variant takes a single step on the output logits, which makes it as cheap as an unguided forward pass (Section 3.6).

Expectation tilting. For attributes that are read directly off a single token type (velocity, duration and pitch register under REMI), the attribute value of each token is known exactly, so no regressor is needed. At a gated step, let q be the model's distribution over that token type and v the tokens' attribute values. We replace q by the exponentially tilted distribution $q_\theta(i) \propto q(i)\, e^{\theta v_i}$, with θ chosen so that its expected value moves a fraction $s \in [0, 1]$ of the way from the model's own

Table 3.1: Guidance methods and where they apply (Ant.: Anticipation). All methods leave the generative model frozen.

Method	Models	Tokenization	Regressor	Strength
Energy guidance (ours)	TONIC	REMI, Ant.	yes	λ
PPLM-style gradient	Llama	REMI, Ant.	yes	logit-step norm
Expectation tilting	GPT-2, Llama	REMI	no	fraction s
Best-of-N	GPT-2, Llama	REMI, Ant.	no (oracle)	N

expectation μ towards the target: $\mathbb{E}_{q_\theta}[v] = \mu + s\,(a^* - \mu)$. This is the minimum-KL change to the model's distribution that satisfies the constraint, the gentlest per-step nudge that reaches it. Because it relies on a hand-written token-to-value map, it is a deliberately strong, attribute-specific baseline; regressor-based guidance also applies to attributes that no single token carries (such as note density). We apply it to GPT-2 and Llama on REMI.

Best-of-N reranking. Sample N unguided continuations and keep the one whose attribute value is closest to the target. The attribute is computed with the same rule-based function used for evaluation, so this is an *oracle* reranker and a standard sanity-check baseline. It applies to any model and tokenization.

Table 3.1 summarises which method is applied to which model and tokenization.

3.6 Computational Cost

The methods differ in how much computation they spend per generated token, so we compare them in *forward-pass equivalents* per token: the cost of one forward pass of the generative model, counting a backward pass as about two forward passes. This measure is independent of hardware and implementation, unlike wall-clock time.

- *AR model, unguided:* one forward pass per token.
- *EBT:* each refinement step needs a forward pass to compute the energy and a backward pass for its gradient with respect to the candidate, about three forward-pass equivalents. With $K = 2$ steps, an EBT costs about six per token. Guidance adds almost nothing to this: the EBT's own energy gradient is computed anyway, and the attribute energy only passes through the embedding table and a small regressor.
- *PPLM-style guidance and expectation tilting:* about one per token. The PPLM-style gradient is taken through the regressor with respect to the output logits only, not through the transformer, and tilting only reweights the output distribution.
- *Best-of-N:* N, since N full continuations are generated and scored.

TONIC is therefore about six times more expensive per token than an AR model with gradient or tilting guidance, whether guided or not. Section 4.3 uses this to set up a compute-matched comparison.

Chapter 4

Experimental Setup

This chapter describes the data and how the guided attributes are distributed in it (Section 4.1), the models and how they were trained (Section 4.2), the protocol used to compare attribute guidance across models (Section 4.3), the metrics (Section 4.4) and statistical tests (Section 4.5), the listening study (Section 4.6), the variants we tried and did not adopt (Section 4.7), and the compute environment and released code (Section 4.8). Where a choice in the setup depends on a measurement, such as the training cost of each model or the accuracy of the attribute regressors, we report that measurement here rather than in Chapter 5.

4.1 Data

All models are trained on GigaMIDI [36], a large crowd-sourced collection of MIDI files. The dataset (version 1.1) is divided into three parts: `drums-only`, `no-drums` and `all-instruments-with-drums`. We use only the last, full multi-instrument arrangements: drum-only files have no pitched notes for our attributes to act on, and we chose to train on arrangements in which drums and pitched instruments play together rather than to add the drum-free part as well. This gives 171,334 MIDI files, about 8% of the corpus, with the dataset's own split into training, validation and test sets (about 80/10/10%). All prompts used for evaluation come from the validation split.

Despite its name, only about 73% of the songs in this part contain drum notes by our own detection. Since the attributes ignore drum notes (Section 3.2), a prompt full of drums leaves guidance little to act on. For the REMI guidance experiments we therefore draw prompts from a pool of 201 of the 17,147 REMI validation songs whose opening 64 tokens, the prompt, contain no drum notes and at least 12 pitched notes (12 to 15). By comparison, only 58 of 201 randomly drawn validation songs have a drum-free opening. The songs themselves are not all drum-free (166 of the 201 have drums later on), but the model is only conditioned on the prompt.

The data is pre-tokenised in both representations. For REMI, we use the MidiTok [17] implementation, and each training example is a full song (about 15.8k tokens on average). For Anticipation (arrival-time) [67], no tokenization pipeline for GigaMIDI existed, so we built one (Section 4.1.1).

4.1.1 Tokenizing GigaMIDI with Anticipation

The reference implementation of the Anticipatory Music Transformer [67] includes data preparation only for the Lakh MIDI dataset. We started from the version of

the library adapted for the JAM_BOT [4] and extended it to GigaMIDI.

Pipeline. Tokenization follows the original two stages:

1. every MIDI file is converted into an intermediate list of notes (onset, duration, pitch, instrument, velocity); files that cannot be parsed are discarded;
2. each song is converted into $(\text{time}, \text{duration}, \text{note})$ triplets, consecutive songs are concatenated with separator tokens, and the stream is cut into sequences of 1024 tokens: one mode token followed by 341 triplets, with times made relative to the start of each sequence.

Songs with fewer than 100 notes, shorter than 10 seconds or with more than 16 instruments are skipped. We added a driver that applies this pipeline to GigaMIDI's train, validation and test splits. Of the 171,334 MIDI files, 93 (0.05%) could not be converted; Table 4.1 gives the remaining counts.

Anticipatory augmentation. Like the original work (Section 2.2.2), we use 10-fold *anticipatory augmentation*: every song is tokenized ten times.

- One copy is a plain autoregressive sequence (mode token `AUTOREGRESS`).
- In the other nine (mode token `ANTICIPATE`), a random subset of events (random time spans, random events, or whole instruments) is turned into *control* tokens that are interleaved into the sequence ahead of the time they occur.

Controls use a separate copy of the event vocabulary, which is why the vocabulary has 55,028 tokens, about twice the 27,515 needed without controls. Unlike the Lakh pipeline, which augments only the training split, we augment all three splits so that validation and test data follow the training distribution. In a sample of 2,000 validation sequences, 90% carry the `ANTICIPATE` mode token and 85% contain at least one control token.

Resulting data. Table 4.1 gives the number of files and sequences per split, together about 106 GB of token text with augmentation. The ten-fold augmentation and the short fixed-length windows explain why the Anticipation data has about 107 times more sequences than REMI: the same step budget takes a REMI model through many epochs, whereas no Anticipation model completes a single epoch (Section 4.2). The engineering needed to tokenize a corpus of this size on a shared cluster is described in Appendix A.

4.1.2 Attribute Distributions in the Corpus

Guidance targets are set in steps of σ_a, a standard deviation of the attribute over the corpus (Section 4.3). How large a move "one step" asks for therefore depends on how the attribute is distributed in the training music. To show this, we sample 2,000 training windows of 256 tokens per tokenization, one per song, and compute each attribute with the same functions used to score generations (Section 3.2). Windows

Table 4.1: The Anticipation data per split: MIDI files after conversion, and the 1024-token sequences they yield without and with the ten-fold augmentation.

Split	MIDI files	Sequences	Sequences (augmented)
Train	136,984	1,464,611	14,711,045
Validation	17,129	185,244	1,860,568
Test	17,128	183,065	1,838,819

Table 4.2: The guided attributes over 2,000 training windows of 256 tokens per tokenization, on their normalised $[0, 1]$ scales (Section 3.2). Last column: the target step σ_a, the same for both tokenizations (Section 4.3).

					Percentile		Target
Tokenization	Attribute	Mean	SD	Skew	5th	95th	step σ_a
REMI	Velocity	0.701	0.126	−0.02	0.496	0.913	0.136
	Duration	0.051	0.039	1.59	0.003	0.123	0.051
	Pitch register	0.449	0.060	−0.46	0.326	0.535	0.070
Anticipation	Duration	0.027	0.020	2.66	0.007	0.065	0.051
	Pitch register	0.453	0.068	−0.54	0.315	0.551	0.070

without pitched notes (10% for REMI, 4% for Anticipation) have no pitch register and are left out of that attribute. Table 4.2 gives summary statistics and Fig. 4.1 the full distributions.

Shape of each attribute.

- **Velocity** (REMI only) is nearly symmetric around 0.70, a MIDI velocity of about 89, with a small spike at the maximum.
- **Duration** is strongly right-skewed in both tokenizations: most windows consist of short notes, and the distribution is bounded by zero just below its peak. The two scales measure different things, so their values cannot be compared directly: REMI uses the rank of the duration bin among its 64 bins, Anticipation the note length divided by the 10-second maximum (a mean of 0.027 is about 0.27 s).
- **Pitch register** is concentrated around 0.45, a MIDI pitch of about 57 (the A below middle C), in both tokenizations, with a longer tail towards low registers than towards high ones.

The target step σ_a. σ_a was computed once, on an earlier sample of 1,821 REMI training windows, and is used for both tokenizations. This has two consequences:

- For velocity and pitch register, σ_a is close to the actual spread in either tokenization (within 17%), so one step asks for a typical move.

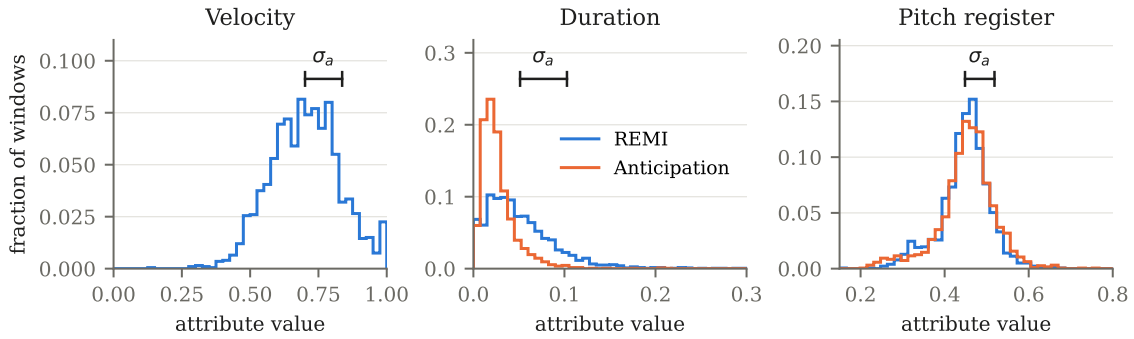


Figure 4.1: Distribution of the guided attributes over the windows of Table 4.2 (Anticipation has no velocity). The bracket shows one target step σ_a, starting at the REMI mean.

- For duration, σ_a is 1.3 times the standard deviation on REMI and 2.6 times on Anticipation, so the same target asks the Anticipation models for a much larger move, relative to their data, than the REMI models. And because duration is bounded by zero, downward targets often lie below anything a model can produce: one step below the corpus mean is already zero for REMI and below zero for Anticipation. Such targets are clipped at zero (Section 4.3).

4.2 Models and Training

This section describes the models and how they were trained: their architecture and optimiser (Section 4.2.1), the training of Tonic (Section 4.2.2) and of the baselines (Section 4.2.3), the cost of training and the choice of checkpoints (Section 4.2.4), and the attribute regressors (Section 4.2.5).

4.2.1 Architectures and Optimisation

Models. We compare Tonic, our EBT (Section 3.1), with two AR baselines, GPT-2 [54] and Llama [68]. GPT-2 is the architecture most used in symbolic music modelling; Llama adds the architectural improvements of recent language models: rotary position embeddings, RMS normalisation and SwiGLU feed-forward layers. All models share the same "small" size: 12 transformer blocks, 12 attention heads and an embedding dimension of 768, the size of GPT-2 small. Each architecture is trained separately on REMI and on Anticipation.

Why the small size. The model size was set by the cost of training an EBT. Its refinement loop needs second-order gradients (Section 3.1), so an EBT needs much more memory per training example than an AR model of the same size. On a single 48 GB GPU, even the small EBT has to be trained with a context of 512 tokens and four sequences per device; the Anticipation EBT does not fit a batch of eight even in evaluation. Each run also spans many scheduler jobs (Section 4.8). Within this budget, one size per tokenization and model family was what we could train to

a useful point, and a planned medium-sized Anticipation EBT was dropped. Since all models share the same size, our comparisons are between model families at this size; they say nothing about how the families scale (Section 6.2).

Optimisation. All runs use the same optimiser settings: AdamW with a peak learning rate of 8×10^{-4}, a linear warm-up over the first 10,000 steps followed by cosine decay to one tenth of the peak at 100,000 steps, weight decay 0.05 and gradient clipping at 1.0. The schedule of 100,000 steps follows the Anticipatory Music Transformer [67].

4.2.2 Training Tonic

Setup. Both Tonic models use a context of 512 tokens and an effective batch of 256 sequences on REMI (4×64 gradient-accumulation steps) and 64 on Anticipation (4×16). Pretraining was stopped on a fixed date, when the guidance experiments had to begin. The REMI EBT had then reached step 33,732. It advances more slowly in optimiser steps than the Anticipation EBT, because each of its steps accumulates four times as many micro-batches. For Anticipation we use step 88,800 rather than the last step, because the validation loss logged during training rose after it. On the shared validation windows evaluated later (Section 4.4.1), a slightly earlier state of the run, around step 77,400, scores a little better (1.088 against 1.102, Section 5.1); all guidance experiments use step 88,800. Both EBTs use $K = 2$ refinement steps with a learnable step size (initial value 1, at most 2) whose learning-rate multiplier is 20 for REMI and 2 for Anticipation; raising it to 20 for Anticipation gave no improvement in a direct test.

Variants tried. Both EBTs are System 1 models in the sense of Gladstone et al. [19], without regularisers (Section 3.1). A second Anticipation EBT was trained with three of the System 2 regularisers (Langevin noise $\sigma = 0.01$, step size resampled from $[\alpha/2, 2\alpha]$, replay buffer of 32) but otherwise kept the System 1 structure: two steps with their own step embeddings, a learned step size and detached predictions. It is therefore only partly a System 2 model. It was trained with a per-device batch of two to fit the buffer. Its best checkpoint (step 42,900) reached a higher validation loss than the run we use, measured on the same validation windows (1.130 against 1.102), so it is not used in the main experiments.

Stability: the step size. The main source of instability was the learnable step size α of the refinement loop. In early REMI runs it was trained with a learning rate 100 times the model's and had no upper bound. Once the warm-up ended, α grew without limit, and the validation loss rose and stayed high for a sustained stretch of training. Two changes fixed this, and all EBTs used in this thesis train with both: α is clamped to at most 2 (Section 3.1), and its learning-rate multiplier was lowered to 20 for REMI (2 for Anticipation). The gradient of each refinement step is also clamped element-wise.

Stability: Anticipation's noisier gradients. Anticipation was harder to train than REMI: its gradient norms were 2 to 2.65 times larger. Of the two differences between the runs that could explain this, a larger step-size multiplier gave no improvement (see above), whereas a short probe with REMI's effective batch of 256 instead of 64 lowered the validation loss much faster than the running job (from 1.086 to 1.038 within 150 steps), consistent with a larger batch averaging out the noisier gradients of a vocabulary 129 times larger. This was a single short probe, and we did not retrain the Anticipation EBT with the larger batch. Anticipation's higher loss is not itself a sign of unstable training: most of the gap to REMI is explained by the size of its vocabulary (Section B.1).

4.2.3 Training the Baselines

Three training recipes. Because TONIC trains with a shorter context and, per optimiser step, on fewer tokens than a standard AR setup, "the same training" can mean several things. We train the baselines in three ways, each answering a different question (Table 4.5):

- *EBT recipe (main comparison).* GPT-2 and Llama are trained exactly like TONIC: same context of 512 tokens, same effective batch, same learning-rate schedule, stopped at TONIC's step. They have therefore seen the same number of tokens and stop at the same point of the schedule. Their checkpoint is the best validation loss at or before that step, the same rule as for TONIC. Differences to TONIC then reflect the model family, not the training setup. All comparisons in Chapter 5 use these baselines unless stated otherwise.
- *Token-matched Llama (reference).* The AMT-style recipe below, stopped when Llama has seen as many tokens as TONIC.
- *AMT-style, 100,000 steps (reference).* The standard setup of the Anticipatory Music Transformer: a context of 1024 tokens, the largest batch that fits, and the full 100,000 steps. These models see 4.5–6 times more tokens than TONIC and show how far an AR model of the same size gets with more training.

Basis for comparison. Two models can be compared at the same training recipe, at the same number of tokens seen, or at the same training compute, and each answers a different question:

- *Same recipe* isolates the model family, which is what RQ2 asks about, so it is our main comparison.
- *Same tokens* asks which family uses data more efficiently; the EBT-recipe baselines are also token-matched, and the token-matched Llama adds the AMT-style recipe at the same budget.
- *Same compute* asks which family is cheaper to train. Since an EBT training step costs several times the floating-point operations of an AR step, this comparison clearly favours the AR models; we discuss it in Section 6.2 rather than training compute-matched models.

Table 4.3: Training cost of the main runs on one L40S GPU, from the logged run time. Every run spans several consecutive jobs that resume from checkpoints (Section 4.8). The baselines ran for the full number of steps (their selected checkpoints are slightly earlier, Table 4.5); TONIC's hours are summed up to the selected step and are approximate. Tokens are steps × effective batch × context 512.

Tokenization	Model	Steps	Tokens	GPU hours	M tokens / hour
REMI	TONIC	33,732	4.42 B	≈510	8.7
	GPT-2 (EBT recipe)	33,732	4.42 B	29.0	152
	Llama (EBT recipe)	33,732	4.42 B	14.0	316
Anticipation	TONIC	88,800	2.91 B	≈305	9.5
	GPT-2 (EBT recipe)	88,800	2.91 B	31.6	92
	Llama (EBT recipe)	88,800	2.91 B	21.9	133

Language-modelling losses are always compared on the same validation windows and at the same context of 512 tokens (Section 4.4.1).

4.2.4 Training Cost and Checkpoints

Training cost. All runs used one L40S GPU. Table 4.3 gives the GPU hours of the main runs, measured from the training logs, together with the tokens each run saw (steps × effective batch × 512).

Three things stand out:

- **TONIC is by far the most expensive model to train.** Per token it is about 17 times slower than GPT-2 and 36 times slower than Llama on REMI, and 10 and 14 times slower on Anticipation. This is more than the forward-pass count of Section 3.6 suggests, because training also back-propagates through the refinement steps themselves (second-order gradients, Section 3.1).
- **Anticipation is slower per token for the AR models,** by 1.7 (GPT-2) to 2.4 (Llama) times, which we attribute to its 55,028-way output layer (Section 2.2.2). For TONIC the throughput per token is about the same in both tokenizations, but the Anticipation EBT uses a four times smaller effective batch (16 instead of 64 accumulation steps), so each of its steps covers fewer tokens.
- **No run gets far through the Anticipation data.** This matters most in practice. The REMI EBT draws a window from each training song about 63 times in its 33,732 steps, whereas the Anticipation EBT sees 5.7 million sequences in its 88,800 steps, 39% of one epoch, after about 305 GPU hours. At the same rate, a single epoch would take about 790 GPU hours, or 33 GPU-days, on our hardware.

Checkpoint selection. For each model we use the checkpoint with the best validation loss, after discarding spurious readings: runs were resumed from checkpoints

many times (Section 4.8), and the first validation pass after a resume sometimes reported a loss far below the surrounding values, which does not reflect the model's quality. Table 4.5 lists the selected checkpoints. Every attribute regressor is trained against exactly the checkpoint it is used with (Section 3.2).

4.2.5 Attribute Regressors

Training. Each attribute regressor (Section 3.2) is trained with a mean-squared-error loss using AdamW (learning rate 10^{-3}, weight decay 10^{-4}, batch size 512) for 10 epochs on up to 500,000 training windows, with up to 50,000 held-out validation windows.

Accuracy. To tell weak control apart from a weak regressor, we evaluate every regressor used in the experiments on 1,000 validation windows of the shape it was trained on, with the true next token as the final input, against the exact attribute value of the window (Table 4.4).

- **REMI, and pitch register on Anticipation:** the regressors predict the attribute almost perfectly (Pearson $r \geq 0.985$). The weak control of pitch register on REMI (Section 5.3) is therefore not caused by the regressor.
- **Duration on Anticipation** is the exception. The regressors still order durations well ($r = 0.94$ on windows aligned to triplets), but their predictions are about a third of the true value.

The error in the Anticipation duration regressors has a simple cause and a small effect:

- *Cause.* Their training windows were cut at arbitrary positions in the token stream rather than at the start of a (time, duration, note) triplet, while the attribute function reads durations from fixed positions in the triplet. In two out of three windows it found no duration and returned zero, so most training labels were zero.
- *Effect on guidance.* Both TONIC and PPLM rescale the attribute gradient before applying the strength (Sections 3.3 and 3.5), so the scale error does not change how strongly guidance pushes, only where it stops. Guidance keeps pushing an upward request after the target is reached, and for a small downward request on a prompt with long notes it can push the wrong way.
- *Effect on the results.* At the operating points, this affects 4 of 96 requests for TONIC and 3 of 96 for PPLM, all at -0.5 standard deviations. Excluding them changes each method's directional accuracy on Anticipation duration by two points (0.78 to 0.80 and 0.95 to 0.97). Both methods use regressors with the same error, so their comparison is unaffected, but progress towards the target on Anticipation duration should be read with this in mind. The regressors were not retrained.

Table 4.4: Accuracy of the attribute regressors on 1,000 validation windows: Pearson correlation and mean absolute error (MAE) between the predicted and the exact attribute value. Windows without pitched notes are skipped for pitch register. †On windows aligned to triplets; the predictions are about a third of the true value (see text).

		EBT (Tonic)		Llama (PPLM)	
Tokenization	Attribute	r	MAE	r	MAE
REMI	Velocity	0.999	0.004	0.999	0.004
	Duration	0.999	0.002	0.999	0.001
	Pitch register	0.986	0.005	0.985	0.005
Anticipation	Duration†	0.939	0.018	0.937	0.018
	Pitch register	0.988	0.005	0.987	0.006

4.3 Attribute Guidance Protocol

The guidance experiments ask how well each model can be steered towards a target value of an attribute, at what cost to the music, as the guidance strength grows. All methods are evaluated with one shared protocol, so that their numbers are directly comparable.

Prompts and sampling. Each experiment uses 16 validation prompts of 64 tokens (two disjoint sets of 16, for tuning and testing, see below). For REMI they come from the drum-free melodic pool (Section 4.1); for Anticipation they are 16 validation sequences drawn with a fixed random seed from those that contain no control tokens. Because the validation data is augmented (Section 4.1.1), most windows carry anticipated controls: real notes that generation and decoding remove, so a prompt drawn from them would be missing part of the music. Checking the mode token is not sufficient, since a window that starts in autoregressive mode can run into the next, augmented copy of a song; we therefore keep only windows without any control token (1,216 of 12,000 sampled validation windows). The same pool provides the Anticipation prompts and real continuations for the unguided-quality evaluation and the real-music reference windows. Every model then generates 256 tokens with temperature 0.7 and nucleus sampling with $p = 0.9$.

Targets relative to the model's own behaviour. Models differ in what they generate unguided, so targets are set relative to each model's own output rather than as absolute values. Each target thus asks for a move of a known size and direction away from what the model would have done anyway. For every model and prompt:

1. *Baseline.* We draw three unguided continuations and take the mean of their attribute values as the prompt's baseline b.

2. *Targets.* Six targets are placed around the baseline,

$$a^* = \max\big(0,\ b + \delta\,\sigma_a\big), \qquad \delta \in \{-2, -1, -0.5, +0.5, +1, +2\},$$

Table 4.5: Training configuration and selected checkpoints. Effective batch = per-device batch × gradient accumulation (one GPU per run). Tokens = step × effective batch × context length, the number of tokens processed up to the selected checkpoint. The EBT-recipe baselines are the main comparison; the others are references.

Tokenization	Model	Recipe	Context	Eff. batch	Step	Tokens
REMI	TONIC	—	512	256 (4 × 64)	33,732	4.4B
	GPT-2	EBT recipe	512	256 (16 × 16)	33,732	4.4B
	Llama	EBT recipe	512	256 (16 × 16)	33,196	4.4B
	Llama	token-matched	1024	256 (16 × 16)	16,866	4.4B
	GPT-2	AMT, 100k	1024	256 (16 × 16)	99,660	26.1B
	Llama	AMT, 100k	1024	256 (16 × 16)	99,660	26.1B
Anticipation	TONIC	—	512	64 (4 × 16)	88,800	2.9B
	GPT-2	EBT recipe	512	64 (8 × 8)	87,900	2.9B
	Llama	EBT recipe	512	64 (8 × 8)	88,500	2.9B
	Llama	token-matched	1024	128 (8 × 16)	22,200	2.9B
	GPT-2	AMT, 100k	1024	128 (8 × 16)	98,900	13.0B
	Llama	AMT, 100k	1024	128 (8 × 16)	100,000	13.1B

where σ_a is the target step of Table 4.2. Clipping at zero only affects downward duration targets (Section 4.1.2).

3. *Guided samples.* For each strength and target, one guided continuation is generated.

Attributes and strengths. On REMI we guide velocity, duration and pitch register; on Anticipation, duration and pitch register (Anticipation has no velocity). For TONIC, the grids of λ are attribute-specific, chosen in preliminary sweeps to span the range from negligible to saturated effect:

- velocity: $\lambda \in \{0.01, 0.02, 0.03, 0.04, 0.05, 0.06, 0.07, 0.09\}$,
- duration: $\lambda \in \{0.005, 0.01, 0.015, 0.02, 0.025, 0.03, 0.04, 0.055\}$,
- pitch register: $\lambda \in \{0.01, 0.02, 0.04, 0.06, 0.08, 0.10, 0.13, 0.16\}$.

These grids were tuned on an earlier checkpoint and regressors and are kept unchanged for comparability. For Anticipation, where λ had not been tuned, we use a logarithmic grid, $\lambda \in \{0.005 \cdot 2^k : k = 0, \ldots, 7\}$ (0.005 to 0.64), refined at the low end, where the usable strengths turned out to lie:

$$\lambda \in \{0.002, 0.004, 0.005, 0.006, 0.008, 0.01, 0.012, 0.015, 0.02, 0.04, 0.08, 0.16, 0.32, 0.64\}.$$

For the AR methods (Section 3.5), the strengths are:

- PPLM-style guidance (Llama): logit-step norms $\{1, 2, 4, 8, 16, 32\}$ on REMI and $\{0.5, 1, 2, 4, 8, 16, 32, 64\}$ on Anticipation;

- expectation tilting (GPT-2 and Llama, REMI only): $s \in \{0.05, 0.1, 0.2, 0.3, 0.5, 0.75, 1.0\}$;
- best-of-N (GPT-2 and Llama, both tokenizations): $N \in \{2, 4, 8, 16\}$, drawn from one shared pool of 16 samples per prompt (best-of-4 selects from the pool's first four samples, and so on).

Table 3.1 lists which method is run on which model and tokenization.

Held-out choice of operating points. Every method has a strength knob, and the best strength differs between methods and attributes. To compare methods, we need one strength per method and attribute, its *operating point*. If we picked it on the same prompts we then report, a method's score would partly reflect how well its strength happened to suit those 16 prompts, and methods with many or sensitive settings would gain most from this. We therefore split the prompts into two disjoint sets of 16 per tokenization, run with the same targets, sampling settings and strength grids:

- The **tuning set** is used only to choose the operating points. For each method and attribute, the operating point is the strength with the highest strict directional accuracy (Section 4.4.3) among those that keep the music acceptable: harsh dissonance at most one standard deviation above the real-music mean, and an overlapping area of at least 0.75 with the same model's unguided output (Section 4.4.2).
- The **test set** is used only to report results. Each operating point is evaluated on it unchanged, so its score shows how well the choice carries over to new prompts.

All numbers and comparisons in Chapter 5 come from the test set.

Compute-matched comparison. Best-of-N can buy accuracy with compute, while TONIC costs about six forward-pass equivalents per token (Section 3.6). Besides each method's own operating point, we therefore also compare against best-of-N with $N = 4$, the largest setting that does not exceed TONIC's cost.

Combined control. Combined control steers two attributes in the same generation. The setup follows the single-attribute protocol, with these differences:

- **Attribute pairs.** One pair per tokenization: velocity and duration on REMI, duration and pitch register on Anticipation.
- **Targets.** For each of the 16 prompts, targets are set for both attributes at once, at ± 1 and ± 2 standard deviations, in all four combinations of directions: both up, both down, and the two mixed cases.
- **Strengths.** No new tuning is done; every method reuses its held-out operating points from the single-attribute experiments:

  - TONIC guides each attribute at its own operating point, using per-attribute strengths (Section 3.4);
  - the AR gradient and tilting methods likewise use their own operating points;
  - best-of-N uses one N for both attributes, the larger of its two single-attribute values, alongside the compute-matched best-of-4.

- **Strengths chosen by ear.** We additionally run TONIC at operating points chosen by ear, which favour musicality over accuracy.

4.4 Evaluation Metrics

4.4.1 Language Modelling

We report validation cross-entropy. The EBT-recipe baselines have seen exactly as many tokens as TONIC, so their losses are compared directly; the reference runs saw more (Table 4.5) and are compared at matched numbers of tokens seen. Losses are not comparable across tokenizations (the vocabularies differ by two orders of magnitude), so for each tokenization we place them between two reference points: the entropy of a uniform distribution over the vocabulary, $\ln V$, and the entropy of the empirical unigram distribution of the validation data (a model that ignores context), estimated on 30M tokens.

Since the TONIC models are trained with a context of 512 tokens and the 100k baselines with 1024, the main comparison evaluates all models on the same 1,000 seeded validation windows at a context of 512 (*context-matched loss*), using the last refinement step for TONIC.

Thinking longer. Gladstone et al. [19] report that an EBT's predictions improve with more computation at inference, in two forms: more refinement steps, and refining several candidates and keeping the one with the lowest energy (self-verification). They obtain these gains with their System 2 recipe, and their own ablation finds no gain from more steps when the number of steps is not randomised during training. For our System 1 models (Section 3.1) we therefore expect little, but test both forms with the context-matched loss. For more steps, we evaluate K from 1 to 16; our EBTs have a step embedding only for the two steps used in training, so extra steps repeat the last trained step, with its learned step size. For self-verification, we refine eight candidates from different random starts and keep, at each position, the one with the lowest energy.

4.4.2 Musical Quality

Strong guidance can reach a target by producing implausible music, so every control result is paired with quality measures.

Token-level measures (REMI). The **bigram log-likelihood** of a generated sequence is the mean of $\log P(x_{i+1} \mid x_i)$ over its positions, under a bigram table estimated from real data; a drop relative to the same model's unguided samples indicates unusual token transitions. The **grammar violation rate** is the fraction of positions at which REMI's deterministic transitions are broken (a pitch token not followed by a velocity token, or a velocity token not followed by a duration token); well-formed output scores close to zero. The Anticipation vocabulary is too large for a dense bigram table, so these measures are reported for REMI only.

MIDI-level measures. Token-level measures say whether a sequence is well formed, not whether it sounds musical. We therefore decode the generated tokens to MIDI and compute eight descriptors of the music itself, regardless of tokenization:

- *Pitch and harmony:* pitch-class entropy; scale consistency (fraction of notes in the best-fitting major or minor scale); harsh dissonance (fraction of simultaneously sounding pitch pairs a minor second, tritone or major seventh apart).
- *Texture:* polyphony; pitch range; empty-beat rate.
- *Rhythm and structure:* groove consistency (similarity of consecutive bars' onset patterns on a 16th-note grid); bar self-similarity.

None of these is "higher is better"; real music has a typical range for each. Several of them (pitch-class entropy, scale consistency, groove consistency) are also used to monitor music quality under activation steering [51].

Following Yang and Lerch [74], a generated set is scored by the *overlapping area* (OA) between the kernel-density estimates of each descriptor over the generated set and over a reference set of real music, from 0 (disjoint) to 1 (identical). Only the continuation is scored, with the prompt's notes removed; pieces with fewer than 8 pitched notes are skipped. The reference set is the real continuation of the same prompts. As a ceiling, two random samples of real music of the same size reach an OA of about 0.88–0.89.

We also measure the continuation's coherence with its prompt as key continuity: the cosine similarity of the duration-weighted pitch-class histograms of prompt and continuation.

How the measures are used. The MIDI-level measures are applied in two settings:

- *Unguided generation.* To compare quality without guidance, each model continues the same 100 validation songs (first 64 tokens as the prompt, 256 generated tokens, same sampling settings), and the continuations are scored with the MIDI-level measures above.
- *Guided generation.* Two references are used. Overlap with *real music* shows how plausible the output is; overlap with the *same model's unguided output* ($\mathrm{OA}_{\mathrm{ung}}$) shows how much guidance changed the music, independently of how good the model was to begin with. Because harsh dissonance is the clearest

sign of guidance breaking the music, it is also reported on its own, against the real-music range and against uniformly random notes (0.246).

4.4.3 Control

The achieved value a of a sample is computed with the exact attribute function (Section 3.2) on the 256 generated tokens, without the prompt. The regressor that guides generation is never used for scoring. For a guided sample with achieved value a, prompt baseline b and requested change $\Delta = \delta\,\sigma_a$ (before clipping the target at zero), we report:

- **Strict directional accuracy**: the fraction of samples that moved away from the baseline in the requested direction, $(a - b)\,\Delta > 0$. Samples that do not move count as misses.
- **Progress towards the target**: $(a - b)/\Delta$, the fraction of the requested change that was achieved (1: target reached, 0: no movement, negative: wrong direction), clipped to $[-3, 3]$ per sample and reported separately for upward and downward targets.
- **Mean absolute error (MAE)** to the unclipped target, $|a - (b + \Delta)|$.
- For combined control, **joint accuracy**: the fraction of samples in which *both* attributes moved in their requested directions.

Directional accuracy says whether guidance has a reliable effect, but not how large it is: a method that moves every sample slightly in the right direction scores as well as one that reaches the target. Progress towards the target and MAE show how far each method actually goes.

How the attribute's values change. To see *how* strong guidance acts, we record two further quantities for Tonic at every strength and for the AR methods at their strongest setting. The first is the *spread* of the attribute's values within a continuation: the entropy of the values of the tokens that carry the attribute (128 MIDI pitches; note lengths in logarithmic bins; velocity in 32 bins), divided by the largest possible entropy, so that 0 means that all values are the same and 1 that they are uniformly random. The second is the *random-token value*: the attribute value of tokens drawn uniformly at random from the attribute's value tokens. It shows where the attribute would end up if guidance overrode the model's choice entirely.

4.4.4 Compute

Compute is reported in forward-pass equivalents per generated token, as defined in Section 3.6.

4.5 Statistical Analysis

Each kind of comparison is paired, so that methods are compared on the same inputs:

- *Language modelling.* Context-matched losses are compared with a paired bootstrap over the 1,000 validation windows.
- *Unguided quality.* Models are compared with paired permutation tests over songs, with Holm correction over the four model comparisons.
- *Guidance.* All guidance comparisons use the same 16 test prompts for every method, so they are paired by prompt. For each prompt, we compute a method's strict accuracy over its six targets. Per-method 95% confidence intervals come from resampling prompts (10,000 bootstrap draws). Differences between a method and TONIC are tested with an exact two-sided sign-flip permutation test over the 16 prompts (2^{16} sign assignments), and the p-values are corrected for multiple comparisons with the Holm procedure over all 31 comparisons (both tokenizations, operating points and compute-matched best-of-4).

Limitations. These tests account for the choice of prompts only. Every model was trained once, with a single seed, so variation between training runs is not captured. With 16 prompts and a conservative correction, only large differences reach significance; we therefore report effect sizes and uncorrected p-values alongside the corrected ones.

4.6 Listening Study

The automatic metrics measure whether an attribute moved and how likely the music is under a reference model; they do not say whether a listener hears the change, or whether the result still sounds like music. We therefore ran a blind online listening study, in English and Italian (Section D.6). The link was shared with personal contacts and in group chats. Participation was voluntary and anonymous, and participants gave their consent on the first page before any question was shown.

A study of selected examples. We chose the clips by ear from the outputs of the final models: for each system, prompt and condition, the continuations that sounded most convincing were kept, for TONIC and for each baseline alike (any guidance strength could be picked; the outputs of TONIC were browsed on a separate page because of their finer strength grid). The study therefore asks what the methods can produce at their best, and whether listeners perceive the requested changes in them; it is not a controlled comparison of average outputs, which is the role of the automatic metrics. The baselines are the AR models trained with the EBT recipe (Section 4.2). Every clip consists of a short prompt from the validation split (median 1.3 s) and its continuation; clips last 7 s (median; interquartile range 5–10 s). MIDI

was rendered with FluidSynth and the MuseScore General soundfont at fixed gain and encoded as MP3. Since Anticipation does not encode velocity, all Anticipation clips, the human originals included, play at one loudness.

Questions. Each participant received a random subset of 25 questions in three parts, about 20 minutes in total:

- *Part 1, unguided continuations* (6 questions + 1 check). Four versions of one prompt's continuation (TONIC, Llama, GPT-2 and the human original as a hidden anchor), labelled A–D in random order. Participants chose the most and the least musical version. The pool contains 23 such sets (10 REMI, 13 Anticipation) over 21 prompts.
- *Part 2, perceived change* (12 questions + 1 check). A system's own unguided continuation (the reference) and the same system's steered continuation for the same prompt, with the attribute to listen for (loudness, note length or pitch). Answers on a five-point scale from, e.g., "clearly softer" through "no difference" to "clearly louder". The pool contains 297 pairs, with targets of 0.5, 1 or 2 σ_a above or below the reference (Section 4.1.2).
- *Part 3, steered versions compared* (5 questions). The steered continuations of two to four systems for the same prompt, attribute and direction; participants chose the most and the least musical (117 questions in the pool).

Every question ended with an optional item: "did any version sound cacophonous (harsh, chaotic, almost random notes)?", ticked per version. System names were hidden, file names were salted hashes, and answers were unlocked only after at least 70% of every clip had been played. A participant never received two questions about the same prompt (Part 1) or the same prompt, attribute and direction (Part 3). Every question could be skipped. Before the questions, participants reported their musical training (none / some / extensive) and how they listened (headphones, laptop or phone speakers, external speakers). The instructions did not say whether to judge the music as a whole or each instrument separately; some participants asked, and we return to this in Section 6.2.

Attention checks. Each participant's Part 1 contains one check set, drawn from a pool of four, in which a TONIC continuation is guided far beyond its operating point and is close to random; it should be chosen as least musical. Part 2 contains one pair in which the reference is compared with itself; the answer should be "no difference". One of the four broken-clip sets is a poor check, because the other three versions in it are nearly identical, so the broken clip is the only one that develops.

Participants and analysis. We received 67 submissions. One, in which every question was skipped, was discarded, leaving 66 participants (56 English, 10 Italian): 23 without musical training, 32 with some and 10 with extensive training (one did not answer); 38 used headphones. The broken-clip check depends partly on musical judgement, and more than a quarter of the participants skipped the identical pair,

so neither check was used to exclude participants; Appendix C repeats the analysis without the 7 participants who failed the broken-clip check on a valid set, and without one participant who ticked exactly one version as cacophonous in every question (apparently reading the optional item as compulsory), whose cacophony ticks are left out throughout. Neither changes the conclusions. Skipped questions are ignored. Part 1 and Part 3 are summarised by a best–worst score per system: the number of times it was chosen as most musical minus the number of times it was chosen as least musical, divided by the number of times it appeared (-1 to $+1$). Part 2 is summarised by the share of answers in the requested direction and by the signed answer (-2 to $+2$, positive in the requested direction). All intervals are 95% bootstrap intervals that resample participants (2,000 resamples), so they account for answers of one person being correlated.

4.7 Approaches Tried and Not Adopted

Several variants were tried before the setup above was fixed. The training variants are described with the models (Section 4.2): an Anticipation EBT with the three regularisers of Gladstone et al. [19], a larger step-size learning-rate multiplier, a larger batch, and a medium-sized model. This section collects the variants of guidance. All were exploratory: they were run on an earlier REMI EBT checkpoint (step 10,584), with regressors reading a window of 20 tokens, on six prompts and one sample per condition, so they indicate directions rather than establish results.

Energy over all positions. In the first implementation, the attribute energy was computed from the predictions at every position of the refinement state. Only the last one is sampled, so most of the gradient went to predictions that are discarded, and the guided music lost coherence. The energy is now computed from the last position only (Section 3.3), and the regressors were retrained on matching windows of committed tokens followed by one soft token (Section 3.2).

Attributes that count notes. We also built regressors for note density, polyphony, rhythmic pace, drum density and syncopation. These attributes count or place notes rather than setting the value of one token. Raising them was possible, but lowering them was not reliable: lowering density means *not* placing a note, and guidance can only bias the next token, never remove one that has already been generated. In the downward direction the attribute tracked the target weakly and not monotonically in λ, while larger strengths degraded the music. Guidance towards these attributes also made the model add drums to prompts that had none, since their gates do not distinguish a pitched note from a drum hit; masking the drum instrument for drum-free prompts prevented this. We kept velocity, duration and pitch register, which are each set by the value of one token, and the drum mask is not used in the final experiments.

Removing notes after generation. To make the downward direction reachable, we tried a discrete edit after generation: repeatedly remove the note (its pitch,

velocity and duration tokens) whose removal most lowers the sum of TONIC's energy and the attribute energy. In tests on density, polyphony and drum density it either found no removal that helped or used its whole budget of 30 removals without reaching the target, and the bigram log-likelihood of the edited music dropped. It was not pursued.

Composing attributes that share a gate. With the note-counting attributes, two pairs share a gate and their energies are therefore summed at the same refinement steps: note density and polyphony (both guided at duration and position tokens), and pitch register and drum density (both at program tokens). We ran each pair in all four combinations of directions, with targets three standard deviations from the prompt's baseline. For density and polyphony, two positively correlated attributes, both moved as requested when asked to move the same way (4 to 6 of 6 prompts for each attribute, at $\lambda = 0.02$ and 0.04). When asked to move in opposite directions, polyphony prevailed: it moved as requested in 3 to 6 of 6 prompts, while density moved the wrong way, following polyphony (0 or 1 of 6). For pitch register and drum density, pitch register moved as requested in every combination (4 to 6 of 6), so the second energy did not disturb it; drum density could only be raised, since the drum-free prompts left nothing to lower. These runs are the only ones in which two attribute energies were minimised at the same step. They suggest that summed energies work when the attributes are independent or agree, and that one attribute dominates when correlated attributes are asked to diverge (Section 6.2).

Decaying the strength during generation. Because strong guidance degrades the music more as generation goes on, we tried holding λ at full strength for the first part of the continuation and then decaying it linearly. It was tested in a single run and not pursued.

4.8 Implementation and Reproducibility

Compute environment. All models were trained and evaluated on the MIT Office of Research Computing and Data (ORCD) cluster, each run on a single NVIDIA L40S GPU (48 GB). Jobs ran under the cluster's Slurm scheduler on shared partitions, with a limit of two GPUs per user on the standard GPU partition and further GPUs available on a preemptable partition, where jobs can be stopped without warning, and with a maximum wall-clock time per job (24 hours for TONIC jobs and 6 hours, the limit of the standard partition, for the baselines). GPU hours per run are given in Table 4.3. In total, the project used about 3,600 GPU hours between March and November 2026, almost all of them on L40S GPUs, including exploratory, abandoned and repeated runs, guidance sweeps and evaluation; the six final models account for about 910 of them.

Checkpointing and automatic resumption. No training run fit into a single job, so training was organised around frequent checkpoints and automatic resumption. Each training job saves checkpoints regularly, and when it reaches its time

limit it saves a final checkpoint and resubmits itself. If a job on the preemptable partition is killed without warning, the scheduler requeues it. In both cases the new job resumes from the checkpoint with the highest training step that is compatible with the run's configuration, rather than from the most recently written file, which after a preemption can be an older or partial one. Checkpoints are written at each validation pass, so the validation interval also sets how much training a preemption can lose. A TONIC step on REMI takes about a minute, against a few seconds for the baselines, so TONIC validates and saves a checkpoint every 100 optimiser steps (about an hour and a half of training) rather than every 500, as the REMI baselines do. A single training run therefore consists of many consecutive jobs, and its logs of several segments, which we stitch together for analysis.

Code, logs and artefacts. The code for training, guidance and evaluation is available at `https://github.com/RebeccaCeccatelli/music_EBT`, and the extended `anticipation` library for GigaMIDI is released separately[1] (Appendix A). All runs were logged with Weights & Biases, and the logs are public[2]: pretraining in the projects `mus_symb_ebt_pretrain` and `mus_symb_baseline_pretrain`, and regressor training and guidance sweeps in `mus_symb_attr_control`. Audio examples are available in the showcase, the listening archive and the interactive demo (Appendix D).

[1] `https://github.com/RebeccaCeccatelli/anticipation-musicEBT`
[2] `https://wandb.ai/rceccatelli-eth-z-rich`

Chapter 5

Results

This chapter reports the results in the order of the research questions: how well the models model music (Section 5.1) and how musical their unguided continuations are (Section 5.2), how well they can be steered towards single attributes (Sections 5.3 and 5.4) and two attributes at once (Section 5.5), what steering costs musically and why (Section 5.6), and what listeners heard (Section 5.7). Unless stated otherwise, the AR baselines are the ones trained with the EBT's recipe (Section 4.2), and all guidance results are on the test prompts at operating points chosen on the tuning prompts (Section 4.3).

5.1 Language Modelling Performance

> **In brief.** With REMI, TONIC models music as well as the AR baselines trained the same way; with Anticipation it is 3–4% behind. Running more refinement steps at inference does not help.

Same training recipe. Table 5.1 compares the validation loss of all models on the same 1,000 validation windows at a context of 512 tokens.

- **REMI:** TONIC is on par with the AR baselines. Its loss is slightly lower than Llama's (-0.004) and slightly higher than GPT-2's ($+0.014$).
- **Anticipation:** TONIC is behind both, by 0.046 (Llama) and 0.032 (GPT-2) nats per token, about 3 to 4%.

These differences are small but measured precisely: their 95% paired bootstrap intervals over windows are within ± 0.004. They are also small compared with the distance to the reference points: every model reduces the loss of a context-free unigram model by about 85% in both tokenizations.

Training curves. Figure 5.1 shows how the models got there.

- *REMI.* TONIC starts behind both baselines and catches up with Llama by the end of training; all three are still improving when training stops.
- *Anticipation.* TONIC keeps pace with the baselines for the first 1.6 billion tokens and then falls behind. Its loss is lowest at about 2.5 billion tokens (1.088 at step 77,400) and rises again afterwards. The checkpoint we use (step 88,800, 1.102) was chosen on the losses logged during training (Section 4.2.2);

Table 5.1: Validation loss (cross-entropy in nats per token; lower is better) on the same 1,000 validation windows for every model. Losses can be compared within a column, not between the two tokenizations.

Model	REMI	Anticipation
Same training recipe (main comparison)		
TONIC	0.696	1.102
Llama	0.700	1.056
GPT-2	0.683	1.070
Standard AR recipe (reference)		
Llama, same number of tokens	0.717	1.136
Llama, 100k steps	0.676	1.003
GPT-2, 100k steps	0.635	0.992
Reference points		
Ignoring the context (unigram)	4.57	7.43
Uniform guess ($\ln V$)	6.06	10.92

the baselines had reached its loss after about 1.9 billion tokens. Even the best TONIC state is behind the baselines at the same point (1.088 against 1.07–1.08).

Effect of interruptions. TONIC's training was interrupted many times by the cluster's time limits, and this shows in both panels:

- *Noisier REMI readings.* To lose less progress at each interruption, TONIC was validated every 100 steps rather than every 500 (Section 4.8), so more of the fluctuation is visible. Short upward spikes between 2.5 and 3.5 billion tokens fall in a period when the run was resumed especially often; we did not establish their cause.
- *Few Anticipation points.* Only the states saved around the interruptions are still available, so TONIC's curve has seven points.

More training. The reference runs show what more training buys:

- With the standard recipe for 100,000 steps, seeing 4.5 to 6 times more tokens, the AR models reach 0.64–0.68 with REMI and 0.99–1.00 with Anticipation.
- With the standard recipe but the same number of tokens as TONIC, Llama is behind TONIC in both tokenizations (0.717 against 0.696 with REMI, 1.136 against 1.102 with Anticipation).

Whether TONIC would keep pace with more training cannot be told from our runs (Section 6.1.1).

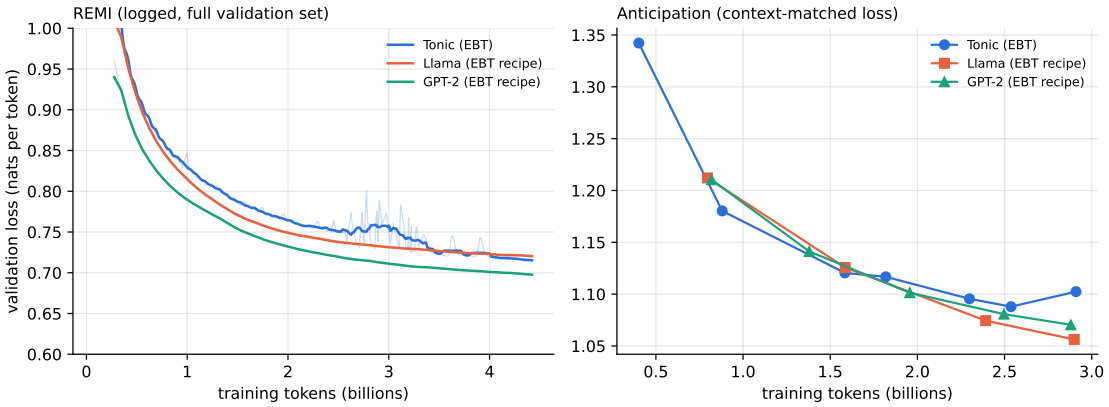


Figure 5.1: Validation loss against training tokens for TONIC and the AR baselines. TONIC's training was interrupted many times by the cluster's time limits, which explains the two irregularities. *Left, REMI:* losses logged during training; bold lines are running means, faint lines the raw readings. TONIC's readings are noisier because it was validated more often, so that less progress was lost at each interruption. *Right, Anticipation:* loss on shared validation windows (Section 4.4.1). TONIC has fewer points because only the states saved around its interruptions are still available.

Comparing the tokenizations. The losses of the two tokenizations are not comparable with each other: most of Anticipation's higher loss is explained by its 129 times larger vocabulary. Per note, the two are close (Section B.1).

Thinking longer. Neither form of "thinking longer" (Section 4.4.1) helps TONIC:

- **More steps make predictions worse** (Table 5.2). The second step improves on the first only slightly (0.699 to 0.696 with REMI, 1.110 to 1.102 with Anticipation). Beyond the two steps used in training, the loss rises with every step and levels off by about twelve steps, +0.007 with REMI and +0.019 with Anticipation after sixteen. The energy keeps falling slightly, and the entropy of the predictions falls while the loss rises.
- **Longer training makes this worse.** The original Anticipation EBT loses +0.010 from extra steps at step 55,500 and +0.019 at the final checkpoint.
- **The System 2 regularisers halve the loss from extra steps** at a comparable stage of training (+0.006 against +0.010), but that model too is best at two steps (Section B.1).
- **Self-verification does not help.** Keeping the lowest-energy of eight candidates gives the same loss as a single start (0.696 with REMI, 1.102 with Anticipation), and even an oracle that kept the candidate with the lowest loss would gain at most 0.003: the eight candidates end up almost identical.

A third variant, a smaller step size, cannot be tested meaningfully with these models (Section B.1). We interpret these results in Section 6.1.6.

Table 5.2: Validation cross-entropy of the two TONIC models after K refinement steps on the windows of Table 5.1; both were trained with $K = 2$. Steps beyond two repeat the last trained step. Energy: mean energy of the prediction.

	K	1	2	3	4	8	16
REMI	loss	0.699	**0.696**	0.698	0.699	0.702	0.703
	energy	7.38	2.40	2.38	2.38	2.37	2.37
Anticipation	loss	1.110	**1.102**	1.109	1.114	1.120	1.121
	energy	6.65	3.12	3.09	3.07	3.07	3.06

5.2 Unguided Music Quality

In brief. Without guidance, all models write music close to real music, and TONIC is not significantly different from the baselines trained the same way.

Table 5.3 compares each model's unguided continuations with the real continuations of the same prompts, averaged over the eight MIDI-level descriptors (Section 4.4.2).

- **All models are close to real music.** The overlap is 0.81–0.87 with REMI and 0.78–0.82 with Anticipation, against a ceiling of 0.89 and 0.88 for two samples of real music.
- **TONIC is on par with the same-recipe baselines.** It lies between them with REMI and slightly below both with Anticipation, but none of these differences is significant (permutation tests over songs, $p \geq 0.08$ before correction). Only the GPT-2 trained for 100,000 steps is significantly closer to real music than TONIC, with REMI (+0.04, Holm-corrected $p = 0.01$).
- **All models write slightly more conventional music than real music**, TONIC included: more notes in the scale, lower pitch-class entropy, a narrower pitch range and more repetition between bars.

The listening study confirms that the unguided continuations are musical (Section 5.7).

Table 5.3: Unguided music quality: how close each model's continuations are to the real continuations of the same prompts (overlapping area averaged over eight descriptors; 1 = identical), with 95% intervals over the 82 REMI and 95 Anticipation songs with at least 8 pitched notes. *Significantly different from TONIC after Holm correction.

Model	REMI	Anticipation
Same training recipe (main comparison)		
TONIC	0.83 [0.77, 0.85]	0.78 [0.73, 0.80]
Llama	0.86 [0.78, 0.87]	0.79 [0.72, 0.81]
GPT-2	0.81 [0.75, 0.83]	0.81 [0.74, 0.82]
Standard AR recipe, 100k steps (reference)		
Llama	0.85 [0.78, 0.86]	0.79 [0.71, 0.81]
GPT-2	0.87* [0.81, 0.88]	0.82 [0.77, 0.84]
Ceiling		
Two samples of real music	0.89	0.88

5.3 Attribute Control with REMI

In brief. With REMI, TONIC steers velocity and duration about as reliably as the best AR methods, but pitch register only in a very narrow range of strengths.

Table 5.4 gives the strict directional accuracy of every method at its operating point: the share of samples in which the attribute moved in the requested direction.

Table 5.4: Single-attribute control: share of samples in which the attribute moved in the requested direction (strict directional accuracy), on the 16 test prompts with 6 targets each. Bold: best in each column. Small grey numbers: the strength chosen on the tuning prompts (λ for TONIC, step norm for PPLM, s for tilting, N for best-of-N).

Method	Velocity	Duration	Pitch register
REMI			
TONIC	0.95 0.05	0.79 0.055	0.51 0.01
Llama PPLM	**0.99** 32	**0.88** 32	0.79 16
Llama tilting	0.97 1.0	0.68 1.0	**0.85** 0.75
GPT-2 tilting	0.98 1.0	0.65 1.0	**0.85** 0.75
Llama best-of-N	0.75 16	0.69 8	0.78 16
GPT-2 best-of-N	0.75 8	0.69 16	0.75 16
Anticipation (no velocity)			
TONIC		0.78 0.005	0.78 0.012
Llama PPLM		0.95 16	0.93 32
Llama best-of-N		**0.97** 8	**0.97** 8
GPT-2 best-of-N		0.94 8	0.94 8

Velocity and duration. TONIC steers both about as well as the best AR methods:

- **Velocity:** 95% of the samples move in the requested direction, statistically indistinguishable from PPLM, the gradient method closest to ours (99%), and from tilting (97–98%).
- **Duration:** 79%, again indistinguishable from PPLM (88%). TONIC is ahead of tilting (65–68%, by 0.12 to 0.15) and of compute-matched best-of-4 (by 0.24 to 0.26), with uncorrected p between 0.01 and 0.05; none of these differences survives the Holm correction over all 31 comparisons.

Best-of-N, which reranks unguided samples, needs eight to sixteen samples to reach 69–78%.

Pitch register. Here TONIC reaches only 51%, behind every baseline (75–85%), significantly so against tilting and Llama best-of-16. This reflects a very narrow usable range of strengths rather than an inability to steer:

- On the tuning prompts, $\lambda = 0.02$ reaches 85% accuracy, but its overlap with the model's unguided output is 0.749, just below the 0.75 required by the selection rule. The rule therefore falls back to $\lambda = 0.01$, which barely moves the attribute.
- At $\lambda = 0.02$, the test accuracy is 79%, the same as PPLM and best-of-16; at $\lambda \geq 0.04$ the music becomes cacophonous (Section 5.6).
- The regressor is not the cause: it predicts pitch register with $r = 0.986$ (Table 4.4).

We report the result of the pre-registered rule and this sensitivity alongside it: TONIC's usable window for pitch register is a single step of the strength grid, whereas the operating points of the AR methods transfer from tuning to test prompts without this problem.

How far the attribute moves. Directional accuracy hides how far the attribute moves. Figure 5.2 shows progress towards the target, split by direction:

- **TONIC is strongly asymmetric.** On duration, requests for longer notes reach and then overshoot the target as λ grows (progress above 1.5), while requests for shorter notes barely move (about 0.1–0.2). On velocity it is the other way round: softer requests overshoot and louder ones stall around half of the requested change.
- **The AR methods are more symmetric and never overshoot.** On duration they fall short in both directions; on velocity, tilting reaches the target in both directions at full strength.

Section 5.6 relates TONIC's asymmetry to how guidance changes the distribution of the attribute's values.

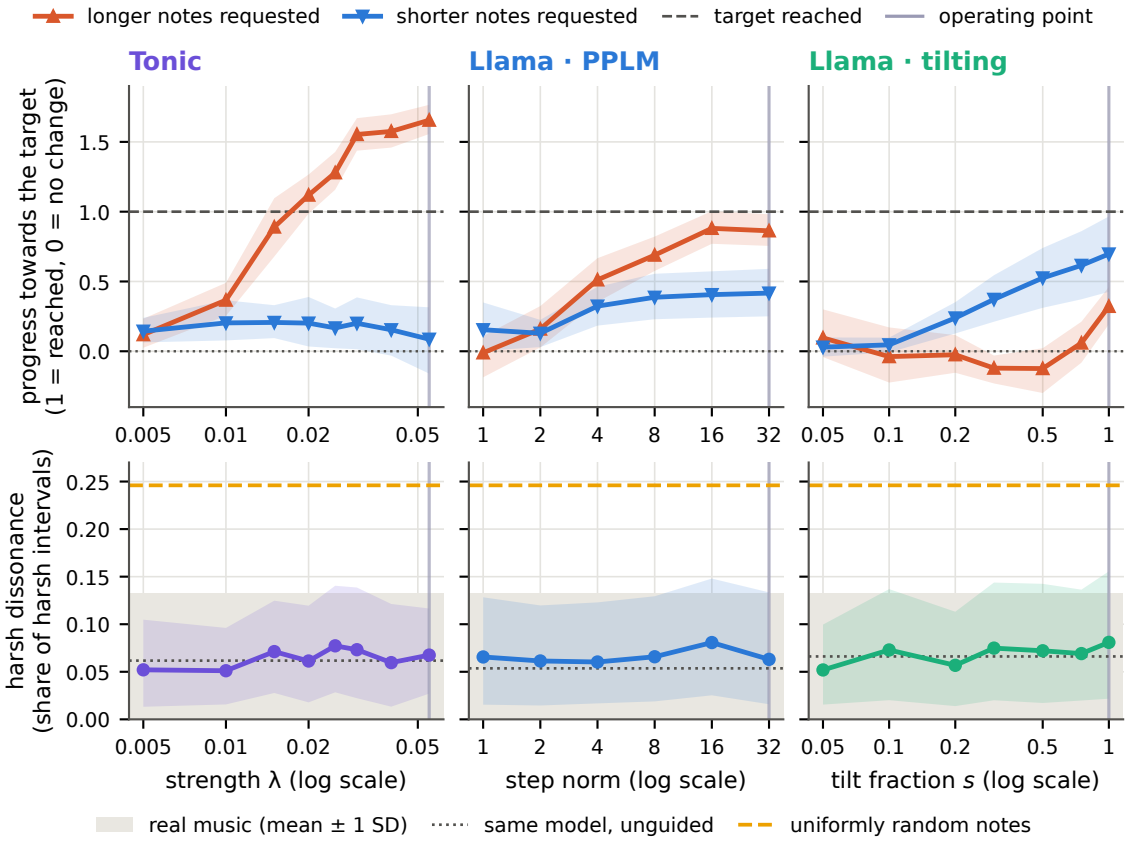


Figure 5.2: REMI duration: how far each method moves the attribute (top) and what this costs in harsh dissonance (bottom), as its strength grows. Top: progress towards the target, separately for requests for longer and for shorter notes (1 = target reached, 0 = no change, above 1 = overshoot). Bottom: share of harsh intervals, with real music as the grey band. Vertical line: the method's operating point. Shading: 95% bootstrap intervals over prompts. Best-of-N, not shown, stays below 0.35 in both directions at every N.

5.4 Attribute Control with Anticipation

> **In brief.** With Anticipation, TONIC steers both attributes in 78% of the samples, behind PPLM and best-of-8 (93–97%), although the gap is not significant after correction.

Compared with the baselines. With Anticipation, TONIC steers both duration and pitch register in the requested direction in 78% of the samples (Table 5.4). Here it is behind the AR baselines: PPLM and best-of-8 reach 93–97%, ahead of TONIC by 0.15 to 0.19 on both attributes. The direction is consistent across attributes and methods and the uncorrected p-values are between 0.004 and 0.03, but none of the differences is significant after the Holm correction. At the compute-matched $N = 4$, GPT-2 best-of-N is not clearly better than TONIC ($+0.09$ and -0.01), while Llama best-of-4 is ahead by 0.16 on both attributes (uncorrected $p \leq 0.05$, not significant after correction).

Usable strengths and regressor accuracy. Two further observations concern TONIC with Anticipation:

- Its usable strengths are small: for duration an order of magnitude smaller than with REMI ($\lambda = 0.005$ against 0.055), for pitch register 0.012, which is why we refined the grid at the low end (Section 4.3); above them the music degrades quickly (Section 5.6, Appendix B).
- The duration regressors, for TONIC and PPLM alike, predict about a third of the true value (Table 4.4): directions are unaffected, except for 4 of 96 requests, but how far guidance gets towards a duration target is less reliable for this attribute than for the others.

5.5 Combined Control

> **In brief.** TONIC steers two attributes at once about as well as PPLM and tilting, and far better than best-of-N, with no extra tuning.

Joint accuracy. Table 5.5 reports control of two attributes at once, with each attribute at its operating point from the single-attribute experiments and no other change. TONIC moves both attributes in the requested directions in 75% of the samples with REMI (velocity and duration) and 84% with Anticipation (duration and pitch register). This is statistically tied with PPLM (86% and 92%) and, with REMI, with tilting (76–77%); PPLM is numerically ahead, by 0.11 and 0.08, but not significantly after the Holm correction. Best-of-N is far behind and significantly worse than TONIC, at 34–56%: among unguided samples, few happen to satisfy two requests at once, even with eight to sixteen candidates and more compute than TONIC. At the compute-matched $N = 4$ it reaches only 25–50%.

Table 5.5: Controlling two attributes at once, each at its operating point. *Both*: share of samples in which both attributes moved in the requested direction, with 95% intervals; *opposite*: the same for requests in opposite directions, such as louder and shorter. *Dissonance*: change in harsh dissonance caused by guidance. *Significantly different from TONIC after correction.

Method	Both	Opposite	Dissonance
REMI: velocity and duration			
TONIC	0.75 [0.65, 0.86]	0.78	+0.009
Llama PPLM	0.86 [0.75, 0.95]	0.86	+0.015
Llama tilting	0.77 [0.62, 0.91]	0.75	+0.014
GPT-2 tilting	0.76 [0.60, 0.90]	0.73	+0.040
Llama best-of-16*	0.44 [0.23, 0.64]	0.34	+0.027
GPT-2 best-of-16*	0.34 [0.19, 0.50]	0.30	+0.006
Anticipation: duration and pitch register			
TONIC	0.84 [0.76, 0.92]	0.81	+0.041
Llama PPLM	0.92 [0.88, 0.97]	0.97	+0.009
Llama best-of-8*	0.56 [0.46, 0.66]	0.67	−0.006
GPT-2 best-of-8*	0.51 [0.44, 0.58]	0.48	−0.006

Opposite directions. Requests in opposite directions, such as louder and shorter, are as easy as requests in the same direction for the guided methods (TONIC 78% and 81%), but not for best-of-N with REMI (30–34%).

Two controls side by side. For TONIC, the joint accuracy is close to the product of the two per-attribute accuracies in the same runs ($0.94 \times 0.81 = 0.76$ with REMI, $0.90 \times 0.92 = 0.83$ with Anticipation): the two attributes are guided on different tokens of each note, so combined control runs two single-attribute controls side by side (Section 3.4).

Musical cost. The musical cost differs between the tokenizations. With REMI, TONIC raises harsh dissonance the least among the guided methods (+0.009, against +0.014 to +0.040 for PPLM and tilting). With Anticipation, it raises it the most, from 0.052 to 0.093, mainly because of the pitch-register strength; at the gentler strengths chosen by ear, dissonance is 0.049, within the range of real music, but both attributes move as requested in only 58% of the samples.

5.6 Control Strength versus Musicality

In brief. At high strength, TONIC changes the music far more than the AR methods: its attribute values become more random and drift towards those of random tokens, which is why guidance is easier in one direction than in the other.

Control against musical change. Figure 5.3 follows each method as its strength grows: up means the attribute moves in the requested direction more often, right means the music differs more from the same model's unguided output.

- **Velocity and duration:** TONIC's curves are close to those of PPLM and tilting, with high accuracy while the music stays close to the unguided output.
- **Pitch register:** TONIC's accuracy keeps rising only while the music changes drastically. At $\lambda \geq 0.04$ the overlap with the unguided output falls to 0.38–0.48 and harsh dissonance rises to 0.19–0.25, close to the 0.246 of uniformly random notes. The AR methods never reach that region.

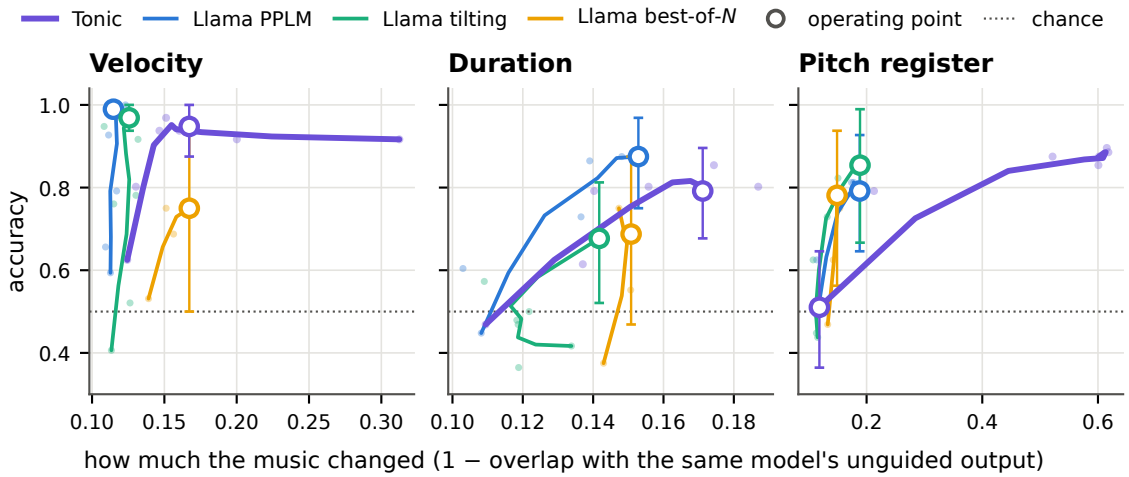


Figure 5.3: REMI: control against musical change. Dots: one per strength; lines: the same points in order of strength, smoothed over neighbouring strengths. Up: the attribute moves in the requested direction more often; right: the music differs more from what the same model writes unguided. A good method reaches the top while staying on the left. Rings: operating points, with 95% bootstrap intervals over prompts. GPT-2 behaves like Llama and is left out (Table 5.4).

Side effects at the operating points. At the chosen strengths, most side effects are small (Table B.1 in Appendix B), with three exceptions:

- **TONIC's velocity guidance sometimes breaks REMI's grammar:** 3.3% of positions violate the order pitch, velocity, duration, against none for the other methods, and the bigram log-likelihood drops by 0.39. A guided step can produce a token of the wrong type; tilting cannot, since it only reweights velocity tokens.
- **With Anticipation, TONIC's pitch-register guidance more than doubles harsh dissonance** (0.041 to 0.100), while PPLM and best-of-N keep it at or below their unguided level.

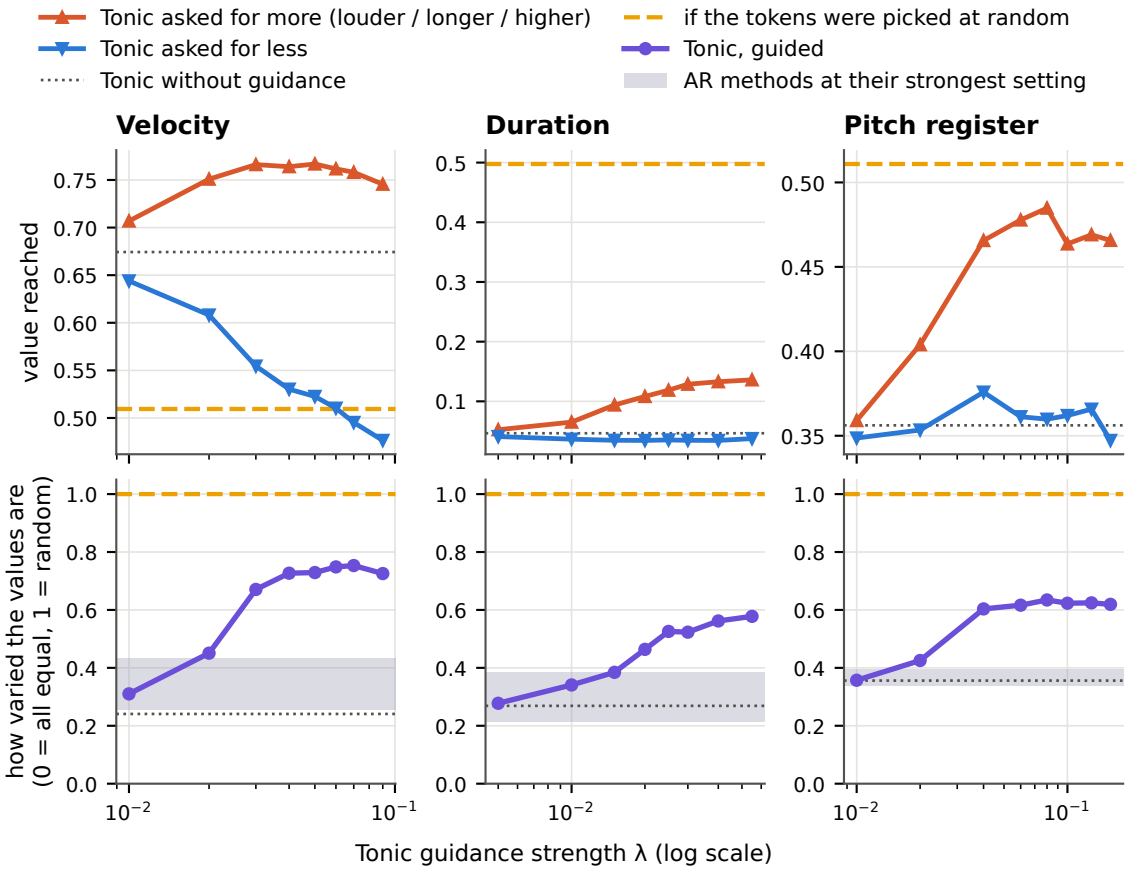


Figure 5.4: REMI: why TONIC's guidance is easier in one direction. Top: the attribute value TONIC reaches when asked for more or for less, as its strength grows; dotted: without guidance; dashed: if the attribute tokens were picked at random. Bottom: how varied the attribute's values are within a continuation, for TONIC and, as a grey band, for the AR methods at their strongest setting.

- **Key continuity with the prompt drops** by a few hundredths for most guided methods, most for TONIC on REMI duration (0.86 to 0.77) and for tilting on REMI pitch register.

Accuracy and error also rank the methods differently: tilting, which moves the attribute by a controlled amount, has the lowest error on velocity and pitch register.

Attribute values under strong guidance. Figure 5.4 shows two observations behind TONIC's asymmetry:

1. **Strong guidance makes the attribute's values more random** (bottom row). Their normalised entropy (Section 4.4.3) rises steadily with λ for every attribute and tokenization, from 0.24–0.47 to 0.58–0.76, whereas no AR method raises it by more than 0.19.

2. **The attribute therefore drifts towards the value of random tokens** (top row, dashed line), whatever direction was requested. Requests that point

towards that value are easy and the others stall: softer velocity (unguided 0.67, random 0.51), longer notes (about 0.05 against 0.5) and, with REMI, a higher register (0.36 against 0.51). With Anticipation, where the two values for pitch register almost coincide (0.47 and 0.50), both directions work.

We interpret these observations in Sections 6.1.3 and 6.1.4.

5.7 Listening Study

In brief. Listeners heard the requested change in most answers; TONIC's unguided REMI continuations were judged as musical as the originals, and no system's steered versions were clearly preferred.

The study (Section 4.6) collected 1,540 answered questions from 66 participants. Participants with extensive musical training did not answer differently in any reliable way from those without, and excluding those who failed the attention checks changes no result noticeably (Appendix C).

Unguided continuations (Part 1). Best–worst scores are shown in Fig. 5.5.

- **REMI: TONIC is rated as musical as the human originals** and clearly ahead of the AR baselines: +0.19 [+0.10, +0.28] for TONIC, +0.17 [+0.06, +0.27] for the originals, −0.11 for GPT-2 and −0.25 for Llama.
- **Anticipation: all four versions are close.** Llama +0.15, TONIC +0.01, the original −0.06, GPT-2 −0.11. The original's low score comes from three very dense clips (low brass in the bass, high strings with semitone clashes, overdriven guitar over repeated piano notes). They are rendered correctly; the corpus simply contains music that listeners do not find pleasant.



Figure 5.5: Listening study, Part 1: best–worst score of each system's unguided continuation (+1: always chosen as most musical, −1: always as least musical), with 95% bootstrap intervals over participants. About 190 appearances per system and tokenization.

Is the steering audible? (Part 2). Listeners heard the change in the requested direction in 61% [57, 64] of the answers, the opposite change in 20% and no difference in 19%. Among the answers that heard a change, the requested direction wins three to one (462 to 152; sign test $p < 10^{-36}$). Per-system results are in Fig. 5.6 and Table C.1.

- **TONIC and the AR baselines are heard equally often** (57% and 62%).
- **The tokenization matters more than the method:** 67% for TONIC and 65% for the baselines with REMI, 45% and 55% with Anticipation.
- **TONIC's clearest and weakest cases:** REMI velocity, heard in 83% [69, 97] of answers; Anticipation pitch register, 22%, which listeners did not hear.
- **Best-of-N is often the least audible:** REMI velocity with Llama best-of-N was heard as "no difference" in 68% of answers.
- **Neither the size of the change nor the listening setup matters much:** 64% for changes of 2 σ_a against 55% and 54% for 0.5 and 1 σ_a, likely because the selection kept small changes mainly when they were audible; 60–63% across headphones, laptop or phone speakers and external speakers.

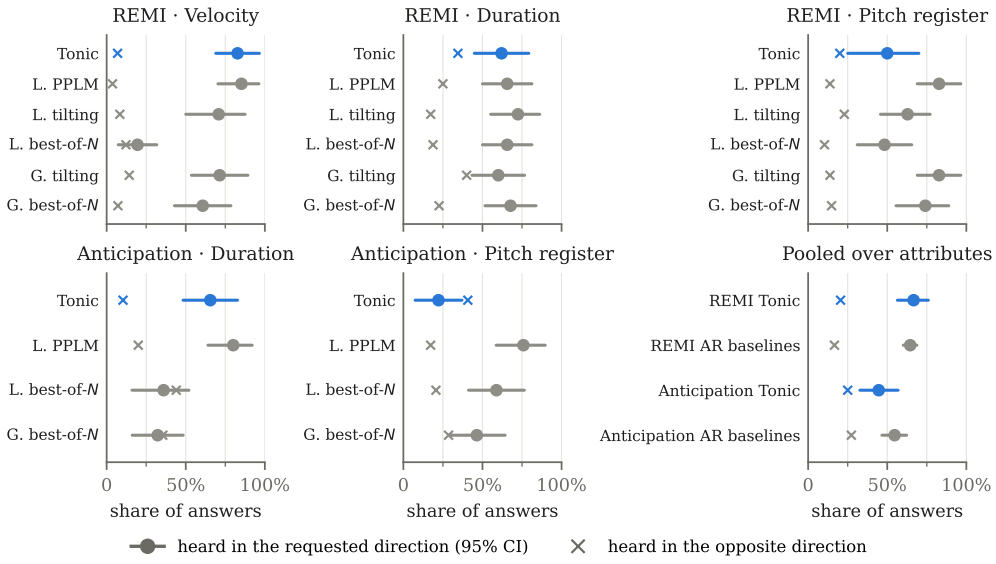


Figure 5.6: Listening study, Part 2: share of answers in which the steered version was heard as changed in the requested direction (dot, with 95% bootstrap interval) and in the opposite direction (cross), per tokenization, attribute and system (L. = Llama, G. = GPT-2). The remaining answers are "no difference". Bottom right: pooled over attributes. 20–41 answers per system and attribute.

Heard versus measured change. For each Part 2 answer we know how much the attribute actually moved (Table C.2).

- **Velocity and pitch register (REMI): perception follows the measurement.** The answer correlates with the measured change (Spearman $\rho = 0.40$ and 0.52). Clips that had not actually moved were heard in the requested direction in only 18% and 32% of answers, against 72% when they had.
- **Note length: no link.** REMI duration changes were heard in the requested direction in 66% of answers, but how strongly is unrelated to the measured change ($\rho = 0.07$); with Anticipation both correlations are weak (0.07 and 0.22). The mean duration does not capture what listeners hear as staccato or legato (Section 6.1.7).

Steered versions side by side (Part 3). No system was clearly preferred (Table C.3).

- **Tonic scores close to zero** with both tokenizations (+0.03 Anticipation, −0.06 REMI).
- **GPT-2 best-of-N is highest** (+0.15 and +0.12, the latter shared with Llama PPLM), with lower interval bounds near zero. Best-of-N only reranks unguided samples, so guidance costs little musicality in these examples.
- **Only two scores are clearly negative:** Llama PPLM with Anticipation (−0.32) and GPT-2 tilting with REMI (−0.16).

Cacophony. Within the Part 2 pairs, the same listener heard an unguided reference and its steered version (Fig. 5.7).

- **Steering adds little cacophony for Tonic:** 2 points with Anticipation and 5 with REMI, similar to best-of-N (1 to 7); PPLM and tilting add 6 to 13.
- **Steered Tonic was ticked least often** with both tokenizations (7% REMI, 5% Anticipation), although the intervals of single systems overlap.
- **The item works:** the near-random catch clip was ticked in 69% of the questions that contained it (41 of 59).

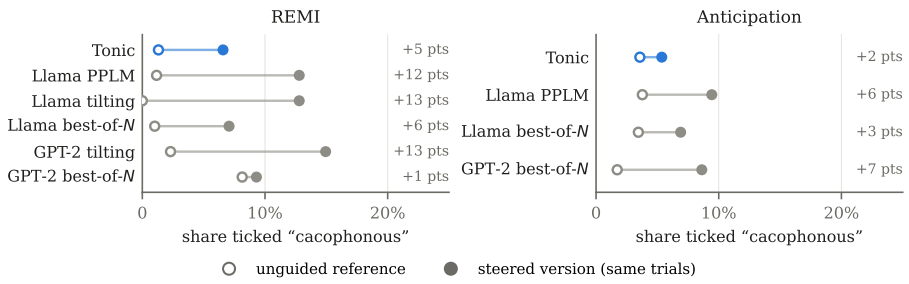


Figure 5.7: Share of versions ticked as cacophonous in the Part 2 pairs, unguided reference against steered version; right: the increase caused by steering. About 50–100 pairs per system.

Chapter 6

Discussion

6.1 Interpretation of Results

This section interprets the results topic by topic. Each topic opens with its main finding, followed by the evidence and our explanation. In brief:

- **Modelling music.** An EBT can model symbolic music about as well as AR Transformers trained the same way; where it falls behind, training most likely stopped too early to tell (Section 6.1.1).
- **The refinement loop as a place for control.** It works: a frozen EBT can be steered, singly and on two attributes at once, about as well as the best plug-and-play methods for AR models, and gently at its operating points. But it does not beat steering an AR model's output, which reaches the same accuracy at a sixth of the compute (Section 6.1.2).
- **How guidance fails.** Strong guidance flattens the model's choice among attribute tokens, because it acts while the prediction is still being formed; this explains both the asymmetric control and the cacophony at high strength (Section 6.1.3).
- **The role of the corpus.** The shape of each attribute's distribution in the corpus decides which direction of control is easy (Section 6.1.4).
- **Control is weaker with Anticipation.** With the Anticipation encoding, TONIC falls behind the AR methods and its guidance costs more dissonance, and the steering of every method is less audible; its large vocabulary, the early stage of training and the representation itself are the likely reasons (Section 6.1.5).
- **EBTs as "thinking" models.** Our EBTs do not gain from extra computation, as expected for the System 1 recipe we could afford; their energy is only reliable along the short path they were trained on (Section 6.1.6).
- **Measured versus heard change.** Changes in loudness and pitch register measured on the MIDI match what listeners heard; changes in note length do not (Section 6.1.7).

6.1.1 How Well TONIC Models Music

On REMI, TONIC models music about as well as the AR baselines trained with the same recipe. All three end within 0.02 nats of each other in context-matched loss (GPT-2 0.683, TONIC 0.696, Llama 0.700), and TONIC, which started behind, had just caught up with Llama when training stopped (Fig. 5.1). Their unguided continuations are equally close to real music on every automatic measure (Section 5.2). In the listening study, TONIC's REMI continuations were judged as musical as the human originals and more musical than those of the AR baselines (Section 5.7).

On Anticipation, TONIC is clearly behind (1.102 against 1.070 and 1.056), but this is a comparison early in training. Every REMI model drew a window from each training song about 63 times, whereas every Anticipation model saw only 39% of its training sequences once (Section 4.1.1 and Table 4.3). The gap also opened late: at about 1.5 billion tokens, on the same validation windows, a stabilised EBT variant (1.130) was level with Llama (1.126) and GPT-2 (1.141). TONIC seems to learn more slowly than the AR models later in a long first pass over a large, diverse dataset, but we cannot tell whether it would stay behind with more training (Section 7.3).

6.1.2 Control in the Refinement Loop

This subsection answers the central question of the thesis (RQ3 and RQ4). The refinement loop of an EBT works as a place for control, but it is not better than the output of an AR model.

What works. A frozen EBT can be steered without any retraining: all it takes is a few small regressors trained on the model's own embeddings. With REMI, this is enough for TONIC to match PPLM, the most similar AR method, on velocity and duration (Section 5.3). It also handles two attributes at once as well as PPLM and tilting, and far better than best-of-N, which rarely finds a sample that satisfies both requests (Section 5.5). And at its operating points it is gentle on the music. On REMI it adds the least harsh dissonance of the guided methods when controlling two attributes. In the listening study, its steered versions were ticked as cacophonous least often, while its changes were heard as often as those of the baselines (Section 5.7).

What does not. TONIC does not do better than an AR model steered at its output. PPLM, which takes a similar gradient step on the logits but only once, reaches the same accuracy or better at a sixth of the cost. TONIC's control is also fragile. With Anticipation it trails PPLM and best-of-N by 0.15 to 0.19, and its usable duration strength is ten times smaller than with REMI. On REMI pitch register, the range of strengths that works is a single step of the grid, which held-out tuning misses, and velocity guidance occasionally produces a token of the wrong type, breaking REMI's grammar (Section 5.6). The next two subsections explain where this fragility comes from.

Other music models. Steering other music models shows similar limits. Activation steering also works only within a narrow range of strengths, beyond which the music degrades [33, 52]. And the failure of best-of-N on two attributes is the reason search methods for music steer the search itself instead of only ranking its results [16, 15].

6.1.3 Why Strong Guidance Degrades the Music

Strong guidance makes TONIC's choice among attribute tokens nearly random. As λ grows, the entropy of the attribute's values rises by a factor of 1.6 to 3, towards that of random tokens, while no AR method raises it much even at its strongest setting (Section 5.6). This one effect explains three results:

- **One direction is much easier than the other.** Random tokens have a typical attribute value, so the attribute drifts towards it whatever the request. Requests in that direction reach the target and overshoot; requests in the other direction move only part of the way (about half for louder velocity, a tenth to a fifth for shorter notes).
- **Dissonance.** Near-random pitch choices give near-random intervals.
- **Broken grammar.** Near-random choices among all tokens at a velocity step give tokens of the wrong type.

Why this happens to TONIC and not to the AR methods. The difference lies in *when* each method steers the prediction. TONIC steers it while it is still being formed:

1. **The prediction starts blurred.** An EBT's prediction begins as random noise and only becomes sharp through its two refinement steps, driven by the gradient of the model's own energy.
2. **Guidance adds a second gradient to the same steps, and at the predicted position it is large.** Its strength is set relative to the model's gradient over the whole sequence (Eq. (3.3)), but it acts only on the one position being predicted. There it can match or exceed the model's own gradient, even for small λ.
3. **This gradient spreads probability instead of focusing it.** The regressor reads the prediction through its expected embedding, so its gradient raises every token with a suitable attribute value, not one plausible token.

The model's own gradient, which would have sharpened the prediction, is therefore outweighed before the prediction has formed. The AR methods, in contrast, steer a distribution that the model has already made sharp: PPLM shifts its logits by a bounded amount, tilting reweights it, and best-of-N does not touch it. When they fail, they fall short of the target rather than scramble the music.

Comparison with diffusion. In masked discrete diffusion, strong guidance does the opposite and concentrates probability on a single mode [58] (Section 2.5.5). The two findings do not contradict each other. Classifier-free guidance amplifies the difference between the model's own conditional and unconditional predictions, which sharpens them; our attribute gradient comes from outside the model and competes with its sharpening. This suggests that, in an iterative generator, the point at which guidance acts can shape how it fails, not only how strong it is.

6.1.4 Why One Direction Is Easier

Since strong guidance pulls an attribute towards its random-token value, the corpus decides which direction of control is easy. The models reproduce the corpus, so their unguided value sits where most of the corpus lies, while the random-token value depends only on the vocabulary. In all four cases where the two differ, the easy direction points from the first towards the second (Section 5.6). Here the corpus analysis of Section 4.1.2 explains the control results (contribution 1):

- **Velocity** is nearly symmetric around 0.70, well above the random-token value of 0.51: softer is easy, louder is hard.
- **Duration** is strongly right-skewed: most windows contain short notes, while a random duration token is long (about 0.5 on the normalised scale). Longer notes are easy and shorter ones hard. Shorter notes are hard twice over, because the distribution is bounded by zero just below its mode: a target of $-1\,\sigma_a$ is already at zero, below anything the model produces. On Anticipation, where σ_a is 2.6 times the attribute's own spread, the requested moves are also much larger relative to the data.
- **Pitch register** is concentrated around 0.45, with a longer tail towards low registers. The REMI test prompts sit lower than a random pitch token, so raising the register is easier; on Anticipation the two are almost equal, and both directions work.

The useful range of a control can therefore be predicted from the corpus before running any guidance: requests that point away from the random-token value, into a sparse tail of the distribution, will be the hardest.

6.1.5 Control with Anticipation

With Anticipation, TONIC falls behind the AR methods it ties with REMI, and every method's steering is harder to hear:

- TONIC moves the attribute as requested in 78% of the samples, against 93–97% for PPLM and best-of-N, which are as accurate as with REMI or more (Section 5.4).
- Its usable duration strength is ten times smaller than with REMI, and its pitch-register guidance more than doubles harsh dissonance (Table B.1).

- Listeners heard the requested change in 45% of the Anticipation answers for TONIC and 55% for the baselines, against 67% and 65% with REMI (Section 5.7).

There are three likely reasons, which we did not test directly.

- **Vocabulary.** The regressor's gradient spreads probability over every token with a suitable attribute value (Section 6.1.3), and with Anticipation far more tokens carry the attribute: duration has 1,000 finely spaced tokens instead of 64, and every pitch appears in up to 129 note tokens, one per instrument. Over so many tokens the choice flattens more easily, which would explain why much smaller strengths already degrade the music. The AR methods act on an already sharp distribution and are much less affected.
- **Stage of training.** The Anticipation models saw less than half an epoch of their data, and TONIC is further behind the baselines there (Section 6.1.1). Guidance relies on the shape of TONIC's energy landscape, which a less trained model has had less chance to form.
- **Representation.** Anticipation has no velocity and no metrical grid, and encodes duration in absolute time, so the mean duration also changes with tempo. How far it moved was only weakly related to what listeners heard (Section 5.7). The duration targets are also larger relative to the data than with REMI (Section 4.1.2), and the duration regressors were mislabelled (Section 6.2).

6.1.6 Thinking Longer

Our EBTs gain nothing from extra computation at inference. More refinement steps make predictions slightly worse while the energy keeps falling, and picking the lowest-energy of eight candidates gains nothing because all eight end up almost the same (Section 5.1). The model has learned a two-step path from noise to a prediction, not an energy whose minimum is the best prediction. This matches Gladstone et al. [19], who obtain gains from extra computation only with their System 2 recipe, and none when the number of steps is not randomised during training.

This matters for control too. Guidance pushes the prediction off the trained path, into regions of the energy landscape that the model never had to shape, which fits the narrow range of usable strengths and the flattening above. Adding a requirement to the energy and minimising both together assumes a landscape that is meaningful away from the model's own predictions. A System 1 EBT does not provide that; whether a System 2 EBT does is the main open question this thesis leaves (Section 7.3).

6.1.7 Measured and Heard Change

For velocity and pitch register on REMI, what listeners heard follows the change measured on the MIDI, and steered clips whose attribute had not actually moved were mostly not heard as changed (Section 5.7). For note length, listeners heard

many changes, but how strongly they heard them was unrelated to how much the mean duration had moved. Two explanations are likely:

- A mean over all notes can move because of one instrument, or because of the drums, while the impression of staccato or legato depends on the leading voices.
- Note length is heard relative to tempo and density, which also change between two continuations.

Duration results should therefore be read as control of a statistic of the score, which is related to, but not the same as, the articulation a listener hears.

6.2 Limitations

Combined control does not test composition. Summing energies is one of the main arguments for EBMs (Sections 2.4.2 and 3.4), but our combined control never sums two active energies. Guidance is gated by token type, and each pair of attributes acts on different tokens, so at every step at most one term of Eq. (3.4) is non-zero. Accordingly, the joint accuracy is close to the product of the single accuracies (REMI 0.75 against 0.76, Anticipation 0.84 against 0.83). The comparison with the AR methods, which act on the same steps, is fair, but it shows no advantage of composition. Preliminary runs with attributes on the same tokens (Section 4.7) suggest that summed energies can conflict when the attributes are correlated.

Training cost and model size. On one L40S GPU, our EBTs processed about 9 million tokens per hour, 10 to 36 times fewer than the AR baselines (Table 4.3); the two EBTs together took about 800 GPU hours. With a shared cluster that limits GPUs per user and preempts jobs (Section 4.8), this meant one small model size, a 512-token context for Tonic, no medium Anticipation EBT, and one seed per configuration. With a single size we cannot test the claim that EBTs scale faster than standard Transformers [19]. Per unit of compute, Tonic is clearly behind the AR baselines, so our case for EBTs rests on control, not on training efficiency.

System 1 EBTs only. Both EBTs use the System 1 recipe (Section 3.1). Our results cannot tell whether a System 2 EBT would be easier to steer, compose energies better, or benefit from thinking longer (Section 7.3).

Anticipation training data. Anticipatory augmentation makes the Anticipation training set about 107 times larger than REMI's, so its models saw only 39% of it; a full epoch would take Tonic about 790 GPU hours (Section 4.2). About 90% of the training sequences also carry anticipated controls, which we do not use. Both points apply to all three Anticipation models, so comparisons between them remain fair, but not necessarily comparisons with REMI.

Anticipation duration regressors. The duration regressors used on Anticipation by both Tonic and PPLM were trained on mislabelled windows and predict about a third of the true duration (Section 4.2). They still order durations correctly, so guidance pushes the right way but has no reliable stopping point. The comparison between the two methods is unaffected.

Listening study. With 66 participants, the study is large enough to show clear effects, such as whether a change is heard, but some differences between single systems remain within the intervals. It did not include combined control, and the instructions did not say whether to judge the music as a whole or each instrument, an ambiguity that per-instrument attributes would remove (Section 7.3).

Chapter 7

Conclusion

This thesis asked whether Energy-Based Transformers, which predict by minimising a learned energy, can generate music, and whether their refinement loop is a better place to control music generation than the autoregressive (AR) Transformers that dominate the field. We trained TONIC, the first EBTs for symbolic music, on GigaMIDI in two tokenizations, together with GPT-2 and Llama baselines trained with exactly the same recipe. We then steered the frozen TONIC towards target values of note velocity, note duration and pitch register by adding the energy of a small attribute regressor to its refinement loop, for one attribute and for two at once, and compared it with three plug-and-play methods for the AR models under one protocol, with operating points chosen on held-out prompts, and in a listening study.

TONIC generates music well: it trains stably, models music about as well as AR Transformers trained the same way, and listeners judged its REMI continuations as musical as the human originals. Its refinement loop is also a workable place for control. TONIC can be steered without retraining, on one attribute or two, about as accurately as the best plug-and-play methods and with little harm to the music. Steering an AR model's output with a gradient step reaches similar accuracy at a lower cost, and TONIC works best within a narrow range of strengths and with the REMI encoding. We relate this to how guidance interacts with an EBT trained, as ours are, with the stable System 1 recipe, which points to System 2 training as the natural next step.

7.1 Answers to the Research Questions

RQ1: Can an EBT be trained on symbolic music and generate coherent continuations? Yes. After fixing an instability in its learnable step size, both TONIC models trained stably (Section 4.2). Their unguided continuations are close to real music by every automatic measure, with an overlap of 0.83 (REMI) and 0.78 (Anticipation) against ceilings of 0.89 and 0.88, not significantly different from the AR baselines (Section 5.2); and listeners judged TONIC's selected REMI continuations as musical as the human originals (Section 5.7).

RQ2: How does it compare with AR Transformers of the same size, trained on the same data with the same recipe? On REMI it is on par: a validation loss of 0.696 against 0.700 for Llama and 0.683 for GPT-2, after catching up with Llama during training. On Anticipation it is behind, 1.102 against 1.056 and 1.070, a gap that opened late in a run that covered less than half an epoch

of the data (Section 5.1). It is far more expensive to train, 17 to 36 times slower per token on REMI (Table 4.3), and, as a System 1 EBT, it gains nothing from spending more computation at inference, neither from more refinement steps nor from choosing among several candidates.

RQ3: Can a frozen EBT be steered towards target values of musical attributes, singly and in combination, without retraining? Yes. With REMI, guidance moves the attribute in the requested direction in 95% of the samples for velocity and 79% for duration; with Anticipation, in 78% for both duration and pitch register (Table 5.4). Two attributes can be steered at once with the same regressors and no other change: both moved as requested in 75% (REMI) and 84% (Anticipation) of the samples (Section 5.5). The exception is REMI pitch register, whose usable range of strengths is a single step of our grid: held-out tuning misses it (51%), while the next strength reaches 79% (Section 5.3).

RQ4: How does this compare with plug-and-play steering of AR models under the same constraints?

Accuracy. At its operating points, TONIC is statistically tied with the best AR methods on REMI, PPLM and expectation tilting, for single attributes and for two at once. It is far ahead of best-of-N when two attributes are requested (by 0.29 to 0.50 in joint accuracy). On Anticipation it trails PPLM and best-of-N by 0.15 to 0.19, consistently but not significantly after correction.

Effect on the music. On REMI, TONIC is gentle: its guidance adds the least harsh dissonance among the guided methods in combined control, and listeners ticked its steered versions as cacophonous least often while hearing the changes as often as for the baselines.

Costs. TONIC has costs the AR methods do not. Its Anticipation pitch-register guidance doubles harsh dissonance, its velocity guidance occasionally breaks REMI's token grammar, and it needs about six forward passes per token, against one for PPLM and tilting (Sections 3.6 and 5.6).

7.2 Further Insights

Beyond the answers to the research questions, the results offer the following insights, each discussed in Chapter 6.

The refinement loop is a viable place for control, though not necessarily a better one. Adding an attribute energy to an EBT's own minimisation works, and it handles several attributes as naturally as one. But in practice it behaves like a normalised gradient step on the prediction, which an AR model's output also admits, so with our models its benefits over that simpler option are not yet visible (Section 6.1.2).

When guidance acts shapes how it fails. TONIC's guidance acts while the prediction is still being formed from noise. Strong guidance therefore flattens the choice among attribute tokens towards random, instead of shifting an already sharp choice; this one effect explains the asymmetric control, the cacophony at high strength and the grammar violations, and it is the opposite of the concentration reported for guided diffusion (Section 6.1.3).

The corpus decides which direction is easy. Because strong guidance pulls an attribute towards the value of random tokens, requests that point from the corpus's usual value towards it are easy, and requests that point away from it, into a sparse tail of the distribution, are hard. The useful range of a control can be predicted from the corpus before any guidance is run (Section 6.1.4).

The representation matters as much as the method. With the Anticipation encoding, steering is less audible for every method, and TONIC falls behind the AR methods it ties with REMI. Its prediction is spread over a vocabulary in which far more tokens carry each attribute: 1,000 finely spaced duration tokens instead of 64, and up to 129 tokens for each pitch (Section 6.1.5).

A System 1 EBT shows only part of what energy-based models promise. Composition, verification and thinking longer all require an energy that is meaningful away from the model's own predictions. Our EBTs, trained with the stable System 1 recipe, learn a short refinement path instead; this is consistent with their lack of gain from extra computation and with the narrow range in which they can be steered (Section 6.1.6).

What is measured is not always what is heard. For loudness and pitch, what listeners heard follows the measured change; for note length, the mean duration of a passage did not predict what listeners perceived (Section 6.1.7).

7.3 Future Work

Working on this thesis brought up many more directions than we could follow. Energy-based models for music, and the ways to control them, are still largely unexplored, and several of these directions look promising. Many follow from a limitation discussed in Section 6.2. The first three need new training runs; the others mainly new experiments with the existing models.

1. **Composing energies on the same tokens.** Composition is one of the main arguments for energy-based models, but in our combined-control experiments every pair of attributes acts on different tokens, so their energies are never summed at the same step. The direct test is a pair of attributes guided at the same tokens, such as note density and polyphony, which are both guided at duration and position tokens, or pitch register and drum density, both guided at instrument tokens. We made a few preliminary runs with these two

pairs on an earlier checkpoint (Section 4.7). The energies combined well when the attributes were independent or asked to move the same way, but when density and polyphony were asked to move in opposite directions, polyphony prevailed and density followed it. This needs to be explored further on the final checkpoints, with more prompts, and with ways to balance the energies, such as weighting them or a System 2 model with a better-shaped energy landscape (below).

2. **A System 2 Tonic.** Our EBTs are System 1 models in the sense of Gladstone et al. [19]: two refinement steps and no regularisation. They learn a short refinement path rather than an energy whose minimum is the best prediction, and gain nothing from more steps or from choosing among several candidates (Section 5.1). The natural next step is a System 2 Tonic, trained with a random number of steps, randomised step sizes, Langevin noise and a replay buffer. Several of the weaknesses we found trace back to an energy landscape that is only shaped along a short path, and a System 2 model addresses exactly that:

   - *Extra computation that helps.* With the System 2 recipe, Gladstone et al. [19] obtain better predictions from more steps and from keeping the lowest-energy of several candidates. For music, the second would let the model reject an implausible note before committing to it, acting as its own verifier.
   - *A wider range of usable strengths.* Guidance pushes the prediction away from the model's usual path, into regions a System 1 model never had to shape. Langevin noise and the replay buffer train exactly those regions, which may widen the very narrow strength window of pitch register (Section 5.3), weaken the flattening behind the asymmetric control and the dissonance at high strength (Section 5.6), and let guidance act gradually over more steps rather than in one or two large ones.
   - *Composition.* Summing two energies at the same step, the test our combined control could not perform, is most likely to find a sensible compromise between the attributes when both energies are well shaped around the prediction; a System 2 Tonic is where the composition experiment above is most worth running.
   - *Calibration.* Our partly System 2 Anticipation EBT was better calibrated than the System 1 one (Section B.1), and an energy that reflects uncertainty could be used to decide where guidance is safe.

   The System 2 recipe needs more memory per example, already our limiting factor, so it would require gradient checkpointing through the refinement loop or larger GPUs, and Gladstone et al. [19] report that such models only catch up at scale. A first step costs no training: the partly System 2 Anticipation EBT we already have could be guided with the same protocol.

3. **Scale, training length and seeds.** All our models are small and were trained once, for a limited budget.

- **Longer training.** With REMI, all models were still improving when training stopped, and TONIC had only just caught up with Llama (Fig. 5.1); with Anticipation, no model completed an epoch. Since Gladstone et al. [19] report that EBTs improve faster with more data, the gap to the AR models could narrow with longer training. Two observations call for care: the Anticipation EBT became unstable late in training (Section 4.2), and longer training made our System 1 model more rigid, with the loss increase from extra steps growing from 0.010 to 0.019 (Section 5.1). Longer training is therefore most promising together with the System 2 recipe.
- **A full epoch on Anticipation**, or training on the unaugmented data, would show whether TONIC's late slow-down there persists (Section 6.1.1).
- **Several model sizes** would show how the gap between EBTs and AR models changes with scale.
- **Several seeds** per model would let the statistical tests account for variation between training runs.

4. **Attributes closer to how music is heard.** Our attributes are averages over all notes, so "louder" is met across the whole arrangement, whereas musicians think in parts: louder strings, a higher melody. Per-instrument or per-voice attributes would allow this, and a duration measure restricted to the leading voices, or relative to tempo, may track perceived note length better (Section 5.7).

5. **Other ways of steering.** Comparing TONIC's guidance with activation steering (Section 3.5) would place the refinement loop among all plug-and-play methods.

6. **Plug-and-play steering of AR models.** The AR methods we used as baselines worked very well on our low-level attributes: PPLM reached TONIC's accuracy at a sixth of the compute, and tilting reached the target in both directions on velocity (Section 5.3). They deserve further study in their own right, for instance on more attributes, such as note density, polyphony or per-instrument loudness, with an adaptive strength that corrects the persistent shortfall we observed [51], or by combining a gradient step with tilting.

7. **Listening studies.** Our listening study used clips selected by ear and single attributes only. A second study should test combined control, that is, whether listeners hear two changes made at once and whether combining attributes costs more musicality than steering one. A controlled study, with prompts chosen by a fixed rule and every system at its held-out operating point, would compare the systems on average rather than at their best; such a design was prepared during this work but set aside in favour of the curated study.

8. **Using the Anticipation models as they were trained.** About 90% of the Anticipation training sequences carry anticipated controls, but we used

the models only for plain continuation. Their native tasks, infilling and accompaniment conditioned on given notes, could be combined with attribute guidance, for instance to generate an accompaniment that is softer or more legato than the melody it follows.

9. **Interactive use.** EBT guidance needs no retraining and can change targets at every step, which suits interactive settings: a musician adjusting loudness or register while the model plays, or shaping a continuation phrase by phrase. Making this fast enough for real time, for instance with fewer refinement steps or a distilled model, would turn the models from a generator into an instrument.

7.4 Closing Remarks

Energy-based models offer an appealing idea for control: write each requirement as an energy, add it to the model's own, and let the model find music that satisfies all of them. This thesis shows that the idea works for music. An EBT can be trained on symbolic music, generates continuations that listeners find as musical as real pieces, and its refinement loop accepts musical requirements without any retraining.

In our experiments, this control is as accurate as steering an autoregressive model, but not yet more so. Our analysis suggests why: a System 1 EBT shapes its energy landscape only along a short path, and guidance has to leave that path. This is a property of how our models were trained rather than a limit of the approach. EBTs trained with the System 2 recipe, whose landscape is meaningful around their predictions, trained for longer and at a larger scale, may be steered more gradually and over a wider range, and may combine several requirements at the same notes, which our experiments could not yet test.

Such models would be especially promising for interpretable music generation. In an EBT, every requirement is an explicit term in the energy, and the energy itself says how well a passage fits. A musician could see which request pulls the music where and adjust it without retraining anything; with attributes that follow how music is heard, such as the loudness of one instrument, the model would respond to the terms a musician uses. Control would then be part of how the music is made, open to inspection at every step, rather than a black box.

We release all our code, the generated music and the accompanying website publicly, in the hope that others will build on this work, and that it helps shape a new kind of collaboration between musicians and AI, in which generative models are tools that musicians can understand, shape and play with.

Appendix A

Anticipation Tokenization Pipeline for GigaMIDI

This appendix describes the changes made to the Anticipation library [67] to tokenize GigaMIDI (Section 4.1.1). The library has been developed in three stages: the original implementation by Thickstun et al. [67]; a fork adapted for the JAM_BOT [4], which among other things removed the upper limit on track length and added drum conditioning to the MIDI preprocessing; and our fork of that version, which this appendix describes.

The fork is released as an open-source repository[1], whose README gives detailed instructions for preprocessing and tokenizing GigaMIDI, resuming interrupted jobs and selecting the vocabulary, and lists the changes made at each stage.

A.1 GigaMIDI Driver

The original tokenization script hard-codes the Lakh MIDI dataset's split structure. We added a driver for GigaMIDI that finds the preprocessed files of each split (train, validation, test) recursively under a configurable dataset root and writes one token file per split. Splits can be processed together or one at a time, and the files are sorted so that the output is deterministic. The augmentation factor is a parameter; we apply it to all splits (Section 4.1.1). A second tokenizer variant can also be selected with an environment variable: a "vanilla" vocabulary without the control block (27,515 instead of 55,028 tokens), for purely autoregressive models. It was implemented and tokenized but not used for the models in this thesis.

A.2 Resumable Tokenization on a Shared Cluster

At GigaMIDI's scale (about 85 GB of tokens for the training split alone), tokenizing a split takes longer than the maximum wall-clock time of a job on the cluster. We therefore made tokenization resumable:

- Each split saves a checkpoint after every 5% of its input files, recording how many files have been processed.
- When resumed, the tokenizer skips the processed files and *appends* to the existing output file instead of overwriting it.

[1] `https://github.com/RebeccaCeccatelli/anticipation-musicEBT`

- A submission script for SLURM clusters, in the main code repository, can first download and preprocess GigaMIDI as a separate job; it then processes the splits one after another, monitors each job, and when a job hits the time limit, resubmits it in resume mode, up to a configurable number of attempts. The data location, partition, time limit and resources are parameters, so it can be adapted to other clusters.

The number of files converted and discarded during preprocessing, the progress of each split, and the number of sequences written per split are logged to Weights & Biases, in the public project `Giga-Midi-Tokenization`[2]. Logging is optional: without a W&B account or network access, the pipeline runs unchanged.

A.3 Packaging

The library was originally a set of stand-alone scripts. To import it from the training code, we turned it into a Python package (module files renamed so that they can be imported, package markers added, imports adjusted), made its requirements compatible with the rest of the project, and removed hard-coded paths, so that the data locations are passed in by the caller.

[2] `https://wandb.ai/rceccatelli-eth-z-rich/Giga-Midi-Tokenization`

Appendix B

Additional Results

This appendix first gives details of the language-modelling results (Section B.1), then repeats two figures of Section 5.6 for the Anticipation models (Section B.2). They are read in the same way as their REMI versions and lead to the same conclusions, with two differences noted below. Anticipation has no velocity, so each figure has two panels: duration and pitch register. The appendix ends with the side effects of every method at its operating point (Table B.1).

B.1 Language Modelling

Comparing the tokenizations. The losses of the two tokenizations are not comparable with each other: Anticipation's vocabulary is 129 times larger, and most of its higher loss is explained by that size. Measured per note instead of per token, the two are close: a REMI note takes 4.3 tokens on average in the windows of Table 5.1 and an Anticipation event exactly three, so TONIC needs about 3.0 nats per note with REMI and 3.3 per event with Anticipation (the baselines 3.0 and 3.2). This is only a rough comparison, since Anticipation encodes timing more finely and its events include the anticipated controls.

System 2 regularisers and calibration. The Anticipation EBT trained with three of the System 2 regularisers (Section 4.2) loses about half as much from extra refinement steps as the original at a comparable stage of training (+0.006 against +0.010), but it too is best at two steps; it lacks the fourth ingredient, a random number of steps. It is also better calibrated: at two steps its loss and the entropy of its predictions nearly coincide (a gap of −0.007 nats, slightly underconfident), whereas the original is overconfident by 0.051 nats; for both, the gap grows with extra steps.

A smaller step size. Extra steps with a smaller step size cannot be tested meaningfully with these models: each step turns its input into probabilities and subtracts the scaled gradient, so the learned step size also sets how sharp the output is, and a smaller one simply blurs the prediction.

B.2 Guidance

Control versus musical change (Fig. B.1). Each dot is one guidance strength, and each line follows a method through its strengths, smoothed as in Fig. 5.3. Higher

means the attribute moved in the requested direction more often; further right means the music departs more from what the same model writes without guidance. A good method reaches the top while staying on the left. As with REMI (Fig. 5.3), the AR methods reach high accuracy while barely changing the music, whereas TONIC gains accuracy only by changing it more. For duration, its accuracy even falls again at high strength.

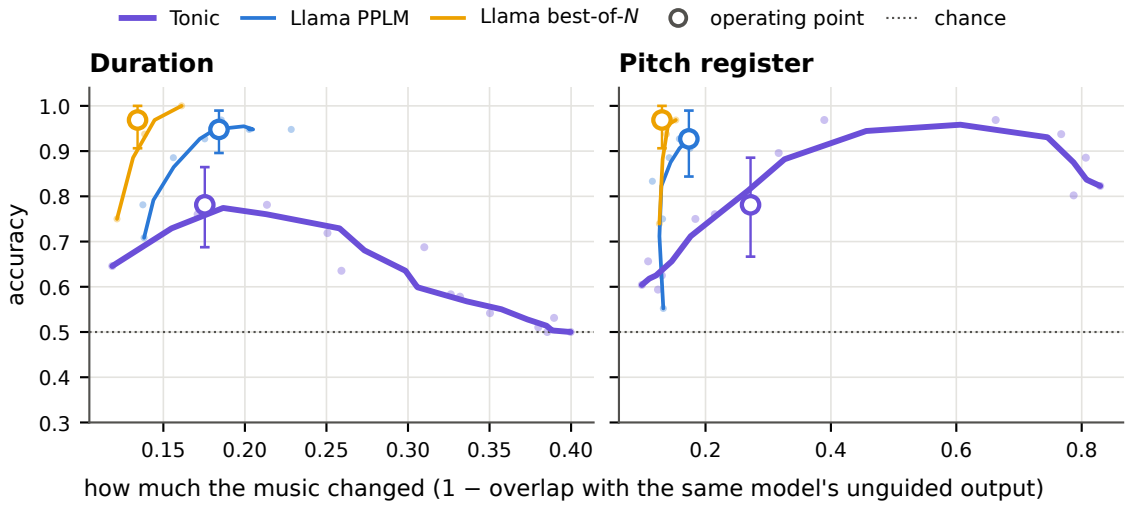


Figure B.1: Anticipation: control against musical change, read as Fig. 5.3. Up: the attribute moves in the requested direction more often; right: the music differs more from the same model's unguided output. Rings: operating points, with 95% bootstrap intervals over prompts. GPT-2 is left out, as in Fig. 5.3.

Why guidance is asymmetric (Fig. B.2). The top row shows the attribute value TONIC achieves when asked for more or for less, as its strength grows. The bottom row shows how spread out the attribute's values are within a continuation, from 0 (all the same) to 1 (uniformly random). As with REMI (Fig. 5.4), strong guidance makes the values more random, and the achieved value drifts towards that of uniformly random tokens (dashed line), whatever was requested. The AR methods stay at or below their unguided spread. Two things differ from REMI:

- requests for *shorter* notes end with *longer* notes than unguided at high strength;
- pitch register can be steered in both directions, because its random-token value (0.50) is close to its unguided value (0.47).

Side effects at the operating points (Table B.1). For every method at its operating point, the table gives the error to the target and the musical side effects of guidance: harsh dissonance, overlap with real music and with the same model's unguided output, key continuity with the prompt and, for REMI, the bigram log-likelihood and grammar violations. Section 5.6 summarises the exceptions.

Table B.1: Error and musical quality at each method's operating point (test prompts). MAE: mean absolute error to the target (attribute units). Harsh: harsh dissonance of the method's unguided → guided samples (real music: mean 0.05 with REMI and 0.07 with Anticipation, quality budget 0.13 and 0.12; random notes 0.25). OA: overlap with real music and with the same model's unguided output (lower = more change). Key: key continuity with the prompt, unguided → guided. Δbigram: change in bigram log-likelihood; Gram.: grammar violation rate (REMI only).

Attribute	Method	MAE	Harsh	OA real	OA ung.	Key	Δbigram	Gram.
REMI								
Velocity	Tonic	0.046	0.053 → 0.049	0.74	0.83	0.83 → 0.83	−0.39	0.033
	Llama PPLM	0.054	0.054 → 0.060	0.68	0.89	0.78 → 0.79	+0.03	0.000
	Llama tilting	0.032	0.066 → 0.059	0.70	0.87	0.81 → 0.78	−0.10	0.000
	GPT-2 tilting	0.031	0.064 → 0.059	0.70	0.86	0.76 → 0.83	−0.11	0.000
	Llama best-of-N	0.125	0.052 → 0.088	0.65	0.83	0.82 → 0.79	+0.02	0.000
	GPT-2 best-of-N	0.136	0.064 → 0.065	0.68	0.87	0.82 → 0.83	−0.03	0.000
Duration	Tonic	0.040	0.062 → 0.068	0.72	0.83	0.86 → 0.77	−0.20	0.002
	Llama PPLM	0.029	0.054 → 0.063	0.73	0.85	0.78 → 0.82	+0.04	0.000
	Llama tilting	0.040	0.066 → 0.081	0.69	0.86	0.81 → 0.77	+0.01	0.000
	GPT-2 tilting	0.039	0.064 → 0.088	0.71	0.87	0.76 → 0.77	−0.03	0.000
	Llama best-of-N	0.049	0.052 → 0.067	0.68	0.85	0.82 → 0.82	+0.02	0.000
	GPT-2 best-of-N	0.047	0.064 → 0.073	0.70	0.86	0.82 → 0.79	−0.02	0.000
Pitch register	Tonic	0.078	0.056 → 0.057	0.73	0.88	0.81 → 0.80	+0.01	0.000
	Llama PPLM	0.064	0.054 → 0.048	0.77	0.81	0.78 → 0.72	−0.01	0.000
	Llama tilting	0.033	0.066 → 0.058	0.73	0.81	0.81 → 0.67	−0.24	0.000
	GPT-2 tilting	0.032	0.064 → 0.069	0.74	0.79	0.76 → 0.69	−0.32	0.000
	Llama best-of-N	0.059	0.052 → 0.065	0.66	0.85	0.82 → 0.77	+0.02	0.000
	GPT-2 best-of-N	0.062	0.064 → 0.060	0.71	0.83	0.82 → 0.79	−0.05	0.000
Anticipation								
Duration	Tonic	0.053	0.040 → 0.035	0.79	0.82	0.73 → 0.67	—	—
	Llama PPLM	0.047	0.065 → 0.043	0.74	0.82	0.68 → 0.69	—	—
	Llama best-of-N	0.056	0.055 → 0.056	0.74	0.87	0.72 → 0.70	—	—
	GPT-2 best-of-N	0.054	0.054 → 0.052	0.73	0.87	0.70 → 0.71	—	—
Pitch register	Tonic	0.054	0.041 → 0.100	0.70	0.73	0.72 → 0.64	—	—
	Llama PPLM	0.038	0.065 → 0.044	0.78	0.83	0.68 → 0.61	—	—
	Llama best-of-N	0.055	0.055 → 0.045	0.73	0.87	0.72 → 0.65	—	—
	GPT-2 best-of-N	0.053	0.054 → 0.055	0.74	0.88	0.70 → 0.69	—	—

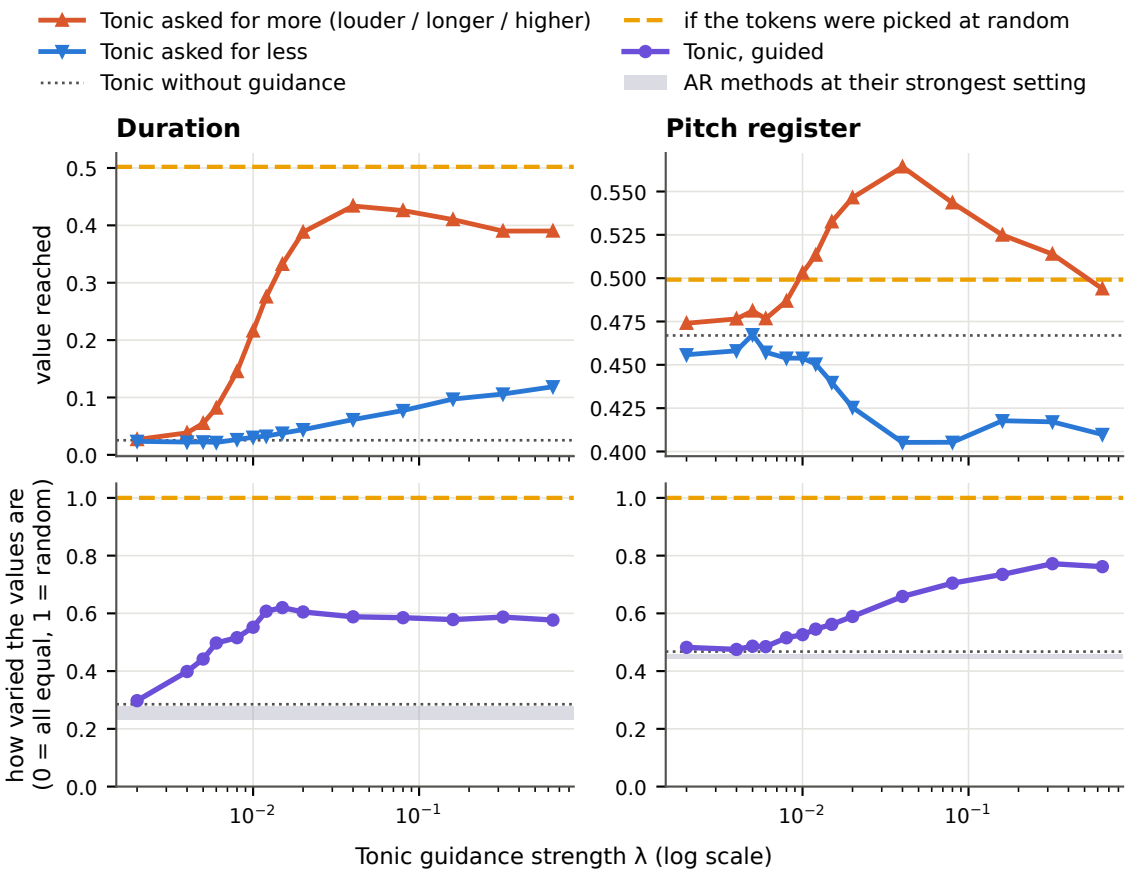


Figure B.2: Anticipation: TONIC's achieved attribute value for requests up and down (top) and the spread of the attribute's values (bottom) as its strength grows. Dotted: unguided. Dashed: uniformly random attribute tokens. Grey band: the AR methods at their strongest setting.

Appendix C

Listening Study: Detailed Results

This appendix gives the detailed numbers behind Section 5.7. Table C.1 breaks Part 2 down by tokenization, attribute and system, and Table C.2 compares what listeners heard with the change measured in the MIDI. Table C.3 gives the Part 3 scores of every steered system. Table C.4 compares the answers of participants with different musical training, and Table C.5 repeats the key results with stricter participant sets. All intervals are 95% bootstrap intervals over participants.

Musical training. Participants with extensive training did not answer differently in any reliable way from those without (Table C.4). They were, if anything, more critical and more accurate: they ticked the near-random clip as cacophonous more often (89% against 57%), always answered "no difference" to the identical pair, and rated the REMI originals higher (+0.37 against +0.11). They heard the steering in the requested direction exactly as often (59% in both groups). Of the 24 measures compared, two have an interval that excludes zero (both cacophony ticks on unguided clips, which the trained group ticked more often), about what chance alone would produce.

Robustness. Excluding the 7 participants who failed the broken-clip check, or the participant flagged for their cacophony ticks, changes no score by more than 0.05 and no Part 2 share by more than 1 point (Table C.5).

Comments. In the optional comments, participants found the task subjective ("more about picking between multiple plausible options than kicking out all the bad ones"), long, or harder with dense music than it would be with classical music; one noted that some continuations kept the same pace for too long, where a human player would have sped up or slowed down. One suggested a separate button to replay only the prompt, and one did not understand what the versions in Part 3 had been changed from. Another found that, up to a point, melodies with more instruments and more varied notes sounded more musical.

Tokenizer	Attribute	System	Signed (CI)	Right % (CI)	Opp. %	None %	n
REMI	Velocity	Tonic	+1.31 [+0.90, +1.66]	83 [69, 97]	7	10	29
REMI	Velocity	Llama PPLM	+1.41 [+1.07, +1.70]	85 [70, 96]	4	11	27
REMI	Velocity	Llama tilting	+1.17 [+0.71, +1.58]	71 [50, 88]	8	21	24
REMI	Velocity	Llama best-of-N	+0.07 [−0.12, +0.27]	20 [7, 32]	12	68	41
REMI	Velocity	GPT-2 tilting	+0.96 [+0.57, +1.32]	71 [54, 89]	14	14	28
REMI	Velocity	GPT-2 best-of-N	+0.71 [+0.36, +1.07]	61 [43, 79]	7	32	28
REMI	Duration	Tonic	+0.59 [+0.00, +1.14]	62 [45, 79]	34	3	29
REMI	Duration	Llama PPLM	+0.66 [+0.09, +1.22]	66 [50, 81]	25	9	32
REMI	Duration	Llama tilting	+0.93 [+0.34, +1.41]	72 [55, 86]	17	10	29
REMI	Duration	Llama best-of-N	+0.69 [+0.22, +1.09]	66 [50, 81]	19	16	32
REMI	Duration	GPT-2 tilting	+0.43 [−0.17, +1.00]	60 [43, 77]	40	0	30
REMI	Duration	GPT-2 best-of-N	+0.90 [+0.42, +1.32]	68 [52, 84]	23	10	31
REMI	Pitch register	Tonic	+0.50 [+0.05, +0.95]	50 [25, 70]	20	30	20
REMI	Pitch register	Llama PPLM	+1.31 [+0.83, +1.69]	83 [69, 97]	14	3	29
REMI	Pitch register	Llama tilting	+0.83 [+0.37, +1.26]	63 [46, 77]	23	14	35
REMI	Pitch register	Llama best-of-N	+0.62 [+0.28, +0.97]	48 [31, 66]	10	41	29
REMI	Pitch register	GPT-2 tilting	+1.31 [+0.76, +1.79]	83 [69, 97]	14	3	29
REMI	Pitch register	GPT-2 best-of-N	+1.04 [+0.59, +1.41]	74 [56, 89]	15	11	27
Anticipation	Duration	Tonic	+0.90 [+0.52, +1.24]	66 [48, 83]	10	24	29
Anticipation	Duration	Llama PPLM	+0.80 [+0.32, +1.20]	80 [64, 92]	20	0	25
Anticipation	Duration	Llama best-of-N	−0.28 [−0.72, +0.16]	36 [16, 52]	44	20	25
Anticipation	Duration	GPT-2 best-of-N	+0.10 [−0.32, +0.52]	32 [16, 48]	35	32	31
Anticipation	Pitch register	Tonic	−0.15 [−0.48, +0.22]	22 [7, 37]	41	37	27
Anticipation	Pitch register	Llama PPLM	+0.97 [+0.52, +1.38]	76 [59, 90]	17	7	29
Anticipation	Pitch register	Llama best-of-N	+0.53 [+0.21, +0.85]	59 [41, 76]	21	21	34
Anticipation	Pitch register	GPT-2 best-of-N	+0.21 [−0.25, +0.64]	46 [29, 64]	29	25	28

Table C.1: Part 2, per tokenization, attribute and system. Signed: mean answer on −2 to +2, positive in the requested direction. Right / Opp. / None: share of answers in the requested direction, in the opposite direction, and "no difference". n: answered questions (one per participant at most).

Tokenizer	Attribute	ρ	n	Moved %	Heard if moved %	Heard if not %
REMI	Duration	+0.07	183	94	65	73 (11)
REMI	Pitch register	+0.52*	169	89	72	32 (19)
REMI	Velocity	+0.40*	177	81	72	18 (34)
Anticipation	Duration	+0.07	110	95	52	60 (5)
Anticipation	Pitch register	+0.22*	118	95	52	50 (6)

Table C.2: Part 2, heard versus measured change. ρ: Spearman correlation between the signed answer and the measured change of the attribute (steered clip minus reference, signed by the requested direction), over answers; * $p < 0.05$, not corrected for clustering by participant. Moved: share of answers whose clip moved in the requested direction. Heard if moved / if not: share of answers in the requested direction, for clips that did and did not move (number of answers in brackets; not given below 5).

Tokenizer	System	Score (CI)	Best %	Worst %	n
REMI	GPT-2 best-of-N	+0.12 [−0.02, +0.26]	33	21	92
REMI	Llama PPLM	+0.12 [−0.02, +0.24]	27	16	121
REMI	Llama tilting	+0.09 [−0.06, +0.25]	30	21	90
REMI	Llama best-of-N	−0.04 [−0.21, +0.12]	22	27	67
REMI	Tonic	−0.06 [−0.16, +0.06]	23	29	175
REMI	GPT-2 tilting	−0.16 [−0.28, −0.04]	24	39	127
Anticipation	GPT-2 best-of-N	+0.15 [−0.01, +0.31]	40	25	99
Anticipation	Llama best-of-N	+0.04 [−0.11, +0.20]	35	31	99
Anticipation	Tonic	+0.03 [−0.11, +0.16]	35	32	120
Anticipation	Llama PPLM	−0.32 [−0.47, −0.17]	9	41	68

Table C.3: Part 3, steered versions of different systems compared: best–worst score, share of appearances chosen as most and as least musical, and number of appearances.

Measure	None (23)	Some (32)	Extensive (10)	Ext. − None (CI)
Cacophonous, all clips	8	11	13	+6 [−1, +13]
Cacophonous, human originals	14	14	16	+2 [−11, +16]
Cacophonous, unguided	4	7	11	+7 [+1, +16]
Cacophonous, steered	8	12	12	+4 [−3, +10]
Cacophonous, broken catch clip	57	75	89	+32 [0, +57]
Broken clip picked as least musical	72	95	86	+13 [−23, +44]
Identical pair: “no difference”	86	92	100	+14 [0, +36]
Part 2: requested direction	59	63	59	0 [−13, +11]
Part 1: REMI original score	+0.11	+0.13	+0.37	+0.26 [−0.06, +0.59]
Part 1: REMI Tonic score	+0.18	+0.24	+0.07	−0.11 [−0.35, +0.16]

Table C.4: Answers by self-reported musical training (number of participants in brackets; one participant did not answer). Shares in %, scores on −1 to +1; the last column is the difference between extensive and no training, with a bootstrap interval resampling each group’s participants. No difference is reliable once the number of comparisons (24 in the full analysis) is taken into account.

	Main	Strict exclusion	Without 686e30f1
Participants	66	59	65
Part 1 REMI, Human original	+0.17	+0.18	+0.17
Part 1 REMI, Tonic	+0.19	+0.19	+0.18
Part 1 REMI, Llama	−0.25	−0.29	−0.25
Part 1 REMI, GPT-2	−0.11	−0.08	−0.10
Part 1 Anticipation, Human original	−0.06	−0.09	−0.06
Part 1 Anticipation, Tonic	+0.01	+0.00	+0.01
Part 1 Anticipation, Llama	+0.15	+0.15	+0.15
Part 1 Anticipation, GPT-2	−0.11	−0.06	−0.10
Part 2 right direction, all	61	62	61
Part 2 right direction, Tonic	57	58	57
Part 2 right direction, AR	62	63	62

Table C.5: Key results under three participant sets: all 66 participants (main analysis); without the 7 who did not choose the near-random catch clip as least musical on a valid check set (strict); and without participant 686e30f1, who ticked exactly one version as cacophonous in every question. Part 1: best–worst score; Part 2: % of answers in the requested direction.

Appendix D

The Tonic Website

Alongside the thesis we built a website that lets anyone listen to the models, browse every generated clip, try the models live and see what the listening study found. Its front page, `https://rebeccaceccatelli.github.io/tonic/`, links to all of it. Table D.1 lists the pages; the sections below show each one with a screenshot and a short description of what it does. All pages except the demo are static sites hosted on GitHub Pages, and all audio is synthesised from the generated MIDI.

Table D.1: The pages of the Tonic website.

Page	What it is for	Section
Front page	entry point, links to everything	D.1
Interactive Demo	generating and steering music live	D.2
Showcase	a curated first listen	D.3
Listening Archive	every clip of the final experiments	D.4
Tonic, simply explained	the thesis for non-specialists	D.5
Listening survey	the blind listening study	D.6
Survey results	the study's findings, for participants	D.7

D.1 Front Page

The front page (Fig. D.1) introduces Tonic in a few lines and links to the other pages: the showcase, the demo, the archive and the survey under *Listen & play*, and the simple explanation, the thesis, the code and the survey results under *Read & build*.

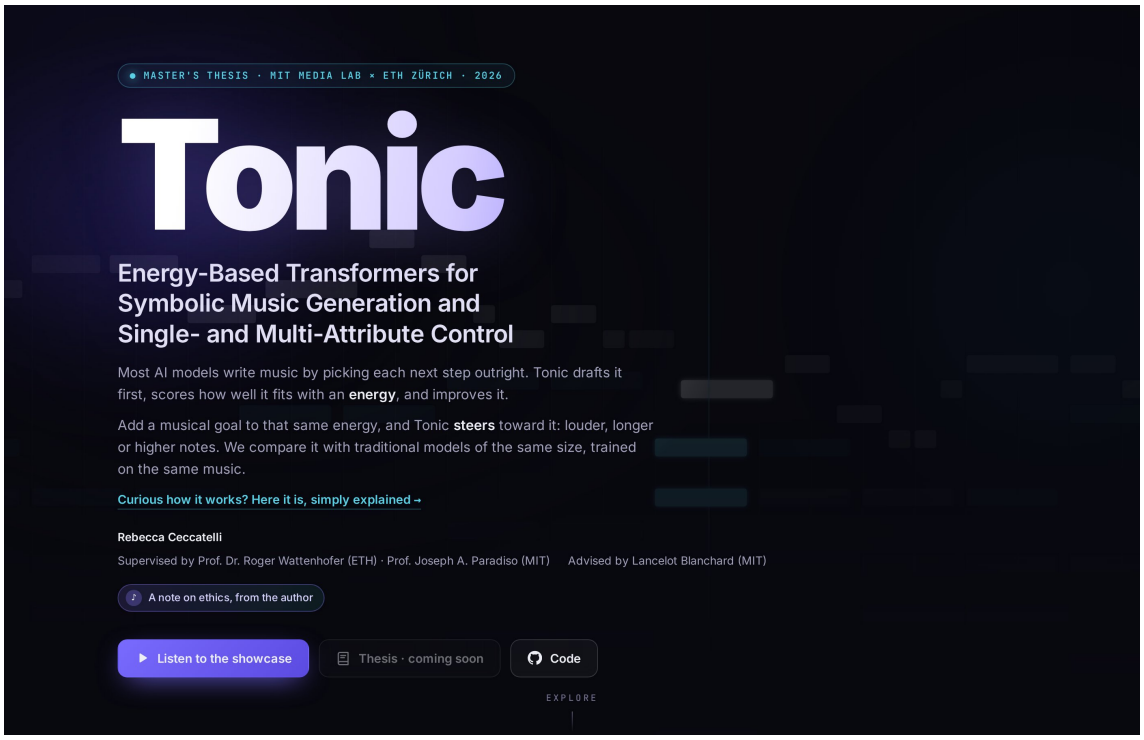


Figure D.1: The front page of the website.

D.2 Interactive Demo

The interactive demo generates new music live, with the same models and steering methods as the thesis. It has five steps:

1. *Choose format and models* (Fig. D.2): the tokenization and which of TONIC, Llama and GPT-2 to compare, each with its training steps and validation loss;
2. *Choose your prompt*: one of 202 songs, and the point where the prompt starts, picked on the song's waveform;
3. *Listen to the unguided generations* of every model, next to the original continuation;
4. *Single-attribute control* (Fig. D.3): up to six *variants*, each with an attribute, a model and steering method (TONIC's energy guidance, or PPLM, tilting or best-of-N on the baselines), a target value and a strength, generated together and played back as aligned piano rolls;
5. *Combined-attribute control* (Fig. D.4): two attributes steered at once, as in the thesis experiments, choosing the directions (for example forte and legato), how far to push, and which systems to compare.

Targets are shown relative to the training music, in corpus standard deviations, and start from each model's own unguided continuation. The demo needs a GPU and is therefore not hosted permanently: it is started from the code repository

(`demo/launch_demo.sh`), which can also open a temporary public link that the front page shows while it is running.

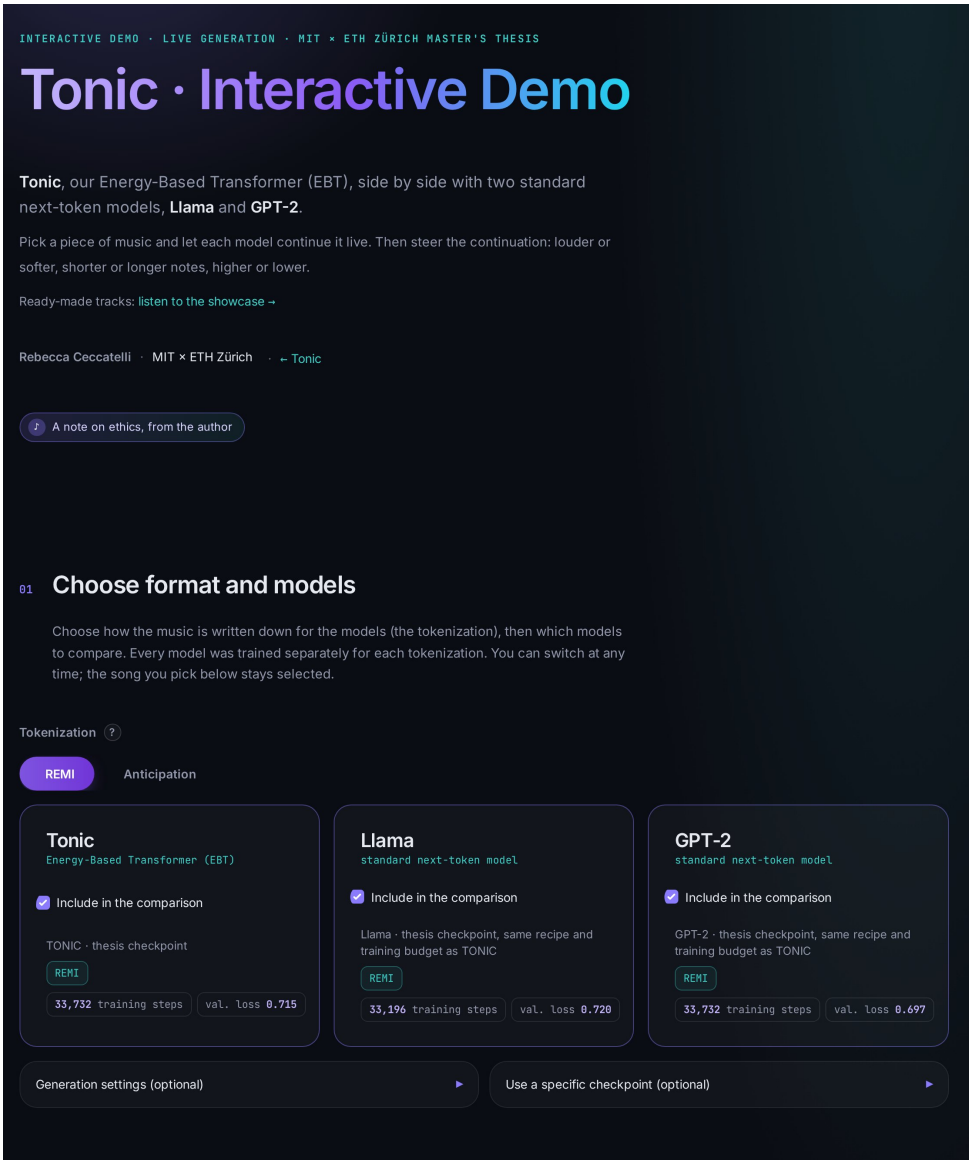


Figure D.2: The top of the interactive demo: choosing the tokenization and the models to compare.

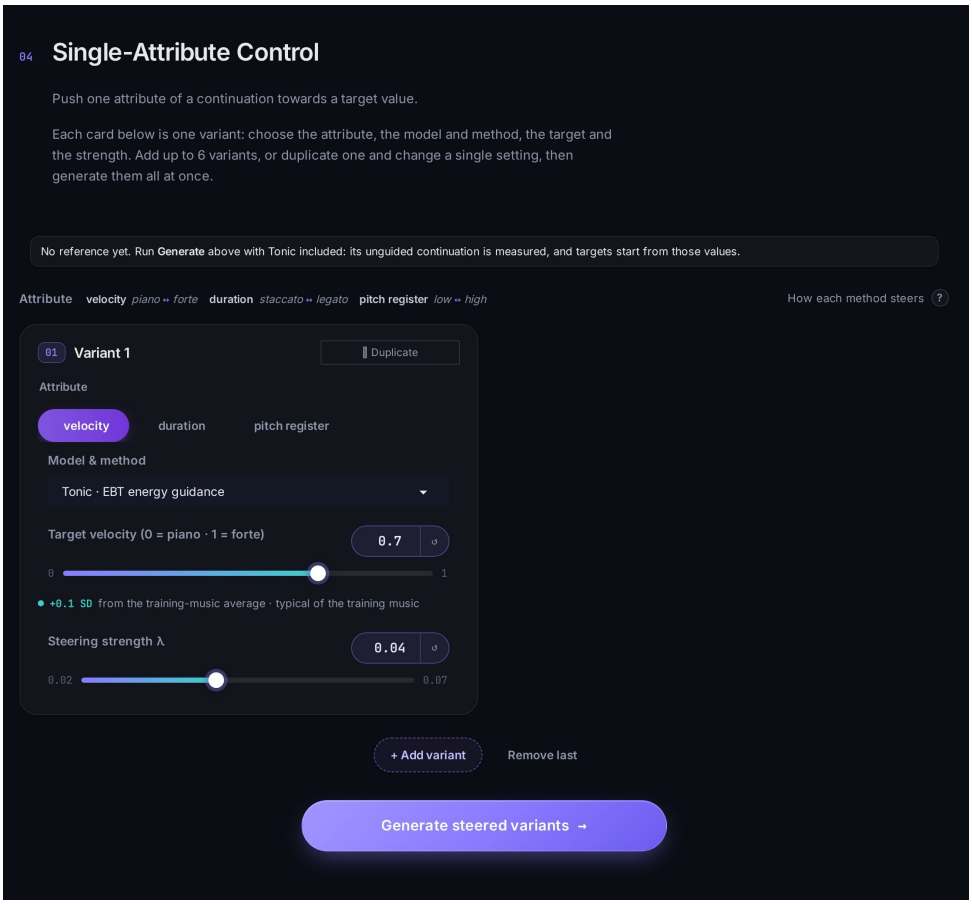


Figure D.3: Single-attribute control in the demo: one variant, with the attribute, model and steering method, target value and strength λ.

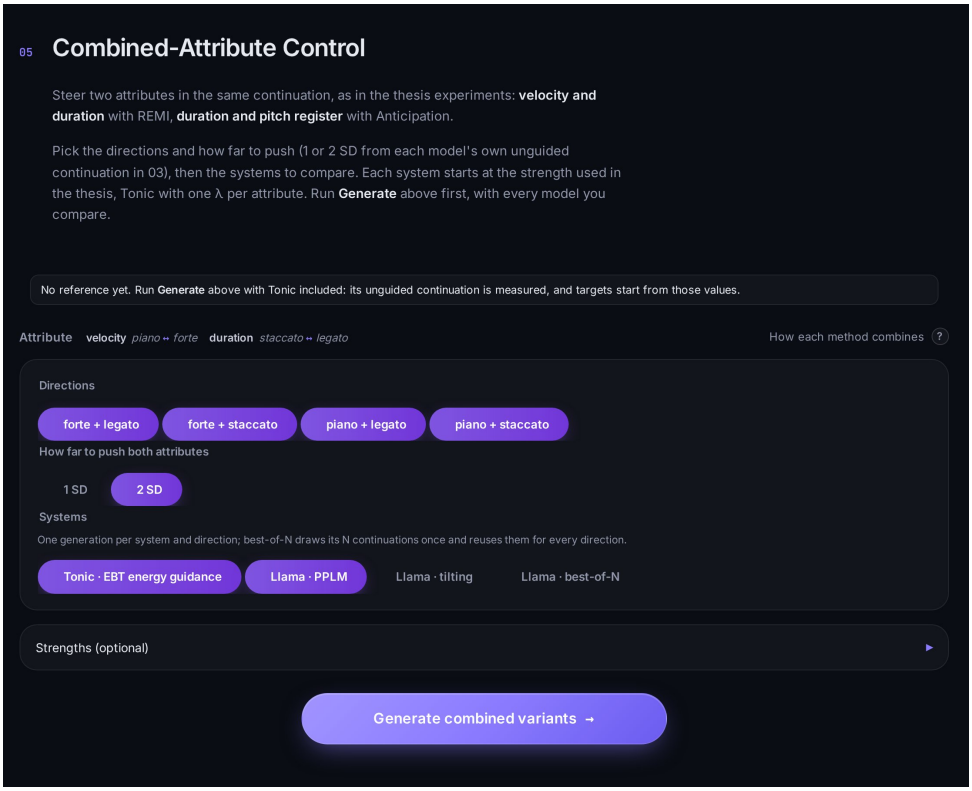


Figure D.4: Combined-attribute control in the demo: directions for the two attributes, how far to push, and the systems to compare.

D.3 Showcase

The showcase[1] is a curated selection for a first listen (Fig. D.5). It has four sections:

- *Unguided generation:* the same prompt continued by the original piece, Llama, GPT-2 and TONIC;
- *Single-attribute control:* one attribute steered up or down;
- *Combined control:* two attributes steered at once;
- *Pushing too hard:* what happens when the guidance strength is too high (Section 5.6).

REMI and Anticipation examples are on separate tabs, and every clip is shown as a piano roll with the prompt shaded. In the control sections (Fig. D.6), each card is one song: TONIC's unguided continuation sits in the middle as the reference, steered versions are on the left (less of the attribute) and on the right (more), and each row is one steering method. The listener chooses how much change to ask for and how hard to push. All examples were selected by ear, so the showcase illustrates the systems but is not evidence for the results; the listening archive is.

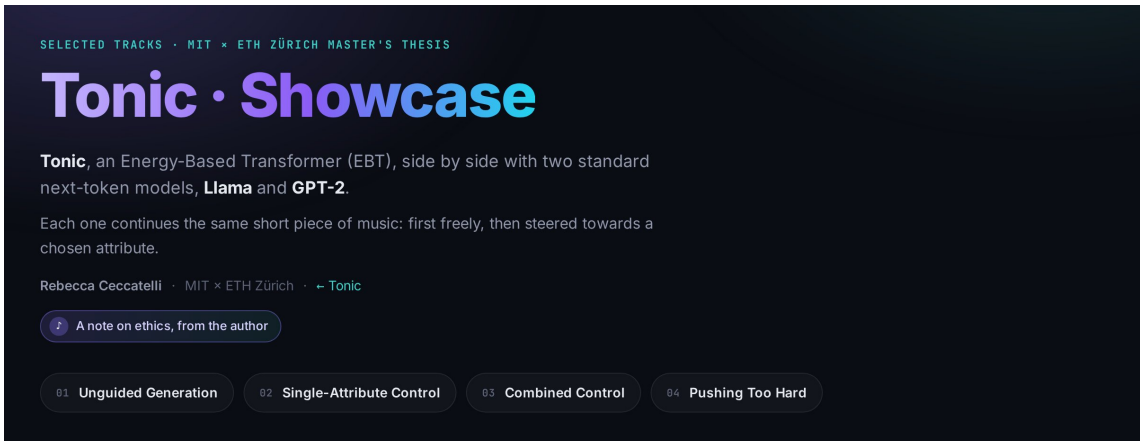


Figure D.5: The top of the showcase, with its four sections.

[1] https://rebeccaceccatelli.github.io/music_EBT/

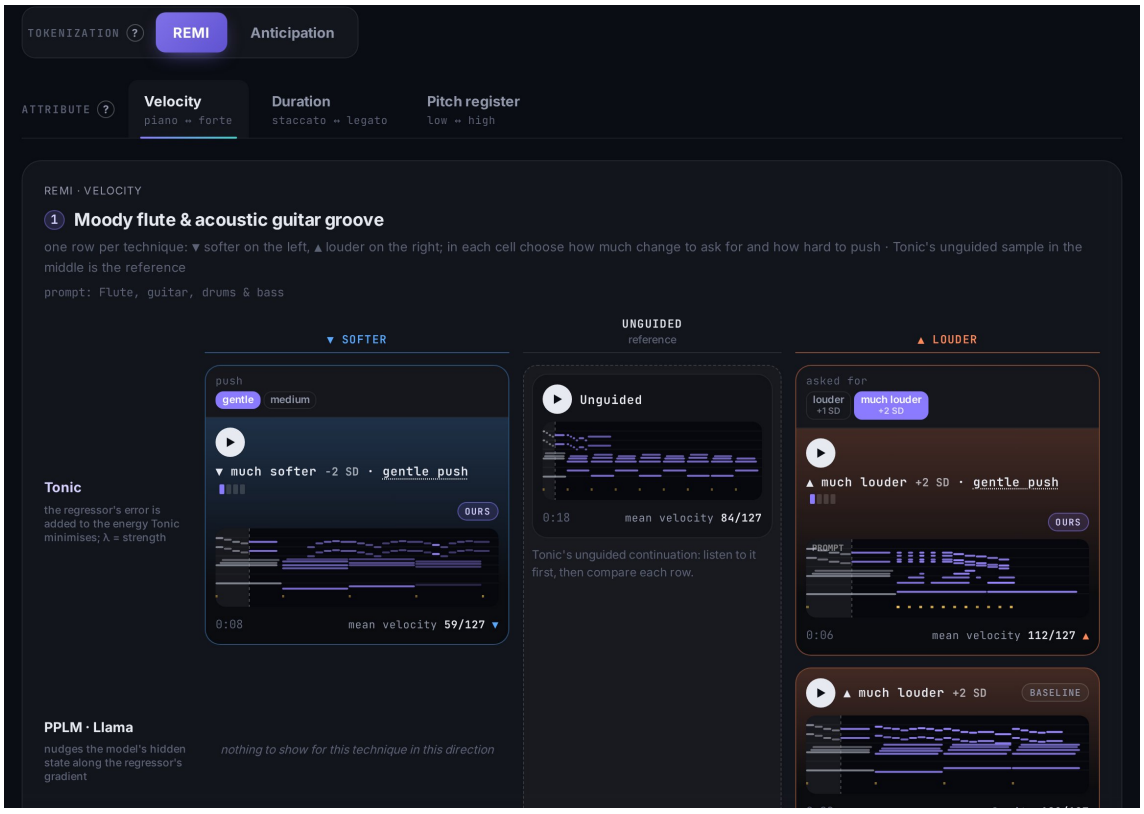


Figure D.6: A single-attribute card of the showcase (REMI, velocity): TONIC's unguided continuation in the middle, softer versions on the left and louder ones on the right, one row per steering method.

D.4 Listening Archive

The listening archive[2] contains every output of the final experiments, 20,441 clips, so that any number in Chapter 5 can be checked by ear. Its front page (Fig. D.7) leads to five pages:

- *unguided* continuations of all systems and the original pieces (2,336 clips);
- *single-attribute control* on REMI (6,144) and on Anticipation (4,286);
- *combined control* on REMI (4,477) and on Anticipation (3,198).

On the control pages (Fig. D.8), each row is one system and steering method at one strength, starting with its unguided reference, and each column is a target, from −2 to +2 corpus standard deviations. Under each clip, a small scale shows how far the attribute moved from the system's unguided level and whether it moved in the requested direction (green) or not (amber), next to the share of harsh intervals. Badges mark the operating points used in Chapter 5 and the strengths chosen by ear. The page can be filtered by attribute and by prompt set (test or tuning prompts), and clips can be saved to a personal list.

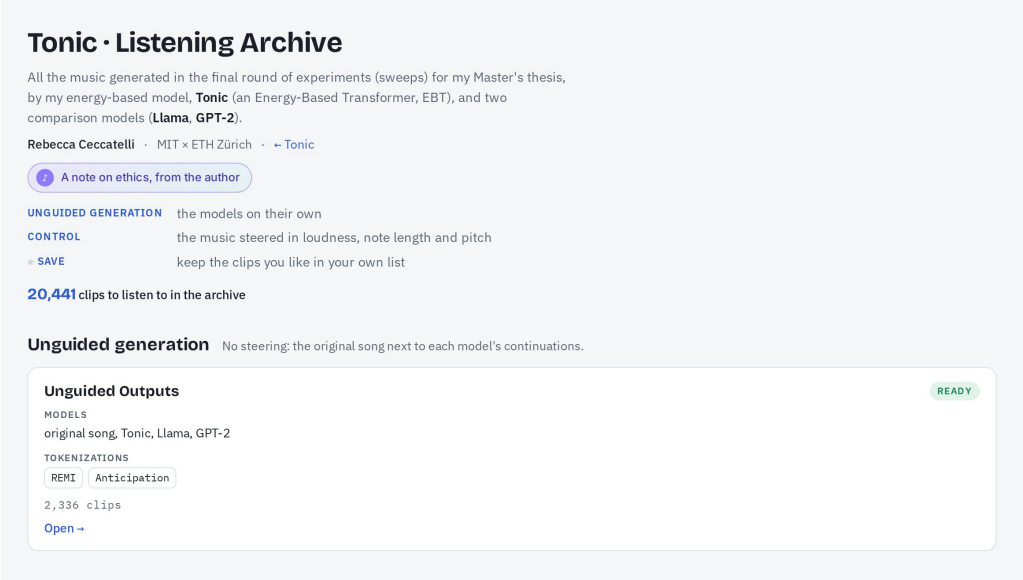


Figure D.7: The front page of the listening archive.

[2] https://rebeccaceccatelli.github.io/listening-archive/

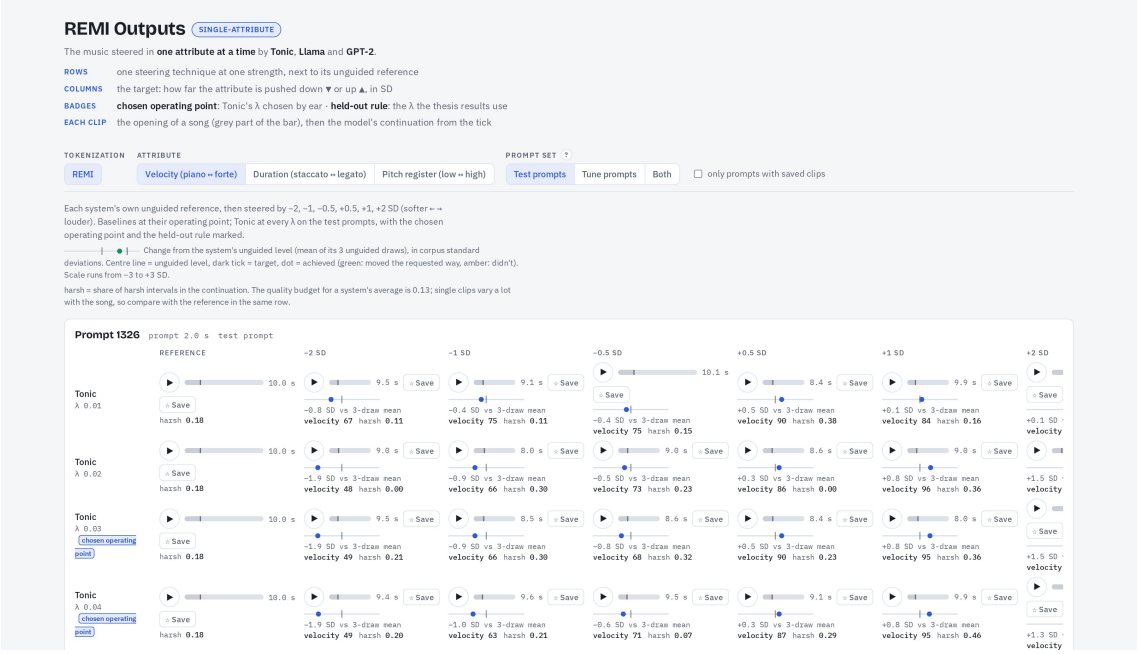


Figure D.8: The single-attribute REMI page of the archive, for velocity. Rows are systems and strengths, columns are targets in corpus standard deviations; under each clip, the achieved change and the share of harsh intervals.

D.5 Tonic, Simply Explained

"How Tonic works"[3] (Fig. D.9) explains the thesis without mathematics, in six short chapters of about eight minutes in total: how a computer writes music, what Tonic does differently, and how it can be steered, with short examples to listen to along the way.

[3] https://rebeccaceccatelli.github.io/tonic/how-it-works/

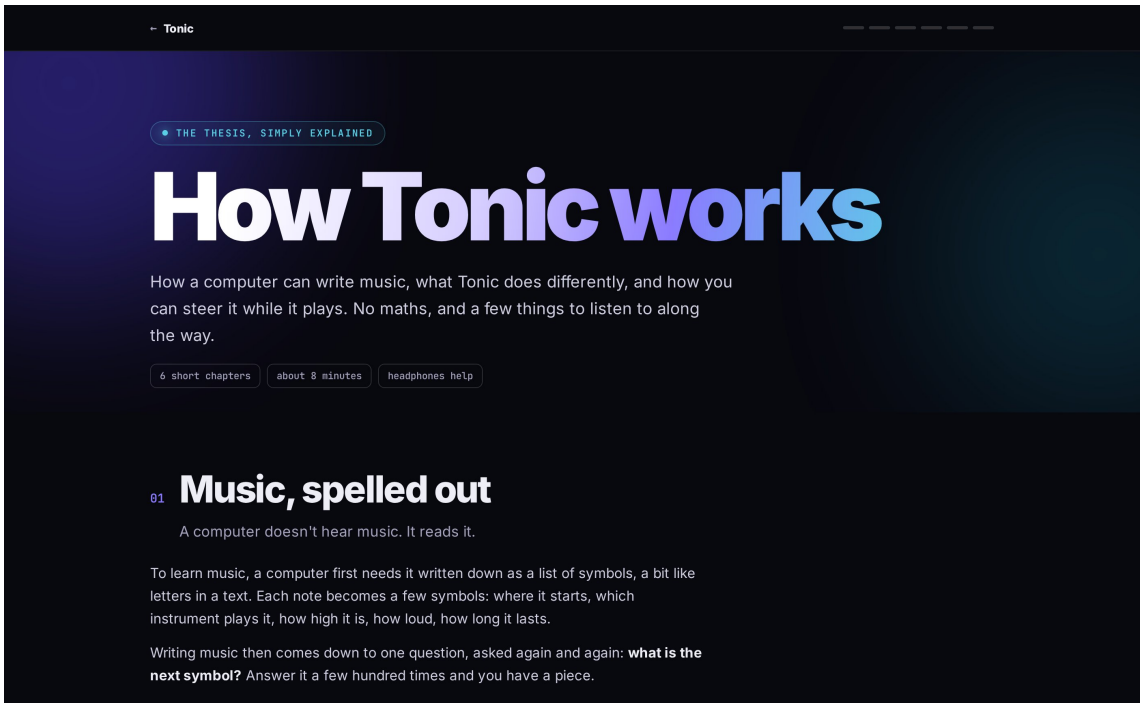


Figure D.9: The opening of "How Tonic works", the thesis explained for non-specialists.

D.6 Listening Survey

The blind listening survey, in English[4] and Italian[5], takes about twenty minutes and needs no musical knowledge (Fig. D.10). Before the questions, a short step explains that each clip starts with a few seconds of a real piece, the prompt, and that only the continuation is judged. Its design is described in Section 4.6 and its results in Chapter 5.

[4] https://rebeccaceccatelli.github.io/listening-study/survey/
[5] https://rebeccaceccatelli.github.io/listening-study/survey/it/

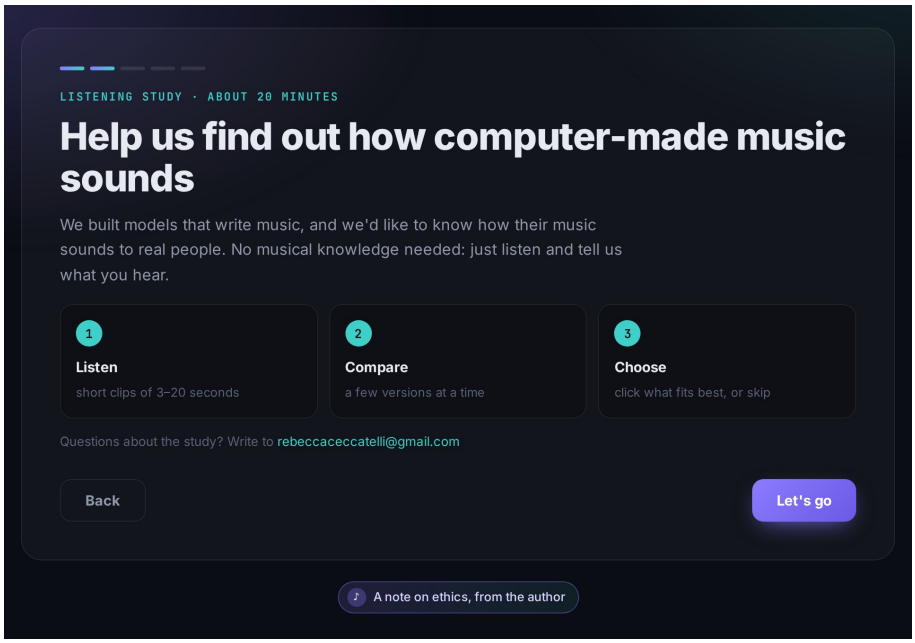


Figure D.10: The welcome step of the listening survey.

D.7 Survey Results

"What you heard"[6] (Fig. D.11) reports the findings of the listening study for participants and other non-specialists: what was in the survey, then one short finding per part, with the same numbers as Section 5.7.

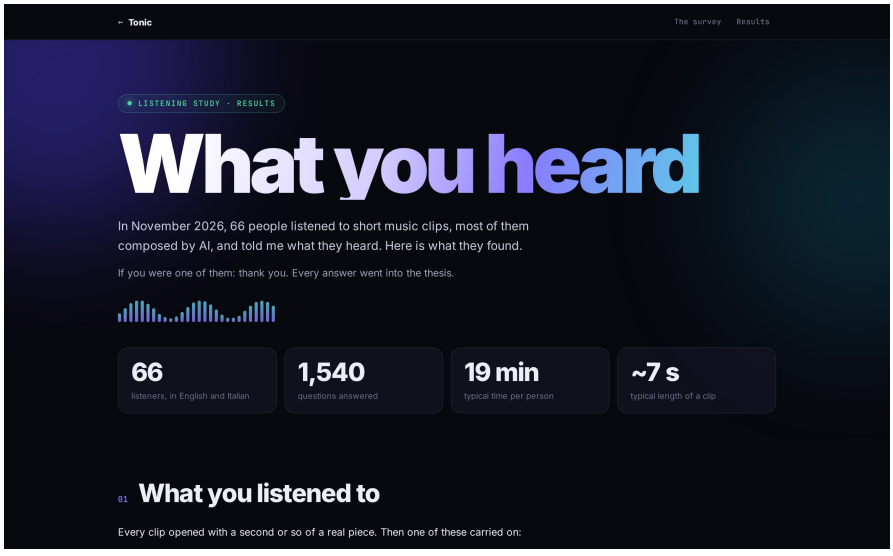


Figure D.11: The opening of the survey results page.

[6] https://rebeccaceccatelli.github.io/tonic/survey-results/

Bibliography

[1] David H. Ackley, Geoffrey E. Hinton, and Terrence J. Sejnowski. "A Learning Algorithm for Boltzmann Machines". In: *Cognitive Science* 9.1 (1985), pp. 147–169. DOI: 10.1207/s15516709cog0901_7.

[2] Andrea Agostinelli et al. "MusicLM: Generating Music From Text". In: *arXiv preprint arXiv:2301.11325* (2023).

[3] Berker Banar, Nick Bryan-Kinns, and Simon Colton. "A Tool for Generating Controllable Variations of Musical Themes Using Variational Autoencoders with Latent Space Regularisation". In: *Proceedings of the AAAI Conference on Artificial Intelligence (Demo Track)*. 2023.

[4] Lancelot Blanchard et al. "The JAM_BOT, a Real-Time System for Collaborative Free Improvisation with Music Language Models". In: *Proceedings of the 26th International Society for Music Information Retrieval Conference (ISMIR)*. 2025.

[5] Nick Bryan-Kinns et al. "Exploring Variational Auto-encoder Architectures, Configurations, and Datasets for Generative Music Explainable AI". In: *Machine Intelligence Research* 21.1 (2024), pp. 29–45.

[6] Jade Copet et al. "Simple and Controllable Music Generation". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023.

[7] Imre Csiszár. "I-Divergence Geometry of Probability Distributions and Minimization Problems". In: *The Annals of Probability* 3.1 (1975), pp. 146–158. DOI: 10.1214/aop/1176996454.

[8] Sumanth Dathathri et al. "Plug and Play Language Models: A Simple Approach to Controlled Text Generation". In: *International Conference on Learning Representations (ICLR)*. 2020.

[9] Alexandre Défossez et al. "High Fidelity Neural Audio Compression". In: *Transactions on Machine Learning Research* (2023).

[10] Prafulla Dhariwal and Alexander Nichol. "Diffusion Models Beat GANs on Image Synthesis". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2021.

[11] Prafulla Dhariwal et al. "Jukebox: A Generative Model for Music". In: *arXiv preprint arXiv:2005.00341* (2020).

[12] Hao-Wen Dong et al. "Multitrack Music Transformer". In: *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2023.

[13] Yilun Du and Igor Mordatch. "Implicit Generation and Modeling with Energy Based Models". In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019, pp. 3608–3618.

[14] Yilun Du et al. “Reduce, Reuse, Recycle: Compositional Generation with Energy-Based Diffusion Models and MCMC”. In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. 2023.

[15] Maral Ebrahimzadeh, Gilberto Bernardes, and Sebastian Stober. “Explicit Tonal Tension Conditioning via Dual-Level Beam Search for Symbolic Music Generation”. In: *Proceedings of the 17th International Symposium on Computer Music Multidisciplinary Research (CMMR)*. arXiv:2511.19342. 2025.

[16] Lucas N. Ferreira et al. “Controlling Perceived Emotion in Symbolic Music Generation with Monte Carlo Tree Search”. In: *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*. 2022.

[17] Nathan Fradet et al. “MidiTok: A Python Package for MIDI File Tokenization”. In: *Extended Abstracts for the Late-Breaking Demo Session of the 22nd International Society for Music Information Retrieval Conference (ISMIR)*. 2021.

[18] Luca Giuliani et al. “MusiComb: a Sample-based Approach to Music Generation Through Constraints”. In: *IEEE 35th International Conference on Tools with Artificial Intelligence (ICTAI)*. 2023.

[19] Alexi Gladstone et al. “Energy-Based Transformers are Scalable Learners and Thinkers”. In: *arXiv preprint arXiv:2507.02092* (2025).

[20] Geoffrey E. Hinton. “Training Products of Experts by Minimizing Contrastive Divergence”. In: *Neural Computation* 14.8 (2002), pp. 1771–1800. DOI: 10.1162/089976602760128018.

[21] Jonathan Ho and Tim Salimans. “Classifier-Free Diffusion Guidance”. In: *arXiv preprint arXiv:2207.12598* (2022).

[22] Benjamin Hoover et al. “Energy Transformer”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023.

[23] Benjamin Hoover et al. “Memory in Plain Sight: A Survey of the Uncanny Resemblances between Diffusion Models and Associative Memories”. In: *arXiv preprint arXiv:2309.16750* (2023).

[24] John J. Hopfield. “Neural Networks and Physical Systems with Emergent Collective Computational Abilities”. In: *Proceedings of the National Academy of Sciences* 79.8 (1982), pp. 2554–2558. DOI: 10.1073/pnas.79.8.2554.

[25] Wen-Yi Hsiao et al. “Compound Word Transformer: Learning to Compose Full-Song Music over Dynamic Directed Hypergraphs”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2021.

[26] Cheng-Zhi Anna Huang et al. “Music Transformer: Generating Music with Long-Term Structure”. In: *International Conference on Learning Representations (ICLR)*. 2019.

[27] Yu-Siang Huang and Yi-Hsuan Yang. “Pop Music Transformer: Beat-based Modeling and Generation of Expressive Pop Piano Compositions”. In: *Proceedings of the 28th ACM International Conference on Multimedia*. 2020.

[28] Yujia Huang et al. "Symbolic Music Generation with Non-Differentiable Rule Guided Diffusion". In: *Proceedings of the 41st International Conference on Machine Learning (ICML)*. arXiv:2402.14285. 2024.

[29] Hsiao-Tzu Hung et al. "EMOPIA: A Multi-Modal Pop Piano Dataset for Emotion Recognition and Emotion-Based Music Generation". In: *Proceedings of the 22nd International Society for Music Information Retrieval Conference (ISMIR)*. 2021.

[30] Naomi Imasato et al. "Using a Language Model to Generate Music in Its Symbolic Domain While Controlling Its Perceived Emotion". In: *IEEE Access* 11 (2023), pp. 52412–52428. DOI: 10.1109/ACCESS.2023.3280603.

[31] Maximos Kaliakatsos-Papakostas, Aggelos Gkiokas, and Vassilis Katsouros. "Interactive Control of Explicit Musical Features in Generative LSTM-based Systems". In: *Proceedings of Audio Mostly (AM'18)*. 2018.

[32] Muhammad Khalifa, Hady Elsahar, and Marc Dymetman. "A Distributional Approach to Controlled Text Generation". In: *International Conference on Learning Representations (ICLR)*. 2021.

[33] Junghyun Koo et al. "SMITIN: Self-Monitored Inference-Time INtervention for Generative Music Transformers". In: *IEEE Open Journal of Signal Processing* (2025).

[34] Ben Krause et al. "GeDi: Generative Discriminator Guided Sequence Generation". In: *Findings of the Association for Computational Linguistics: EMNLP*. 2021.

[35] Yann LeCun et al. "A Tutorial on Energy-Based Learning". In: *Predicting Structured Data*. Ed. by Gökhan Bakır et al. MIT Press, 2006.

[36] Keon Ju Maverick Lee et al. "The GigaMIDI Dataset with Features for Expressive Music Performance Detection". In: *Transactions of the International Society for Music Information Retrieval* (2025).

[37] Yingru Liu, Dongliang Xie, and Xin Wang. "Energy-Based Recurrent Model for Stochastic Modeling of Music". In: *Proceedings of the IEEE International Conference on Multimedia and Expo (ICME)*. 2019, pp. 236–241.

[38] Yinghao Ma et al. "Foundation Models for Music: A Survey". In: *arXiv preprint arXiv:2408.14340* (2024).

[39] Martin E. Malandro. "Composer's Assistant 2: Interactive Multi-Track MIDI Infilling with Fine-Grained User Control". In: *Proceedings of the 25th International Society for Music Information Retrieval Conference (ISMIR)*. 2024.

[40] Gautam Mittal et al. "Symbolic Music Generation with Diffusion Models". In: *Proceedings of the 22nd International Society for Music Information Retrieval Conference (ISMIR)*. 2021.

[41] Fabio Morreale et al. "Reductive, Exclusionary, Normalising: The Limits of Generative AI Music". In: *Transactions of the International Society for Music Information Retrieval* 8.1 (2025).

[42] Alexandru Oarga and Yilun Du. "Generalizable Reasoning through Compositional Energy Minimization". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2025.

[43] Sageev Oore et al. "This Time with Feeling: Learning Expressive Musical Performance". In: *Neural Computing and Applications* 32 (2020), pp. 955–967.

[44] Longshen Ou and Ye Wang. "PhraseVAE and PhraseLDM: Latent Diffusion for Full-Song Multitrack Symbolic Music Generation". In: *arXiv preprint arXiv:2512.11348* (2025).

[45] Philippe Pasquier, Jeff Ens, et al. "MIDI-GPT: A Controllable Generative Model for Computer-Assisted Multitrack Music Composition". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2025.

[46] Ashis Pati and Alexander Lerch. "Latent Space Regularization for Explicit Control of Musical Attributes". In: *ICML Workshop on Machine Learning for Music Discovery*. 2019.

[47] Ashis Pati and Alexander Lerch. "Attribute-based Regularization of Latent Spaces for Variational Auto-Encoders". In: *Neural Computing and Applications* (2021).

[48] Ashis Pati and Alexander Lerch. "Is Disentanglement Enough? On Latent Representations for Controllable Music Generation". In: *Proceedings of the 22nd International Society for Music Information Retrieval Conference (ISMIR)*. 2021.

[49] Bao Pham et al. "Memorization to Generalization: Emergence of Diffusion Models from Associative Memory". In: *arXiv preprint arXiv:2505.21777* (2025).

[50] Matthias Plasser, Silvan Peter, and Gerhard Widmer. "Discrete Diffusion Probabilistic Models for Symbolic Music Generation". In: *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI)*. 2023, pp. 5842–5850.

[51] Ioannis Prokopiou et al. "Closing the Loop: PID Feedback Control for Interpretable Activation Steering in Symbolic Music Generation". In: *Proceedings of the 43rd International Conference on Machine Learning (ICML)*. 2026.

[52] Ioannis Prokopiou et al. "Latent Space Disentanglement via Activation Steering for Interpretable Attribute Control in Symbolic Music Generation". In: *arXiv preprint arXiv:2605.31295* (2026).

[53] Yikai Qian et al. "MusicAOG: an Energy-Based Model for Learning and Sampling a Hierarchical Representation of Symbolic Music". In: *arXiv preprint arXiv:2401.02678* (2024).

[54] Alec Radford et al. *Language Models are Unsupervised Multitask Learners*. Tech. rep. OpenAI, 2019.

[55] Colin Raffel. "Learning-Based Methods for Comparing Sequences, with Applications to Audio-to-MIDI Alignment and Matching". PhD thesis. Columbia University, 2016.

[56] Hubert Ramsauer et al. "Hopfield Networks is All You Need". In: *International Conference on Learning Representations (ICLR)*. 2021.

[57] Adam Roberts et al. "A Hierarchical Latent Vector Model for Learning Long-Term Structure in Music". In: *Proceedings of the 35th International Conference on Machine Learning (ICML)*. 2018.

[58] Kevin Rojas et al. "Improving Classifier-Free Guidance in Masked Diffusion: Low-Dim Theoretical Insights with High-Dim Impact". In: *arXiv preprint arXiv:2507.08965* (2025).

[59] Abhinaba Roy, Geeta Puri, and Dorien Herremans. "Text2midi-InferAlign: Improving Symbolic Music Generation with Inference-Time Alignment". In: *arXiv preprint arXiv:2505.12669* (2025).

[60] Dimitri von Rütte et al. "FIGARO: Controllable Music Generation using Learned and Expert Features". In: *International Conference on Learning Representations (ICLR)*. 2023.

[61] Alexander Scarlatos et al. "ReaLJam: Real-Time Human-AI Music Jamming with Reinforcement Learning-Tuned Transformers". In: *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA)*. 2025.

[62] Adi Silberschein, Megan Wei, and Tamar Rott Shaham. "Harmonic Representations in Symbolic Music Transformers are Localized and Controllable". In: *arXiv preprint arXiv:2610.15957* (2026).

[63] Nikhil Singh, Manuel Cherep, and Pattie Maes. "Discovering and Steering Interpretable Concepts in Large Generative Music Models". In: *International Conference on Learning Representations (ICLR)*. 2026.

[64] Damien Sprockeels and Peter Van Roy. "Expressing Musical Ideas with Constraint Programming Using a Model of Tonal Harmony". In: *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI)*. 2024.

[65] Hongju Su et al. "Amadeus: Autoregressive Model with Bidirectional Attribute Modelling for Symbolic Music". In: *arXiv preprint arXiv:2508.20665* (2025).

[66] Hao Hao Tan and Dorien Herremans. "Music FaderNets: Controllable Music Generation Based on High-Level Features via Low-Level Feature Modelling". In: *Proceedings of the 21st International Society for Music Information Retrieval Conference (ISMIR)*. 2020.

[67] John Thickstun et al. "Anticipatory Music Transformer". In: *Transactions on Machine Learning Research* (2024).

[68] Hugo Touvron et al. "LLaMA: Open and Efficient Foundation Language Models". In: *arXiv preprint arXiv:2302.13971* (2023).

[69] Alexander Matt Turner et al. "Activation Addition: Steering Language Models Without Optimization". In: *arXiv preprint arXiv:2308.10248* (2023).

[70] Ziyu Wang, Lejun Min, and Gus Xia. "Whole-Song Hierarchical Generation of Symbolic Music Using Cascaded Diffusion Models". In: *International Conference on Learning Representations (ICLR)*. 2024.

[71] Shih-Lun Wu et al. "MIDI-LLM: Improving Text-to-MIDI Music Generation via Adapting Large Language Models". In: *arXiv preprint arXiv:2511.03942* (2025).

[72] Shih-Lun Wu and Yi-Hsuan Yang. "MuseMorphose: Full-Song and Fine-Grained Piano Music Style Transfer with One Transformer VAE". In: *IEEE/ACM Transactions on Audio, Speech, and Language Processing* (2023).

[73] Minkai Xu et al. "Energy-Based Diffusion Language Models for Text Generation". In: *International Conference on Learning Representations (ICLR)*. 2025.

[74] Li-Chia Yang and Alexander Lerch. "On the Evaluation of Generative Models in Music". In: *Neural Computing and Applications* 32 (2020), pp. 4773–4784.

[75] Kevin Yang and Dan Klein. "FUDGE: Controlled Text Generation With Future Discriminators". In: *Proceedings of NAACL-HLT*. 2021.

[76] Ruihan Yang et al. "Deep Music Analogy via Latent Representation Disentanglement". In: *Proceedings of the 20th International Society for Music Information Retrieval Conference (ISMIR)*. 2019.

[77] Jianrong Zhang, Hehe Fan, and Yi Yang. "EnergyMoGen: Compositional Human Motion Generation with Energy-Based Diffusion Model in Latent Space". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2025.

[78] Mingzhe Zhang et al. "Emotionally Guided Symbolic Music Generation Using Diffusion Models: The AGE-DM Approach". In: *Proceedings of the 6th ACM International Conference on Multimedia in Asia (MMAsia)*. 2024. DOI: 10.1145/3696409.3700289.

[79] Tingyu Zhu et al. "Efficient Fine-Grained Guidance for Diffusion Model Based Symbolic Music Generation". In: *Proceedings of the 42nd International Conference on Machine Learning (ICML)*. 2025.